

LAPORAN PROYEK AKHIR

RANCANG BANGUN SISTEM MONITORING MACHINE LEARNING DENGAN

METODE REPLAY PADA STUDI KASUS SAMPAH PLASTIK



Disusun oleh:

Alief Rahman Hakim

20/457255/SV/17702

PROGRAM STUDI D-4 TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2024

JUDUL PROYEK AKHIR
RANCANG BANGUN SISTEM MONITORING MACHINE LEARNING
DENGAN METODE REPLAY PADA STUDI KASUS SAMPAH PLASTIK

Proyek akhir

Program Studi Teknologi Rekayasa Perangkat Lunak

Diajukan sebagai syarat kelengkapan studi jenjang Sarjana Terapan untuk
memperoleh gelar Sarjana Terapan Komputer (S. Tr. Kom) pada Program Studi
Teknologi Rekayasa Perangkat Lunak

Oleh:

ALIEF RAHMAN HAKIM

20/457255/SV/17702

PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2024

HALAMAN PENGESAHAN

HALAMAN PERNYATAAN

KATA PENGANTAR

Segala puji bagi Allah Yang Maha Esa atas segala nikmat, karunia, dan perlindungan-Nya sehingga dapat menyelesaikan proyek akhir ini dengan baik. Sholawat serta salam senantiasa

Proyek Akhir ini disusun untuk melengkapi salah satu syarat mencapai gelar Sarjana Terapan Program Studi Teknologi Rekayasa Perangkat Lunak, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada. Dengan selesainya Proyek Akhir ini, Penulis mengucapkan terima kasih kepada :

1. Nur Rohman Rosyid, S.T., M.T., D.Eng., selaku Ketua Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada.
2. Dr. Umar Taufiq, S.Kom., M.Cs., selaku Ketua Program Studi Teknologi Rekayasa Perangkat Lunak, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada.
3. Dr. Umar Taufiq, S.Kom., M.Cs., selaku Dosen Pembimbing, yang telah banyak membimbing.
4. Dosen Penguji yang telah memberikan saya pengetahuan dan kemudahan.
5. Dinar Nugroho Pratomo, S.Kom., M.IM., M.Cs., selaku Dosen Pembimbing Akademik yang telah banyak membantu dalam akademik.
6. Kedua orangtua saya yang telah banyak memberikan dukungan terbesar yaitu berupa uang. Diri saya sendiri yang telah banyak berjuang hingga titik ini.

DAFTAR ISI

JUDUL PROYEK AKHIR.....	i
HALAMAN PENGESAHAN.....	ii
HALAMAN PERNYATAAN	iii
KATA PENGANTAR.....	iv
DAFTAR ISI	v
DAFTAR GAMBAR	ix
DAFTAR TABEL.....	xii
DAFTAR SIMBOL	xiii
INTISARI.....	xiv
ABSTRACT.....	xv
BAB I PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Rumusan Masalah	3
1.3. Batasan Masalah.....	4
1.4. Tujuan Proyek Akhir	5
1.5. Manfaat Proyek Akhir	5
1.6. Sistematika Penulisan.....	6
BAB II TINJAUAN PUSTAKA	8
2.1. Studi Pustaka	8
2.2. Dasar Teori	15
2.2.1. Label Resin Plastik	16
2.2.2. Machine Learning.....	17
2.2.3. Memory-Based Learning.....	17
2.2.4. Replay	18

2.2.5. Domain-incremental learning	19
2.2.6. Continual Learning	20
2.2.7. Model YOLO dan Ultralytics	21
2.2.8. Dataset	22
2.2.9. Frontend.....	23
2.2.10. Backend	24
2.2.11. Basis Data	25
2.2.12. Metrik Evaluasi.....	26
2.3. Hipotesis.....	28
BAB III METODE PROYEK AKHIR	30
3.1. Alat dan Bahan	30
3.1.1. Bahan	30
3.1.2. Peralatan	30
3.2. Tahapan Proyek Akhir	32
3.2.1. Pra-penelitian.....	33
3.2.2. Pengambilan Data dan Uji Model Baseline.....	35
3.2.4. Prediksi dan Koreksi Data	35
3.2.5. Data <i>Batch</i>	35
3.2.6. Pemilihan dataset dan Pelatihan Model.....	36
3.2.7. Testing dan Evaluasi Metrik	36
3.3. Usulan Rancangan Metode Replay	36
3.3.1. Data survei dan Distribusi data.....	37
3.3.2. Tahap prediksi dan Batch.....	39
3.3.3. Pemilihan dataset dan Pelatihan	43
3.3.4. Pengujian Model.....	45

3.4.	Perancangan Aplikasi Web	49
3.4.2.	Wireframe Antarmuka Pengguna.....	49
3.4.3.	Arsitektur Sistem	54
3.4.5.	Use Case Diagram	55
3.4.6.	Activity Diagram	56
BAB IV HASIL DAN PEMBAHASAN.....		61
4.1.	Persiapan Data	61
4.2.	Analisis dan Implementasi Sumber Kode Ultralytics	61
4.2.1.	Kode Pengujian dan Metriks dengan Model Baseline.....	62
4.2.2.	Implementasi Modifikasi Fungsi Pelatihan	65
4.3.	Hasil Implementasi Backend.....	68
4.3.1.	Inference dan History Prediction	68
4.3.2.	Riwayat Prediksi dan Menambah Prediksi ke Batch.....	70
4.3.3.	Membuat Replay Buffer (Batch)	71
4.3.4.	Pelatihan Model	85
4.3.5.	Pengujian Model.....	89
4.4.	Hasil Implementasi Frontend	91
4.4.1.	Antarmuka Prediksi dan Kode.....	91
4.4.2.	Antarmuka Riwayat Prediksi.....	92
4.4.3.	Antarmuka Batch	93
4.4.4.	Antarmuka Pelatihan	94
4.5.	Hasil Pengujian dan Evaluasi Model.....	95
4.5.1.	Evaluasi Model Baseline dan Dataset Baseline.....	95
4.5.2.	Batch 1	98
4.5.3.	Batch 2	104

4.6. User Acceptance Testing	Error! Bookmark not defined.
BAB V KESIMPULAN DAN SARAN.....	107
5.1. Kesimpulan.....	107
5.2. Saran.....	107
DAFTAR PUSTAKA	108

DAFTAR GAMBAR

Gambar 2.1 Tiga jenis pendekatan berbasis memori [5]	18
Gambar 2.2 Proses mengulang atau replay pada pelatihan model	19
Gambar 2.3 Tiga perbedaan <i>Incremental-Learning</i>	20
Gambar 2.4 Keseimbangan stabilisasi dan <i>plastic</i> (fleksibilitas) pada CL .	21
Gambar 3.1 Kumpulan gambar sampah plastik.....	30
Gambar 3.2 Tahapan Proyek Akhir.....	33
Gambar 3.3 Data survei	37
Gambar 3.4 Sampel gambar dari basis data WaDaBa	38
Gambar 3.5 Distribusi Data	39
Gambar 3.6 Flowchart proses prediksi hingga kumpulan koreksi	40
Gambar 3.7 <i>Generate</i> prediksi di <i>batch</i> menjadi dataset.....	41
Gambar 3.8 Kasus penggunaan replay yang.....	43
Gambar 3.9 Visualisasi penggunaan random sapling replay	Error!
Bookmark not defined.	
Gambar 3.10 Contoh modifikasi fungsi <i>training</i>	45
Gambar 3.11 Skenario testing.....	46
Gambar 3.12 Halaman utama	51
Gambar 3.13 Tampilan modal tinjauan prediksi.....	51
Gambar 3.14 Halaman daftar <i>batch</i>	52
Gambar 3.15 Halaman detail batch	52
Gambar 3.16 Tampilan daftar dataset	53
Gambar 3.17 Tampilan daftar evaluasi model	53
Gambar 3.18 Tampilan detail dari model	54
Gambar 3.19 Arsitektur web sistem monitoring dan peningkatan ML	54
Gambar 3.20 Diagram <i>Entity Relationship</i>	55
Gambar 3.21 <i>Use Case Diagram</i>	56
Gambar 3.22 <i>Activity</i> diagram melakukan prediksi.....	57
Gambar 3.23 <i>Activity Diagram</i> Meninjau Riwayat Prediksi	58
Gambar 3.24 <i>Activity</i> diagram menambahkan <i>batch</i>	59

Gambar 3.25 <i>Activity</i> diagram membuat <i>replay buffer</i>	60
Gambar 3.26 <i>Activity</i> diagram melakukan Training	60
Gambar 4.1 Contoh struktur direktori data pada set data <i>batch</i> 1	61
Gambar 4.2 Struktur pustaka Ultralytics	62
Gambar 4.3 Kode pengujian dan pengujian model baseline	63
Gambar 4.4 Contoh isi dari <code>testing_background.yaml</code>	64
Gambar 4.5 <i>Output</i> program pengujian ketika dijalankan	64
Gambar 4.6 Folder yang akan modifikasi	65
Gambar 4.7 Modifikasi pada <i>file dataset.py</i>	66
Gambar 4.8 Modifikasi <i>file default.yaml</i>	67
Gambar 4.9 Kode endpoint inference dan menyimpan prediksi	69
Gambar 4.10 <i>Endpoint</i> untuk riwayat prediksi dan menambahkan prediksi ke <i>batch</i>	70
Gambar 4.11 <i>Endpoint</i> untuk membuat batch atau <i>replay buffer</i>	73
Gambar 4.12 Fungsi untuk membuat <i>replay buffer</i>	78
Gambar 4.13 Kode program augmentation	81
Gambar 4.14 Program <i>random sampling</i> untuk <i>replay buffer</i>	84
Gambar 4.15 <i>Endpoint</i> model-detail	86
Gambar 4.16 Fungsi pelatihan dengan multi dataset.....	87
Gambar 4.17 Program testing	90
Gambar 4.18 Antarmuka prediksi untuk <i>end-user</i>	91
Gambar 4.19 Antarmuka riwayat prediksi.....	92
Gambar 4.20 Antarmuka detail prediksi	93
Gambar 4.21 Antarmuka fitur <i>batch</i>	93
Gambar 4.22 Antarmuka detail batch	94
Gambar 4.23 Antarmuka daftar model	94
Gambar 4.24 Antarmuka detail model dan fitur pelatihan model	95
Gambar 4.25 <i>Confusion matriks</i> model <i>baseline</i> dengan dataset testing <i>baseline</i>	96
Gambar 4.26 <i>Confusion matrix</i> model baseline dengan dataset testing baru	

.....	96
Gambar 4.27 Automasi <i>inference</i> pada endpoint prediksi.....	99
Gambar 4.28 Program untuk merekap data prediksi	101

DAFTAR TABEL

Tabel 2.1 Mode oleh <i>ultralytics</i> untuk Model YOLO	21
Tabel 2.2 Confusion Matrix	26
Tabel 3.1 Confusion matrix data baseline dengan model baseline	46
Tabel 3.2 Confusion matrix data baru dengan model baseline	47
Tabel 3.3 Confusion matrix data baseline dengan model batch 1	47
Tabel 3.4 Confusion matrix data baru dengan model batch 1	47
Tabel 4.1 Matriks evaluasi model <i>baseline</i>	97
Tabel 4.2 Distribusi dataset <i>batch 1</i>	102
Tabel 4.3 Hasil eksperimen pelatihan <i>batch 1</i>	103
Tabel 4.4 Distribusi dataset <i>batch 2</i>	105
Tabel 4.5 Seluruh eksperimen pada pelatihan <i>batch 2</i>	105

DAFTAR SIMBOL

INTISARI

ABSTRACT

BAB I PENDAHULUAN

1.1 Latar Belakang

Permasalahan pengelolaan sampah di Daerah Istimewa Yogyakarta (DIY) telah mencapai titik kritis seiring dengan peningkatan volume sampah yang signifikan setiap tahunnya. Menurut data dari Badan Perencanaan Pembangunan Daerah (Bappeda) Provinsi DIY, volume produksi sampah meningkat dari 644,69 ton per hari pada tahun 2019 menjadi 1.231,55 ton per hari pada tahun 2023. Namun, peningkatan ini tidak diimbangi dengan volume pengelolaan sampah yang ditangani hanya sebesar 740 ton per tahun [1]. Salah satu komponen terbesar dari sampah tersebut adalah sampah plastik yang turut menyumbang jenis sampah anorganik, sampah plastik mencapai 27,94% dari total komposisi sampah di DIY [2].

Plastik yang paling banyak ditemukan pada limbah rumah tangga adalah plastik jenis *Polyethylene Terephthalate* (PET), *High-Density Polyethylene* (HDPE), *Polypropylene* (PP), dan *Polystyrene* (PS) [3]. Dengan adanya tipe daur ulang pada plastik tersebut diharapkan untuk melakukan sistem manajemen sampah plastik agar dapat didaur ulang dan menghasilkan manfaat yang lebih baik. Seperti jurnal [4] mendaur ulang sampah plastik HDPE, PET, PP, LDPE menjadi butiran plastik (*pellet*) yang dapat digunakan kembali sebagai bahan baku industri plastik. Sampah plastik juga dapat menjadi bahan bakar cair dengan proses pirolisis [5] di mana plastik seperti PP, PS, dan *Polyethylene* (PE) diubah menjadi hidrokarbon cair yang kemudian dimurnikan menjadi bahan bakar.

Saat ini para ilmuwan memiliki insiatif untuk mengembangkan metode untuk melakukan manajemen sampah plastik. Salah satu upaya untuk melakukan manajemen dan memilah sampah plastik secara otomatis, cepat, dan akurat dengan menggunakan metode *machine learning* (ML) berbasis *computer vision*. Saat ini banyak model *computer vision* (CV) untuk melakukan klasifikasi objek sampah plastik dengan menggunakan convolutional neural network (CNN). Tujuan akhir dari model mengklasifikasikan jenis plastik, informasi ini diteruskan ke sistem fisik otomatis, seperti lengan robot atau *conveyor belt* yang dapat memilah sampah

berdasarkan jenis plastik. Seperti penelitian [5] yang menggunakan arsitektur *You Only Look Once* (YOLO) untuk melakukan deteksi sampah plastik PET dan PET-G. Model YOLO di-implementasi pada alat pemilah sampah plastik dan menggunakan sensor untuk mendeteksi sampah pada konveyor berjalan.

Pemilahan sampah plastik menggunakan pemrosesan gambar merupakan sebuah hal yang menantang untuk mendeteksi sampah plastik. Merujuk pada *literature review* [6] tantangan untuk melakukan deteksi sampah plastik adalah kurangnya data pada saat pelatihan dan kualitas dataset sampah plastik yang ada saat ini. Masalah yang signifikan adalah kurangnya pengetahuan yang sesuai pada kondisi di dunia nyata. Setiap objek sampah plastik, baik dalam bentuk utuh maupun berserakan, harus diberi anotasi untuk melatih detektor objek. Solusi yang dapat dilakukan adalah dengan meningkatkan kualitas kumpulan data sampah plastik dengan memberikan anotasi tervalidasi pada data. Meskipun pada model saat ini dapat memiliki akurasi yang tinggi dalam mendeteksi sampah plastik, tetapi model belum tentu dapat mendeteksi sampah yang ada pada kondisi objek yang realistis.

Hal serupa terjadi pada model YOLO yang akan diteliti. Model memiliki tugas untuk mengklasifikasi tipe sampah plastik. Yang di mana model masih sering terjadi kesalahan prediksi pada objek sampah plastik. Hal tersebut terjadi dikarenakan keterbatasan dan kualitas dataset. Objek sampah plastik sering kali berubah ubah bentuk pada kenyataannya seperti tipe sampah plastik *Polypropylene* (PP) terdapat beberapa macam bentuk seperti masker, cangkir, sedotan, dan lain lain. Jika model terus menerus melakukan pelatihan keseluruhan terus dengan adanya data terbaru akan tidak efisien. Dengan begitu perlunya melakukan pelatihan model kembali tanpa harus melatih secara keseluruhan dan model dapat beradaptasi pada objek baru.

Pada penggunaan model YOLO untuk melakukan pelatihan tambahan memiliki 2 metode yang memungkinkan yaitu *Transfer Learning/Fine tuning* (TL) dan kemungkinan kedua yaitu menggunakan metode *replay/rehearsal* [7] yang ada dalam paradigma *Continual Learning* (CL). CL Menurut [8] merupakan suatu kondisi dimana sistem diharapkan dapat beradaptasi terhadap aliran masukan yang

berasal dari keadaan yang dinamis. Keadaan yang berubah sering kali dimodelkan sebagai serangkaian tugas (*task/batch*) yang perlu dipelajari secara berurutan tanpa melupakan tugas yang telah dipelajari sebelumnya. CL memiliki beberapa strategi seperti *regularization*, *parameter isolation*, dan *memory-based* yang merupakan pendekatan yang ampuh dan sederhana untuk melakukan pembelajaran berkelanjutan [9].

Secara singkat penelitian ini bertujuan untuk meningkatkan performa model *machine learning* (ML) yang telah ada sebelumnya atau disebut model *baseline*. Model *baseline* menggunakan arsitektur YOLOv8 dengan varian klasifikasi. Model tersebut bertugas mengklasifikasikan empat kelas tipe sampah plastik, yaitu PET, HDPE, PP, dan PS. Karena model masih berbentuk prototipe dan masih sering melakukan kesalahan, penelitian ini mengembangkan sistem pemantauan untuk meninjau hasil prediksi model dan meningkatkan performanya menggunakan metode *replay*. Pada pengujiannya, penelitian ini terdapat 2 tugas. Tugas-tugas ini diasumsikan sebagai aliran data hasil prediksi yang dilakukan oleh pengguna akhir. Tugas-tugas tersebut pada penelitian ini berisi dengan objek plastik yang baru atau belum pernah dilihat oleh model. Hasil prediksi yang salah akan dianotasi dengan benar oleh pengguna, kemudian data tersebut akan dikumpulkan dalam bentuk dataset (*batch*). Dataset tersebut yang akan digunakan dalam *re-training* model. Dan data *batch* dapat digunakan *replay buffer* untuk melakukan pelatihan kedepan. Penelitian ini akan mengevaluasi performa model menggunakan metrik *F1-Score* yang berguna untuk mengatasi ketidakseimbangan kelas, *Backward Transfer* (BWT) untuk mengetahui apakah model dapat mempertahankan pengetahuan sebelumnya, dan *Forward Transfer* (FWT) berguna untuk mengetahui performa pengetahuan baru yang dipelajari. Pada akhirnya penelitian ini dapat membandingkan metode *replay* dengan metode *fine tuning* konvensional.

1.2 Rumusan Masalah

- Bagaimana merancang skenario untuk memonitoring dan meningkatkan performa deteksi pemilahan sampah plastik dengan menggunakan metode *replay*?

- Bagaimana melakukan pelatihan model YOLO menggunakan metode *replay*?
- Bagaimana eksperimen terbaik untuk mempertahankan dan meningkatkan performa akurasi model?
- Bagaimana cara mengimplementasi sistem monitoring dan peningkatan performa model berbasis web?

1.3 Batasan Masalah

Adanya rumusan masalah yang sudah dijelaskan pada poin sebelumnya adanya batasan masalah ini adalah untuk menjelaskan batasan penelitian yang akan dilakukan.

- Pada penelitian yang sudah ada sebelumnya mengusulkan metode dan algoritma yang lebih kompleks seperti iCaRL [10], ER [11], MIO [12] untuk mencari objek representatif untuk meningkatkan performa akurasi dari model. Sedangkan pada penelitian ini untuk menentukan episodik memori data yang akan dilatih ulang hanya akan menggunakan *random sampling* pada dataset baseline dan kumpulan data hasil prediksi (*batch*) untuk mempertahankan pengetahuan sebelumnya serta meningkatkan pengetahuan baru.
- Pada penelitian yang sudah ada, model *machine learning* untuk mengklasifikasi sampah plastik diimplementasi pada sebuah rangkaian alat otomatis. Seperti penelitian [13] menggunakan sensor *near-infrared spectroscopy* (NIRS) dan model YOLO untuk meningkatkan akurasi klasifikasi sampah plastik. Secara teknis, model YOLO digunakan untuk mendeteksi dan mengklasifikasikan jenis plastik berdasarkan bentuk fisiknya. Sistem ini menggunakan kontrol pneumatik dengan nozzle udara untuk memisahkan plastik. Penelitian tersebut dilakukan di sebuah pabrik atau pengolahan sampah plastik. Penelitian lainnya [14] menggunakan model YOLOv4 dengan *backbone* CSPdarknet53 untuk mendeteksi jenis sampah yang dapat di daur ulang. Penelitian ini memanfaatkan kamera dengan frame rate tinggi untuk menangkap video secara langsung dari ban berjalan di fasilitas pengolahan sampah di Kota Kozani, Yunani. Pada penelitian ini berfokus pada peningkatan performa akurasi model dalam mendeteksi tipe sampah plastik dengan metode *replay*. Sehingga implementasi

untuk melakukan pemilahan sampah plastik pada alat fisik seperti penelitian yang sudah ada dapat dilakukan pada proyek selanjutnya. Penelitian ini hanya akan menggunakan kamera sederhana seperti *smartphone* dan alat penangkap gambar lainnya yang pengguna untuk melakukan klasifikasi sampah plastik berdasarkan tipe daur ulangnya.

- Dalam paradigma penelitian *continual learning* seperti yang diuraikan dalam [15], [16], dan [11] penggunaan metrik *average accuracy* sangatlah relevan. Hal ini disebabkan oleh fokus penelitian tersebut pada masalah *catastrophic forgetting*, di mana dilakukan banyak pelatihan (*task*) untuk mengevaluasi kemampuan model dalam mempertahankan pengetahuan yang telah dipelajari dari tugas-tugas sebelumnya. Berdasarkan hal tersebut, penelitian ini menggunakan metrik *F1-score* yang bertujuan untuk mengantisipasi ketidakseimbangan kelas, dikarenakan keterbatasan data dan lebih tepat pada jumlah pelatihan yang sedikit. Selain itu, untuk mengetahui penurunan dan peningkatan performa model dari tugas-tugas sebelumnya dan baru, penelitian ini tetap menggunakan metrik *Backward Transfer* (BWT) dan *Forward Transfer* (FWT) untuk mengukur seberapa baik dalam pengetahuan baru.

1.4 Tujuan Proyek Akhir

Melakukan peningkatan performa akurasi dari model YOLO sebelumnya dan menciptakan sistem pemantauan hasil prediksi model. Mendapat nilai evaluasi terbaik dari pelatihan dengan menggunakan metrik evaluasi *f1-score*, *BWT*, *FWT*. Model dapat digunakan pengguna untuk mengklasifikasi tipe sampah plastik dengan performa model yang baik.

1.5 Manfaat Proyek Akhir

Penelitian ini memiliki dampak yang baik untuk mengakumulasi wawasan penelitian khususnya pengklasifikasian sampah plastik menggunakan kecerdasan buatan. Menambah wawasan untuk penelitian dengan topik penggunaan pendekatan *replay* dengan data yang terbatas. Menambah wawasan bagaimana melakukan pendekatan *replay* pada model *pre-trained* YOLO.

1.6 Sistematika Penulisan

Penelitian ini terdiri atas beberapa tahapan penulisan. Tahapan yang akan dilakukan dalam melakukan penelitian ini adalah sebagai berikut :

BAB 1 PENDAHULUAN

Bab pendahuluan yang berisi penjelasan ringkas mengenai latar belakang, rumusan masalah, batasan masalah, tujuan proyek akhir, manfaat proyek akhir, dan sistematika penulisan.

BAB 2 TINJAUAN PUSTAKA

Bab tinjauan pustaka berisi ringkasan tentang penelitian yang relevan dan sudah ada sebelumnya untuk menjadikan studi banding dan atau rujukan dalam penelitian proyek akhir ini.

BAB 3 METODE PROYEK AKHIR

Bab metode proyek akhir adalah uraian sistematika dan proses yang akan dilakukan pada penelitian ini. Hal ini meliputi bahan-bahan yang digunakan, peralatan yang dibutuhkan, sistematika pelaksanaan proyek akhir, serta analisis data yang dilakukan. Metode yang dilakukan ditujukan untuk memahami bagaimana proyek akhir ini dilakukan dan mengetahui cara-cara yang diambil dalam penelitian untuk memastikan hasil yang baik dan akurat.

BAB 4 HASIL DAN PEMBAHASAN

Bab hasil dan pembahasan meliputi semua keluaran yang dihasilkan dari seluruh proses penelitian proyek akhir. Seluruh pembahasan dijelaskan dengan cara ilmiah dan rasional mengenai hasil penelitian yang dilakukan. Hasil penelitian bisa berupa bentuk kuantitatif, kualitatif, dan statistik. Hasil proyek akhir disajikan secara obyektif, mengikuti metodologi pengujian yang telah ditetapkan.

BAB 5 PENUTUP

Bab kesimpulan dan saran berisikan pernyataan ringkas mengenai semua hasil yang telah dilakukan pada penelitian proyek akhir. Bagian ini meliputi saran yang berdasarkan evaluasi dari penelitian yang dilakukan. Bertujuan untuk memberikan penjelasan mengenai hasil penelitian untuk penelitian kedepannya.

TINJAUAN PUSTAKA

Daftar Pustaka berisi daftar buku teks atau artikel ilmiah/jurnal yang

mendukung Proyek Akhir dan rujukan dalam penulisan Proyek Akhir. Penulisan daftar pustaka mengikuti sistem Harvard (sitasi-nama-tahun) dengan pemisah tanda titik (.), dan diurutkan sesuai dengan urutan abjad nama belakang pengarang. Contoh cara penulisan daftar pustaka disajikan di Lampiran 8.

LAMPIRAN

Lampiran memuat keterangan tambahan untuk melengkapi Proyek Akhir. Lampiran data digunakan untuk menyajikan prosedur, program komputer, hasil simulasi, buku, atau keterangan lain yang tidak mungkin disingkat sehingga terlalu panjang untuk dimuat di bagian hasil dan pembahasan Proyek Akhir. Lampiran juga dapat digunakan untuk menampilkan data primer yang diperoleh dalam Proyek Akhir yang tidak dapat diinterpretasikan secara langsung.

BAB II

TINJAUAN PUSTAKA

2.1. Studi Pustaka

Penelitian ini memiliki beberapa referensi yang berguna untuk membantu penelitian ini dan pengembangan sistem pembelajaran berkelanjutan, proses penggunaan *machine learning* dengan metode *deep leaning* pada klasifikasi sampah plastik.

Penelitian [3] melakukan klasifikasi sampah plastik berdasarkan jenis nya dengan membandingkan arsitektur *convutional neural network* (CNN) dengan struktur 15 lapisan dengan resolusi 120x120 yang mereka buat dan lapisan 23 dari AlexNet dengan resolusi 227x227. Model CNN nantinya akan diintegrasikan dengan kamera micro yang berjalan dengan OS RaspPi. Eksperimen dilakukan pada pembagian dataset dengan 90% (training data), 10% (test data), 80%–20%, 70%–30%, dan 60%–40%. Hasil penelitian tersebut memiliki akurasi 74% dari lapisan 15 dengan jumlah parameter 4 juta dan lapisan 23 jumlah parameter 222 juta akurasi 72%. Struktur CNN dengan lapisan 15 pada penelitian yang dilakukan memiliki performa yang lebih baik dibanding dari AlexNet. Jumlah parameter yang sedikit menguntungkan jika model diimplementasikan untuk perangkat seluler seperti Raspberry Pi.

Penelitian [17] mengembangkan sistem untuk identifikasi dan kategorisasi material plastik menggunakan pendekatan deep learning. Penelitian ini mengeksplorasi beberapa model *convutional neural network* (CNN) yang populer yaitu InceptionV3, MobileNet, VGGNet dan penelitian ini juga mempertimbangkan metode machine learning tradisional seperti Decision Tree, SVM (Support Vector Machine), dan KNN (K-Nearest Neighbors) untuk perbandingan. Akurasi 90% dicapai oleh model CNN yang dibuat dalam mendeteksi beberapa polimer plastik, termasuk PET, PVC, dan HDPE. Model tersebut mencapai nilai akurasi dan perolehan yang sangat baik untuk setiap kelas, dengan nilai *precision* 0,95 dan nilai *recall* 0,94. Skor F1 model ditentukan sebesar 0,94 yang menunjukkan tingkat presisi yang baik dalam mengenali dan mengkategorikan berbagai bahan plastik.

Penelitian [18] melakukan penelitian deteksi dan klasifikasi untuk sampah plastik dengan menggunakan model YOLOv3 dengan enam klasifikasi kantong plastik, botol plastik, botol yang hancur, cangkir, styrofoam, dan sedotan. Penelitian tersebut melatih model dengan iterasi 2000 gambar kustom dataset. Hasil nilai evaluasi merupakan nilai *confidence* suatu deteksi dengan nilai yang baik pada botol plastik dan styrofoam 85% dan 75%. Sedangkan sedotan 65% dan yang lainnya antara 30 dan 40%. Ini menunjukkan bahwa algoritma dapat mendeteksi dan mengklasifikasi sampah plastik dengan benar. Namun, kedepannya gambar dapat memiliki objek yang bervariasi, kondisi cahaya, posisi, dan resolusi gambar untuk mencapai nilai *confidence* yang tinggi.

Penelitian yang lebih lanjut [13] dengan mendeteksi sampah plastik PET dan PET-G menggunakan sensor *near-infrared spectroscopy* (NIRS) yang terintegrasi dengan model YOLOv8. Sistem diimplementasikan disebuah alat *conveyor* untuk mengklasifikasikan sampah plastik PET dan PET-G dari sampah plastik lain. Modbus TCP memungkinkan komunikasi antara hasil inferensi YOLO dan kontrol nozzle udara untuk pemilahan pneumatik. Algoritma deep learning dilatih menggunakan data lapangan dan augmentasi data untuk meningkatkan akurasi. Pengujian dilakukan dengan peralatan pemilahan nyata, menggunakan sampel PET dan PET-G pada conveyor belt berkecepatan 2 m/s. Algoritma mencapai akurasi klasifikasi 91.7 mAP, menunjukkan keefektifannya. Sistem menunjukkan kinerja real-time, sesuai dengan kecepatan conveyor belt.

Penelitian tentang klasifikasi sampah lainnya yang terimplementasi di industri pembuangan sampah seperti penelitian [14] menggunakan model YOLOv4 untuk membuat sebuah deteksi objek yang akurat dan pintar yang diuji dalam fasilitas pengumpulan sampah (*Waste Collection Facility/WCF*) terletak di kota Kozani, Yunani. Penelitian ini menambahkan CSPDarknet53 sebagai *backbone network* pada arsitektur YOLOv4. Penelitian tersebut sukses membuat sebuah klasifikasi sampah daur ulang seperti plastik dengan menggunakan dataset TACO dan dataset kustom yang diambil dari lingkungan sekitar. Pada penelitian tersebut juga membuat sebuah model adaptif yang dapat melibatkan pengguna akhir untuk dalam proses pengembangan untuk memastikan kebutuhan industri. Sistem

mencapai akurasi 92,43% dalam mengklasifikasikan material sampah menjadi empat kategori: kertas, plastik, aluminium, dan lainnya.

Adanya penelitian terkait penggunaan *machine learning* khususnya arsitektur CNN, tidak menutup kemungkinan untuk dilakukannya pembelajaran berkelanjutan sehingga model dapat terus berkembang dan beradaptasi pada objek dan kondisi yang baru. Seperti pada tujuan penelitian ini akan melakukan pembelajaran berkelanjutan pada model klasifikasi sampah plastik. Maka dari itu, studi pustaka pada materi yang mengacu pada pembelajaran berkelanjutan yang dapat membantu proyek dan penelitian ini adalah sebagai berikut.

Penelitian [10] ini memperkenalkan *incremental classifier and representation learning* (iCaRL), sebuah strategi untuk pembelajaran *class-incremental* yang memungkinkan model untuk belajar secara terus-menerus dari aliran data yang masuk, di mana kelas-kelas baru muncul seiring waktu. iCaRL mampu mempelajari pengklasifikasi dan representasi fitur secara bersamaan dan penelitian tersebut menggunakan distilasi pengetahuan untuk mengambil beberapa data dari fitur representasi untuk menjaga kemiripan baru dengan yang lama dan *rehearsal prototype* yang berfungsi untuk menggunakan contoh dari kelas kelas sebelumnya untuk ditambah kan di parameter pelatihan. Eksperimen tersebut menggunakan CIFAR-100 dan ImageNet ILSVRC 2012. Hasil akurasi dari metode tersebut meningkat dari pembelajaran kelas dari 2 sampai 50 dengan akurasi 57% - 68.6% dibanding metode lainnya 11% - 64% (LwF.MC).

Penelitian [16] melakukan *continual learning* (CL) menggunakan *pretrained* model ResNet-50 dengan dataset ImageNet. Penelitian ini menggunakan COrE50 UB sebagai benchmark yang dirancang untuk meneliti CL. Metode yang diusulkan adalah *head protection*, *replay memory management*, dan augmentasi. Metode pengambilan sample *replay* dengan membagi membagi dua kategori memori *main* (M), *temporal* (T). M sebagai memory lama yang akan digabung dengan T dan T sebagai tempat memori baru. Pemindahan memori T ke M dilakukan secara acak sehingga nantinya T semakin berkurang. Menggunakan pendekatan *reservoir sampling* digunakan untuk memilih data yang akan disimpan dalam T. Setelah itu penelitian melakukan pelatihan *replay* pada model. Hasil

penelitian tersebut menunjukkan keunggulan dari segi metrik akurasi rata-rata dan kelupaan rata-rata (70.84% dan 89.88%), iCaRL dengan rata-rata akurasi ~30%, dan LwF dengan rata-rata akurasi ~60%.

Penelitian [11] ini mengeksplorasi penggunaan *Experience Replay* (ER) dengan memori episodik yang sangat kecil untuk mengatasi *catastrophic forgetting* dalam *Continual learning* (CL). ER menyimpan sebagian kecil data dari tugas sebelumnya dalam memori dan menggunakannya kembali saat pelatihan pada tugas baru. Penelitian ini menggunakan empat benchmark dataset yang umum digunakan dalam literatur CL: Permuted MNIST, Split CIFAR, Split miniImageNet, dan Split CUB. Arsitektur jaringan yang digunakan bervariasi tergantung pada dataset, termasuk jaringan *fully-connected* untuk MNIST dan varian ResNet untuk dataset lainnya. ER dengan reservoir sampling menunjukkan kinerja terbaik secara keseluruhan, ER tidak hanya mengungguli baseline dalam hal akurasi dan forgetting, tetapi juga lebih efisien secara komputasi. Akurasi dan forgetting pada penelitian ER ini dengan 15 sampel per kelas mencapai akurasi rata-rata 79.8% dan forgetting 0.04 (permuted MNIST), akurasi rata-rata 68.5% dengan forgetting 0.05 (Split CIFAR-100), akurasi rata-rata 61.3% dengan forgetting 0.04 (Split miniImageNet), akurasi rata-rata 76.5% dengan forgetting 0.03 (Split CUB-200).

Penelitian [9] berfokus pada strategi yang baik pada *Continual learning* (CL) berbasis *replay*, yang merupakan teknik penting untuk mengatasi masalah lupa *catastrophic forgetting* dalam pembelajaran berkelanjutan. Tujuan utama penelitian ini adalah untuk memberikan rekomendasi praktis tentang bagaimana mengembangkan strategi berbasis replay yang efisien, efektif, dan dapat diterapkan secara umum. Penelitian ini mengeksplorasi tiga aspek utama dari strategi berbasis replay yaitu ukuran memori, kebijakan pembobotan sampel, augmentasi data. Eksperimen dilakukan dengan buffer replay diseimbangkan berdasarkan tugas, artinya memori berisi jumlah sampel yang sama dari setiap tugas. Untuk strategi Replay, mereka menggunakan pengambilan sampel acak. Hasil dan kesimpulan dari penelitian tersebut, faktor ukuran memori mengetahui bahwa semakin besar ukuran memori semakin baik kinerja CL terutama untuk tugas yang kompleks. Faktor kebijakan pembobotan menghasilkan kesimpulan bahwa pentingnya memori

terbaru yaitu memberikan bobot lebih pada sampel yang lebih baru (*Increasing* dan *MiddleHigh*) cenderung menghasilkan kinerja yang lebih baik dari pada memberikan bobot lebih pada sampel yang lebih lama (*Decreasing*). Terakhir Augmentasi data pada sampel dalam memori replay dapat meningkatkan kinerja, terutama ketika ukuran memori terbatas.

Penelitian [12] berjudul "*Improving Continual Learning of Acoustic Scene Classification via Mutual Information Optimization*" dan bertujuan untuk meningkatkan kinerja continual learning dalam tugas klasifikasi adegan akustik (acoustic scene classification, ASC) dengan mengoptimalkan mutual information (MI). Penelitian ini berfokus pada bagaimana model ASC dapat terus belajar dari data baru (kelas baru) sambil mengurangi efek lupa terhadap tugas-tugas sebelumnya. Peneliti mengusulkan penggunaan pendekatan *replay-based continual learning* dengan optimasi *mutual information* untuk memilih sampel memori yang lebih representatif dan informatif. Penelitian ini diuji pada dua dataset, *Urban Acoustic Scenes 2022* (TAU) yaitu Dataset yang terdiri dari 10 kelas suara akustik dari lingkungan perkotaan. Dataset ESC-50 adalah dataset yang lebih kecil dengan 50 kelas suara dari berbagai kategori, seperti suara binatang, manusia, dan aktivitas sehari-hari.. Penelitian menunjukkan bahwa pendekatan *mutual information optimization* (MIO) yang diusulkan memiliki performa yang lebih baik dibandingkan metode lain seperti random sampling, herding, dan uncertainty-based selection. Model yang dioptimalkan dengan MIO mampu mempertahankan lebih banyak pengetahuan dari tugas lama (BWT lebih tinggi) dan memiliki generalisasi yang lebih baik pada tugas baru (FWT lebih tinggi).

Tabel 1.1 Daftar tinjauan pustaka

Penulis dan Tahun Penelitian	Judul	Metode	Hasil Penelitian
Janusz Bobulski dan Mariusz	<i>Deep Learning for Plastic Waste Classification</i>	CNN 15 lapisan dengan resolusi 120x120	Akurasi 74% dengan Parameter sebanyak 4 juta untuk klasifikasi

KubaneK (2021)	<i>System</i>		tipe sampah plastik
Viswanadham <i>et al</i> (2023)	<i>An Identification and Categorization of Plastic Material using Deep Learning Approach</i>	CNN (InceptionV3, MobileNet, VGGNet), ML (Decision Tree, SVM, dan KNN)	Nilai akurasi 90%, <i>precision</i> 95%, <i>recall</i> 94%, dan <i>F1-Score</i> 94% dengan model CNN untuk klasifikasi tipe sampah plastik
Raka Ardana <i>et al</i> (2020)	<i>Application of Object Recognition for Plastic Waste Detection and Classification Using YOLOv3</i>	Arsitektur YOLOv3	Hasil nilai evaluasi merupakan nilai confidence suatu deteksi dengan nilai yang baik pada botol plastik dan styreoform 85% dan 75%. Sedangkan sedotan 65% dan yang lainnya antara 30 dan 40%.
Choi <i>et al</i> (2023)	<i>Advancing Plastic Waste Classification and Recycling Efficiency: Integrating Image Sensors and Deep Learning Algorithms</i>	Arsitektur YOLOv8 dan sensor <i>near-infrared spectroscopy</i> (NIRS)	Berhasil klasifikasi plastik PET dan PET-G pada conveyor belt berkecepatan 2 m/s. Algoritma mencapai akurasi klasifikasi 91.7 mAP,

Ziouzios <i>et al</i> (2022)	<i>Intelligent and Real-Time Detection and Classification Algorithm for Recycled Materials Using Convolutional Neural Networks</i>	Arsitektur YOLOv4 dengan CSPDarknet53 sebagai backbone	Sistem mencapai akurasi 92,43% dalam mengklasifikasikan material sampah menjadi empat kategori: kertas, plastik, aluminium, dan lainnya pada kondisi dunia nyata
Rebuffi <i>et al</i> (2017)	<i>iCaRL: Incremental Classifier and Representation Learning</i>	<i>Nearest-Mean-of-Exemplars, Herding, dan Replay</i>	Hasil akurasi dari metode tersebut meningkat dari pembelajaran kelas dari 2 sampai 50 dengan akurasi 57% - 68.6%.
Graffieti <i>et al</i> (2022)	<i>Continual Learning in Real-Life Applications</i>	<i>Replay</i>	Model yang menggunakan replay memory dan pendekatan balancing memiliki performa lebih baik dalam menjaga akurasi dibandingkan metode yang tidak menggunakan replay dengan akurasi rata-rata sebesar 70.84% dan akurasi <i>Top-1</i> 89.88%

Chaudhry <i>et al</i> (2019)	<i>On Tiny Episodic Memories in Continual Learning</i>	<i>Experience replay (ER)</i>	Akurasi dan forgetting pada penelitian ER ini dengan 15 sampel per kelas mencapai akurasi rata-rata 79.8% dan forgetting 0.04 (permuted MNIST)
Merlin <i>et al</i>	<i>Practical Recommendations for Replay-based Continual Learning Methods</i>	<i>Replay dan random sampling</i>	Hasil dari metode penelitian ini yang paling sering menghasilkan performa yang baik. Dengan nilai akurasi 90% pada Split-MNIST
Yang <i>et al</i>	<i>Improving Continual Learning of Acoustic Scene Classification via Mutual Information Optimization</i>	<i>Mutual Information Optimization (MIO)</i>	Pada dataset TAU menggunakan ukuran memori 1000 menghasilkan nilai akurasi sebesar 64.1 %, dengan nilai BWT dan FWT terbaik dari metode lain sebesar - 22.5% dan 74.8%. Dataset ESC-50 dengan akurasi 55.3%, BWT - 26.5%, FWT 57.3%

2.2. Dasar Teori

Hal-hal yang akan dijelaskan pada subbab berikut adalah teori-teori umum yang digunakan sebagai dasar dalam penelitian ini. Landasan teori yang dijelaskan berasal dari berbagai sumber yang relevan.

2.2.1. Label Resin Plastik

Label daur ulang plastik atau tipe material plastik merujuk pada simbol-simbol yang digunakan untuk mengidentifikasi jenis resin plastik yang digunakan dalam produk, serta memberikan informasi kepada konsumen tentang potensi daur ulang dari plastik tersebut. Simbol ini sering kali dikenal dengan *International Plastic Resin Symbols* atau kode identifikasi resin yang dikembangkan oleh *Society of the Plastics Industry* (SPI) [19]. Simbol-simbol tersebut biasanya berupa segitiga dengan nomor di dalamnya, yang mewakili berbagai jenis plastik, dapat dilihat pada Gambar 2.1.

SPI number	Plastic name	Symbol	Examples of applications
1	PET		Disposable bottles for drinks, fibres (clothing, carpet), film, cosmetic packaging, food containers
2	HDPE		More durable non-food containers, buckets, crates, recycling bins, floor tiles
3	PVC		Piping, cables, garden furniture, fencing, decking, panelling, floor tiles and mats, traffic cones, electrical equipment
4	LDPE		Plastic bags, trays and computer components, wrapping films, trays
5	PP		Car parts, signal lights, oil funnels, brushed, yogurt containers, bicycle racks, bottle caps, reusable food containers
6	PS		Thermal insulation, foam packing, take-out food containers, disposable cutlery
7	Other		Polycarbonate (refillable plastic bottles, consumer electronics) nylon (clothing, carpets), biodegradable resins, mixed plastics and blends (electronics housing, plastic lumber)

Gambar 2.1 *International Resin Identification Coding System for plastics* [19]

Label resin tersebut biasa terdapat pada produk produk yang menggunakan bahan plastik. Label resin biasanya terdapat pada bagian bawah botol, di belakang kemasan, atau bagian yang tidak kontras.

2.2.2. Machine Learning

Machine learning adalah cabang kecerdasan buatan atau *Artificial Intelligence* (AI) yang berfokus pada pengembangan algoritme dan model statistik yang memungkinkan sistem komputer meningkatkan kinerjanya pada tugas tertentu melalui pengalaman, tanpa diprogram secara eksplisit [20]. Pembelajaran ini mencakup berbagai teknik yang memungkinkan komputer belajar dari data dan membuat prediksi atau keputusan berdasarkan data [21].

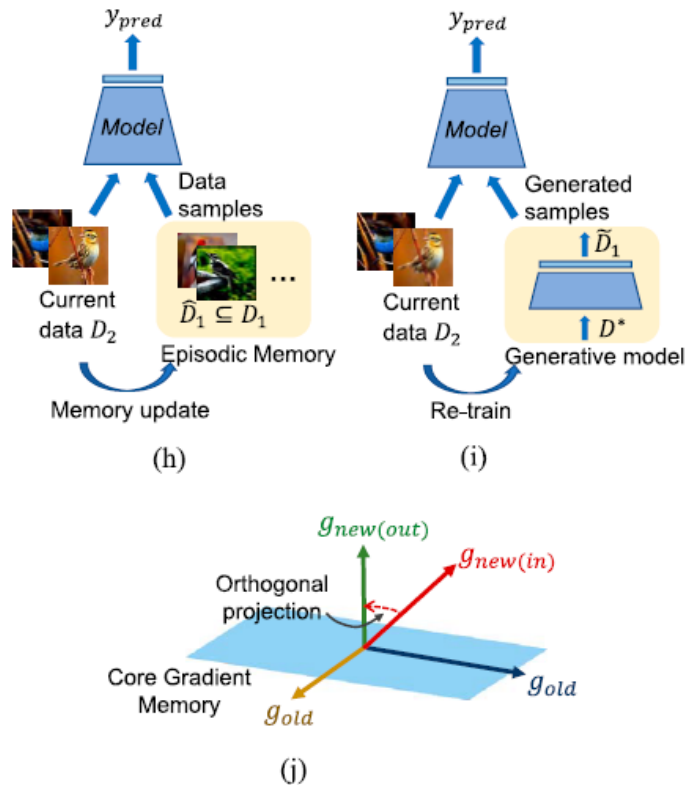
Metode-metode ini secara luas dapat dikategorikan menjadi *Supervised learning*, *unsupervised learning*, dan *reinforcement learning* [22]. *Supervised learning* melibatkan pelatihan model pada data berlabel untuk membuat prediksi atau klasifikasi. *Unsupervised learning* bertujuan untuk menemukan pola atau struktur tersembunyi dalam data tak berlabel. *Reinforcement learning* memungkinkan agen mempelajari perilaku optimal melalui interaksi dengan lingkungan [23].

Bidang pembelajaran mesin telah mengalami kemajuan pesat dalam beberapa tahun terakhir, didorong oleh peningkatan daya komputasi, ketersediaan kumpulan data besar, dan terobosan dalam pendekatan algoritmik [24]. Aplikasi pembelajaran mesin mencakup banyak domain, termasuk visi komputer, pemrosesan bahasa alami, robotika, perawatan kesehatan, dan keuangan [25].

2.2.3. Memory-Based Learning

Memory-Based learning terinspirasi pada cara kerja memori pada otak biologis manusia bagaimana cara menyimpan dan mengingat informasi masa lalu. Ingatan dan pembelajaran adalah dua komponen yang saling terkait yang membentuk landasan pembelajaran seumur hidup manusia. Pembelajaran adalah menggeneralisasi situasi baru dengan mengekstraksi struktur statistik dari peristiwa yang terjadi, sedangkan ingatan mempertahankan peristiwa tertentu (memori episodik) atau pengetahuan faktual (memori semantik) [26]. Pada metode berbasis memori ada terdapat tiga konsep, bisa dilihat pada Gambar 2.2. Konsep (h) *replay/rehearsal*, (i) *generative-replay*, (j) *gradient-based data-private methods*. Menurut [8] dari ketiga konsep yang telah disebutkan *replay/rehearsal* merupakan

konsep yang paling sederhana dimana subset contoh dari tugas sebelumnya disimpan untuk diputar atau diulang pada pelatihan mendatang. Metode *gradient-base/data-private* metode data-pribadi memelihara sekumpulan vektor memori, fitur atau ruang gradien yang mewakili esensi setiap tugas. Pada metode *generative replay* melakukan sintesis data lama menggunakan model generatif.

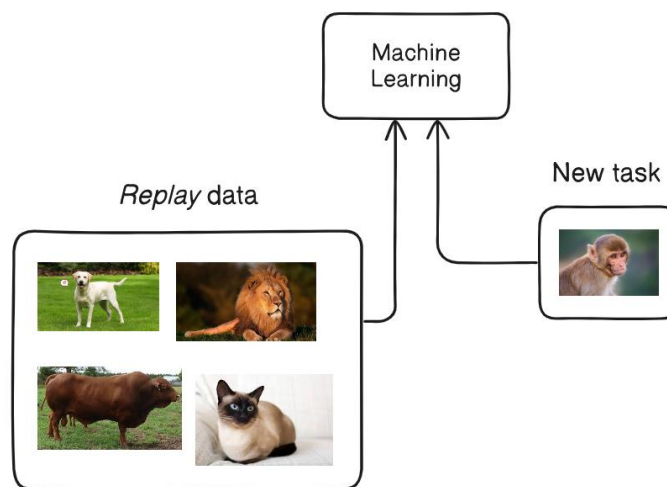


Gambar 2.2 Tiga jenis pendekatan berbasis memori [8]

2.2.4. Replay

Replay atau *Rehearsal* [5], merupakan salah satu pendekatan dalam pembelajaran mesin yang berbasis pada penggunaan memori. Tujuan utama dari replay adalah untuk memitigasi fenomena *catastrophic forgetting*, yaitu kecenderungan model untuk melupakan pengetahuan yang telah dipelajari sebelumnya ketika dihadapkan pada data atau tugas baru. Mekanisme *replay* bekerja dengan menyimpan subset dari contoh data sebelumnya dalam sebuah *buffer*, dan kemudian mengambil sampel dari *buffer* ini bersama dengan observasi data saat ini selama proses pelatihan, bahkan dalam beberapa kasus, selama inferensi. Pemilihan data yang tepat untuk *buffer replay* merupakan langkah krusial

dalam pembelajaran berkelanjutan berbasis memori. Data yang dipilih diharap representatif terhadap karakteristik penting dari distribusi masukan yang relevan dengan setiap tugas. Pada dasarnya, penyimpanan sampel data ini dapat dianggap sebagai memori episodik yang menyimpan pengalaman-pengalaman spesifik yang dapat dimanfaatkan di masa mendatang. Konsep replay dapat divisualisasikan pada Gambar 2.2, di mana data replay direpresentasikan sebagai kilasan informasi atau pengalaman dari masa lalu.

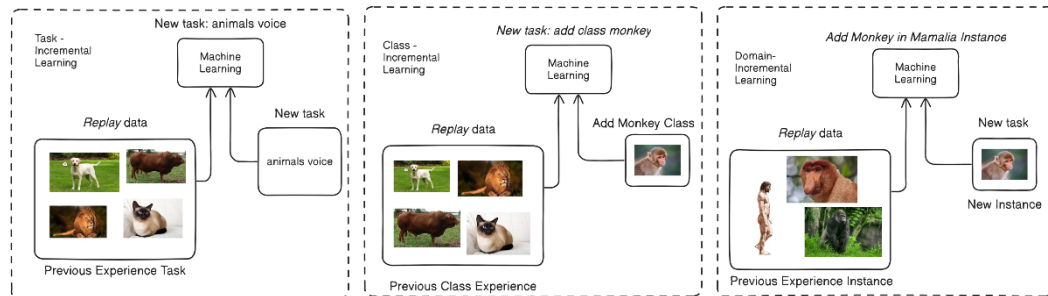


Gambar 2.3 Pengulangan memori pada pelatihan *machine learning*

2.2.5. Domain-incremental learning

Berdasarkan definisi yang dikemukakan oleh [27], *Domain-incremental Learning* (Domain-IL) merujuk pada suatu paradigma pembelajaran mesin di mana model dihadapkan pada tugas klasifikasi yang konsisten, namun dengan variasi dalam distribusi data masukan atau konteksnya. Tantangan utama dalam Domain-IL adalah memastikan bahwa model mampu beradaptasi dengan perubahan konteks ini tanpa mengalami fenomena *catastrophic forgetting*, yaitu kehilangan kemampuan untuk menyelesaikan tugas aslinya. Sebagai ilustrasi, pertimbangkan model yang dilatih untuk mengenali objek baik di dalam maupun di luar ruangan, atau sistem mengemudi otonom yang harus beroperasi dalam kondisi cuaca yang beragam. Studi kasus yang dibahas dalam penelitian ini melibatkan adaptasi terhadap bentuk dan lingkungan yang dinamis, sehingga dapat diasumsikan bahwa proses peningkatan kinerja model termasuk dalam skenario Domain-IL.

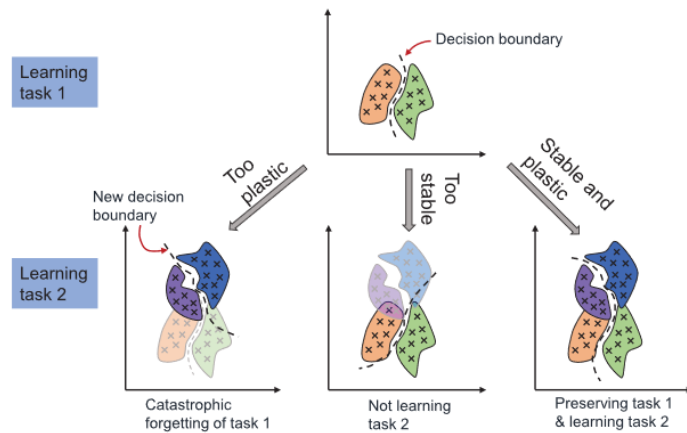
Hal ini berbeda dengan skenario Task-incremental Learning, di mana model harus mempelajari tugas-tugas yang berbeda, dan Class-incremental Learning, di mana model harus mengenali kelas-kelas objek baru. Perbedaan fokus pembelajaran pada ketiga skenario ini dapat divisualisasikan pada Gambar 2.3.



Gambar 2.4 Tiga perbedaan *Incremental-Learning*

2.2.6. Continual Learning

Continual Learning (CL) atau pembelajaran berkelanjutan merupakan kemampuan memperoleh, memproses, dan belajar dari informasi baru tanpa melupakan pengetahuan sebelumnya. Otak manusia telah berevolusi untuk mampu menghadapi keadaan yang selalu berubah dengan baik dan belajar dari pengalaman dengan bantuan mekanisme neurofisiologis yang kompleks. Meskipun kecerdasan buatan mirip dengan kecerdasan manusia, jaringan saraf tradisional tidak memiliki kemampuan untuk beradaptasi dengan lingkungan yang dinamis. Ketika informasi baru disajikan, jaringan syaraf tiruan sering kali benar-benar melupakan pengetahuan sebelumnya, sebuah fenomena yang disebut *catastrophic forgetting* atau *catastrophic interference* [8]. CL merupakan definisi umum dari pelatihan suatu model *machine learning* (ML) yang berkelanjutan. Istilah lain dari CL adalah *lifelong learning*, *incremental learning*, dan *sequential learning* dan pada tahun-tahun terakhir istilah CL banyak digunakan oleh komunitas *deep learning* [28]. Pada gambar berikut merupakan satu tujuan yang diinginkan pada paradigma CL dapat dilihat pada gambar 2.3 berikut:



Gambar 2.5 Keseimbangan stabilisasi dan *plastic* (fleksibilitas) pada CL

2.2.7. Model YOLO dan Ultralytics

Menurut [29] Model *You Only Look Once* (YOLO) merupakan model *deep learning* untuk deteksi objek yang dapat mengidentifikasi dan mengklasifikasikan objek dalam satu lintasan gambar. Model ini merupakan bagian dari *Convolutional Neural Networks* (CNN) yang banyak digunakan untuk data citra. Menurut [30] YOLOv8 adalah versi terbaru dari model YOLO yang populer karena akurasi dan ukurannya yang ringkas dan dapat dibagi menjadi 4 tugas dari masing masing tipe model yaitu deteksi, klasifikasi, segmentasi, dan pose. Contoh penggunaan model YOLO untuk tugas klasifikasi, menggunakan *suffix -cls* (YOLOv8-cls). Tim pengembang YOLOv8 dan versi sebelumnya YOLOv5 merupakan *Ultralytics*. Dengan ada nya *Ultralytics* model YOLOv8 dapat digunakan dengan mudah dan juga memiliki forum untuk berdiskusi dengan pengembang. *Ultralytics* juga menyediakan pustaka untuk menggunakan model YOLO. Fungsi dari pustaka tersebut dapat mempermudah pengguna model mulai dari mode *training*, *validation*, *predict*, *export*, dan lain lain.

Fungsi-fungsi yang disediakan oleh pustaka *ultralytics* dan yang digunakan pada penelitian ini berikut ada.

Tabel 2.1 Fungsi penggunaan model YOLO oleh *ultralytics*

No	Nama Fungsi	Penggunaan	Keterangan
1.	<i>validation</i>	<code>val()</code>	Mengevaluasi model terlatih pada set

			validasi untuk mengukur akurasi dan kinerja generalisasinya
2.	<i>training</i>	<code>train()</code>	Melatih model khusus pada dataset dengan hyperparameter yang ditentukan
3.	<i>predict</i>	<code>model(img)</code>	Membuat prediksi menggunakan model terlatih pada gambar atau video baru

2.2.8. Dataset

Dataset adalah sekumpulan data. Konteks dataset pada *image classification* adalah kumpulan foto digital yang dipilih dan digunakan untuk pelatihan, pengujian, dan evaluasi kinerja algoritme pembelajaran mesin [31]. Contoh nya dataset ImageNet, Dataset ImageNet merupakan sebuah database besar atau kumpulan gambar yang terstruktur secara hierarkis. Database ini dibangun berdasarkan struktur WordNet [32]. ImageNet bertujuan untuk menyediakan sumber daya yang komprehensif dan beragam bagi para peneliti di bidang visi komputer dan lainnya.

Khususnya penelitian ini menggunakan model YOLO klasifikasi. Struktur dataset untuk melakukan pelatihan model kumpulan data harus diatur dalam struktur direktori terpisah tertentu di bawah direktori *root* untuk memfasilitasi pelatihan, pengujian, dan proses validasi opsional yang tepat. Struktur ini mencakup direktori terpisah untuk fase pelatihan (*train*) dan pengujian (*test*), dengan direktori opsional untuk validasi (*val*). Berikut merupakan contoh struktur dataset dari CIFAR-10 yang digunakan pelatihan model YOLO dilihat pada Gambar 2.6.

```

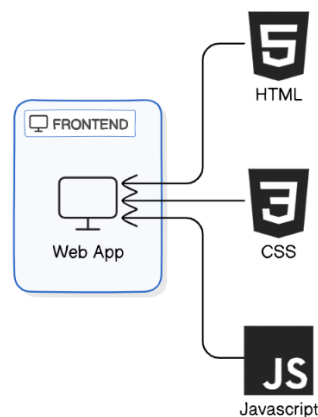
cifar-10-/
|-- train/
|   |-- airplane/
|   |   |-- 10008_airplane.png
|   |   |-- 10009_airplane.png
|   |   |-- ...
|   |-- ... (kelas lainnya)
|-- validate/
|   |-- airplane/
|   |   |-- ...
|   |-- ... (kelas lainnya)
|-- test/
|   |-- airplane/
|   |   |-- ...
|   |-- ... (kelas lainnya)

```

Gambar 2.6 Struktur direktori dataset klasifikasi

2.2.9. Frontend

Menurut [33] *Frontend* segala sesuatu yang dapat dilihat dan berinteraksi dengan pengguna melalui peramban (*browser*). Pengembangan *front-end* bertanggung jawab atas tampilan visual dan elemen interaktif dari sebuah situs web. Pengembang *front-end* biasanya bekerja dengan alat-alat seperti *HTML*, *CSS*, dan *JavaScript* untuk menciptakan antarmuka pengguna yang dapat dilihat dan digunakan di peramban. Gambar 2.7 menunjukkan penjelasan mengenai struktur pembangun *front-end*.

Gambar 2.7 Struktur pembangun *front-end*

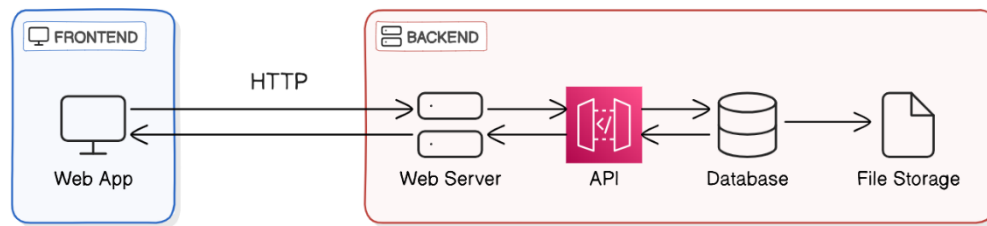
HTML (HyperText Markup Language) adalah bahasa pemrograman dasar yang digunakan untuk membuat halaman web. HTML berfungsi untuk mengatur struktur konten halaman web dengan menggunakan elemen-elemen seperti paragraf, header, tautan, gambar, dan banyak lagi. HTML juga digunakan untuk membuat tautan hiper (hypertext) yang memungkinkan pengguna beralih dari satu halaman ke halaman lainnya.

Cascading Style Sheets (CSS) digunakan untuk mengatur tampilan dan layout elemen HTML. Dengan CSS, pengembang dapat menyesuaikan warna, font, margin, dan tata letak elemen-elemen di halaman web. CSS juga mendukung konsep warisan (inheritance), yang memungkinkan beberapa elemen mewarisi gaya dari elemen induknya, dan mendukung mekanisme cascading, di mana urutan penerapan aturan CSS dapat mempengaruhi tampilan akhir elemen.

JavaScript (JS) digunakan untuk menambahkan interaktivitas dan logika dinamis pada halaman web. JS memungkinkan pengembang untuk memanipulasi elemen HTML, menangani event seperti klik tombol, dan memperbarui tampilan halaman secara real-time tanpa perlu memuat ulang halaman. JS juga dapat berfungsi sebagai mengirim permintaan ke backend atau *server-side* sekaligus menangani respons yang akan digunakan di *frontend* atau *client-side*.

2.2.10. Backend

Menurut [33] *Backend*, atau sering disebut sebagai *server-side development*, adalah bagian dari aplikasi web yang menangani logika bisnis, penyimpanan data, dan komunikasi antara server dan database. *Backend* bertanggung jawab untuk menerima permintaan dari *front-end* (biasanya melalui HTTP), memproses permintaan tersebut, dan mengembalikan data dalam format yang dibutuhkan oleh *front-end*. Gambar 2.8 Arsitektur Backend merupakan visualisasi dasar mengenai cara kerja *backend*.



Gambar 2.8 Arsitektur Backend

HyperText Transfer Protocol (HTTP) adalah protokol komunikasi yang digunakan antara *frontend* dan *backend* (server). Protokol ini mengatur pertukaran informasi antara klien dan server melalui permintaan (request) dan balasan (response). HTTP request ini bisa berupa metode GET, POST, PUT, dan DELETE yang biasa disebut sebagai REST API. Server mengirimkan HTTP response kembali ke *frontend*, yang dapat berisi data seperti JSON atau pesan status.

Application Programming Interface (API) bertindak sebagai penghubung antara frontend dan backend. Dalam gambar, API terletak di antara web server dan database. API digunakan untuk menyediakan akses ke data dan fungsi backend tanpa harus mengungkapkan detail internal tentang bagaimana backend bekerja. API memungkinkan frontend untuk melakukan operasi CRUD (Create, Read, Update, Delete) pada data di database. API dibangun

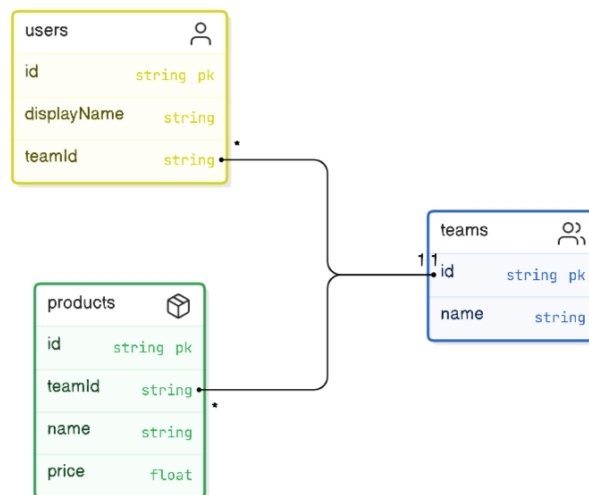
Database adalah komponen backend yang menyimpan data yang diperlukan oleh aplikasi web. Ketika pengguna melakukan permintaan untuk membaca, menulis, memperbarui, atau menghapus data, backend berinteraksi dengan database untuk mengeksekusi operasi tersebut.

2.2.11. Basis Data

Data adalah fakta atau informasi yang dapat digunakan untuk membuat keputusan [34]. Menurut [35], basis data adalah kumpulan data yang disimpan secara sistematis dalam komputer sehingga program komputer dapat memperoleh data dari basis data, dan perangkat lunak yang digunakan untuk mengelola dan mengeluarkan data disebut *Database Management System* (DBMS).

DBMS terbagi ke banyak jenis, salah satunya untuk menggunakan basis data

relationship. DBMS *relationship* tabel data terhubung dengan tabel lain yang dimana mengartikan ketergantungan data. Terhubungnya tabel tabel ini dapat direpresentasikan kedalam diagram yaitu *Entity Relation Diagram* (ERD). Contoh gambar berikut merupakan contoh visualisasi dari ERD:



Penelitian ini menggunakan DBMS *relationship* yaitu MySQL, MySQL merupakan DBMS populer yang banyak digunakan dikarenakan sifat nya yang *open-source*. Menggunakan MySQL dapat dilakukan pada *shell*, tetapi pada penelitian ini menggunakan aplikasi XAMPP untuk manajemen basis data dengan mudah.

2.2.12. Metrik Evaluasi

F1-score salah satu metrik yang digunakan pada evaluasi model klasifikasi. Untuk mengevaluasi model klasifikasi pada data yang tidak seimbang, metrik terbaik yang direkomendasikan adalah *F1-score* [36]. *F1-Score* terdiri dari hasil rata-rata harmonik pada *precision* dan *recall*. Semua variabel pada metrik-metrik yang ada nanti didapatkan dari *confusion matrix* lihat pada tabel 1.2, Saat pengklasifikasian objek keluaran hasil terdiri dari matriks 2×2 : *true positive* (TP), *true negatif* (TN), *false positive* (FP), dan *false negative* (FN)..

Tabel 2.2 Confusion Matrix

	Predicted	
	Positive	Negative

Actual	Positive	TP	TN
	Negative	FP	FN

1. Precision

Precision adalah metrik untuk skenario klasifikasi yang berfokus pada pengukuran yang menghindari FP. Dengan kata lain, metrik ini digunakan dalam situasi di mana kasus pengklasifikasian FP harus dihindari semaksimal mungkin. Persamaan matematika berikut dapat digunakan untuk menentukan *precision*:

$$Precision = \frac{TP}{TP+FP}$$

2. Recall

Recall adalah sebuah metrik untuk skenario klasifikasi yang berfokus pada pengukuran yang menghindari FN (False Negative). Dengan kata lain, *Recall* digunakan pada skenario di mana kasus pengklasifikasian FN harus dihindari semaksimal mungkin. Rumus matematika *recall* dapat dijelaskan pada persamaan berikut:

$$Recall = \frac{TP}{TP+FN}$$

3. F1-Score

F1-score adalah metrik yang menggabungkan Precision dan Recall menjadi satu nilai, memberikan keseimbangan antara keduanya. Persamaan matematika dapat dijelaskan pada persamaan berikut untuk menentukan *F1-Score*.

$$F1\ Score = \frac{2 \times precision \times Recall}{Precision + Recall}$$

4. Backward Transfer dan Forward Transfer

Pada konteks pembelajaran berkelanjutan untuk mengukur evaluasi model pada kasus *forgetting* merujuk pada sumber [8] kelupaan dapat diukur menggunakan metrik *backward transfer* (BWT). BWT mengukur dampak mempelajari tugas baru terhadap kinerja pada tugas-tugas sebelumnya. Ini

mencerminkan seberapa baik model mempertahankan pengetahuan yang diperoleh sebelumnya setelah mempelajari informasi baru. Jika mengacu pada *FI-Score* persamaan matematika untuk menentukan BWT adalah:

$$BWT_i = FI_{i, Baru} - FI_{i, Awal}$$

Persamaan FWT untuk tugas ke-i, untuk mencari rata rata BWT yang sudah dilakukan dapat dirumuskan:

$$FWT_i = FI_{i, Baru} - FI_{i, Awal}$$

Maka dapat dijelaskan bahwa:

BWT/FWT = positif: Mempelajari tugas baru meningkatkan kinerja pada tugas ke-i. Jika negatif maka model terjadi kelupaan

BWT/FWT = Nol : Mempelajari tugas baru tidak berdampak pada kinerja tugas ke-i.

Penelitian ini akan memilih metrik BWT untuk melihat performa pada dataset testing *baseline* berguna untuk melihat apakah model mempertahankan pengetahuan sebelumnya dan FWT berguna untuk melihat performa apakah model belajar pada pengetahuan yang baru menggunakan dataset testing baru.

2.2.13. User Acceptance Testing

User Acceptance Testing (UAT) adalah fase akhir dalam proses pengujian perangkat lunak di mana pengguna akhir atau pemangku kepentingan menguji perangkat lunak untuk memastikan bahwa sistem memenuhi kebutuhan bisnis dan siap untuk digunakan secara operasional [37]. UAT berfokus pada validasi fungsionalitas, kegunaan, dan keandalan perangkat lunak dari perspektif pengguna dalam skenario dunia nyata [38]. Ini memastikan bahwa perangkat lunak dapat menangani tugas dan proses yang diperlukan sebelum diterapkan ke lingkungan produksi.

2.3. Hipotesis

Penelitian ini tujuan untuk melakukan perbaikan performa pada model yang

sudah ada sebelumnya dan terinspirasi dari hal yang bersifat berkelanjutan agar model dapat memiliki pengetahuan sesuai kebutuhan. Dengan menggunakan metode *replay* untuk melatih model *machine learning* diharapkan memiliki hasil performa yang baik dan tidak mengalami masalah penurunan performa pada pengetahuan sebelumnya. Model ML yang dikembangkan diharapkan dapat diimplementasi pada alat pemilahan sampah otomatis seperti konveyor berjalan atau robot canggih yang dapat melakukan pemilahan.

BAB III

METODE PROYEK AKHIR

3.1. Alat dan Bahan

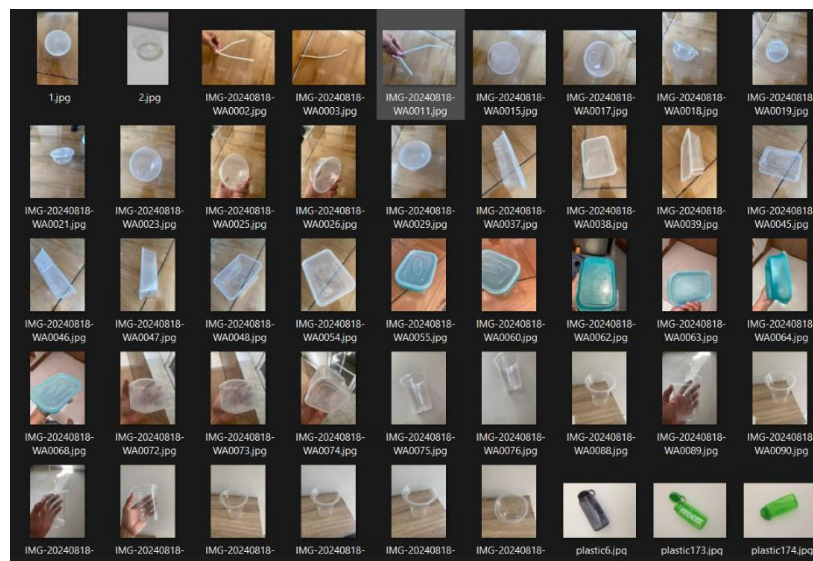
3.1.1. Bahan

1. Sumber Kode YOLO

Sumber kode YOLO yang dikembangkan oleh tim *ultralytics*. Pustaka *ultralytics* ini merupakan kumpulan tools yang dapat membantu pengguna untuk menggunakan model YOLO dengan mudah. Sumber kode: <https://github.com/ultralytics/ultralytics/tree/main/ultralytics>

2. Gambar

Gambar objek yang akan dilakukan untuk melakukan penelitian menggunakan sampah plastik yang ada di lingkungan.



Gambar 3.1 Kumpulan gambar sampah plastik

3.1.2. Peralatan

1. Perangkat keras

Satu buah laptop, spesifikasi dapat dilihat pada tabel.

Laptop	Spesifikasi	OS
<i>Hewlett-Packard Pavillion</i> (HP) 14	8 GB RAM, Intel i5 1135G7 @ 2.40GHz (8 CPUs), 500GB SSD,	Windows 11

	Nvidia MX 450 2GB memory	
--	--------------------------	--

2. Perangkat lunak

Penelitian ini menggunakan beberapa perangkat lunak beserta dengan pustaka yang dapat dilihat pada table berikut.

No	Nama	Keterangan	Versi
1.	Windows OS	Sistem operasi	Windows 11
2.	Python	Bahasa pemrograman	3.11
3.	Flask	Kerangka kerja untuk pengembangan backend dan frontend	3.0.3
4.	Xampp	Perangkat lunak untuk mengakses database Mysql	3.3.0
5.	Mysql	Sistem manajemen basis data	15.1
6.	Google Chrome	Perangkat lunak untuk peramban web	128
7.	Visual Studio Code	Perangkat lunak penyunting kode	1.93.1
8.	Git	Sistem kontrol versi pada proyek	2.43

3. Pustaka

Berikut adalah pustaka python untuk penunjang proyek aplikasi berbasis web ini:

No.	Nama	Keterangan	Versi
1.	numpy	Pustaka fundamental untuk komputasi numerik di Python	1.26.4
2.	pandas	Alat analisis dan manipulasi data	2.2.2
3.	matplotlib	Pustaka visualisasi data	3.9.2
4.	pyYAML	Pustaka untuk membaca dan menulis file	6.0.2

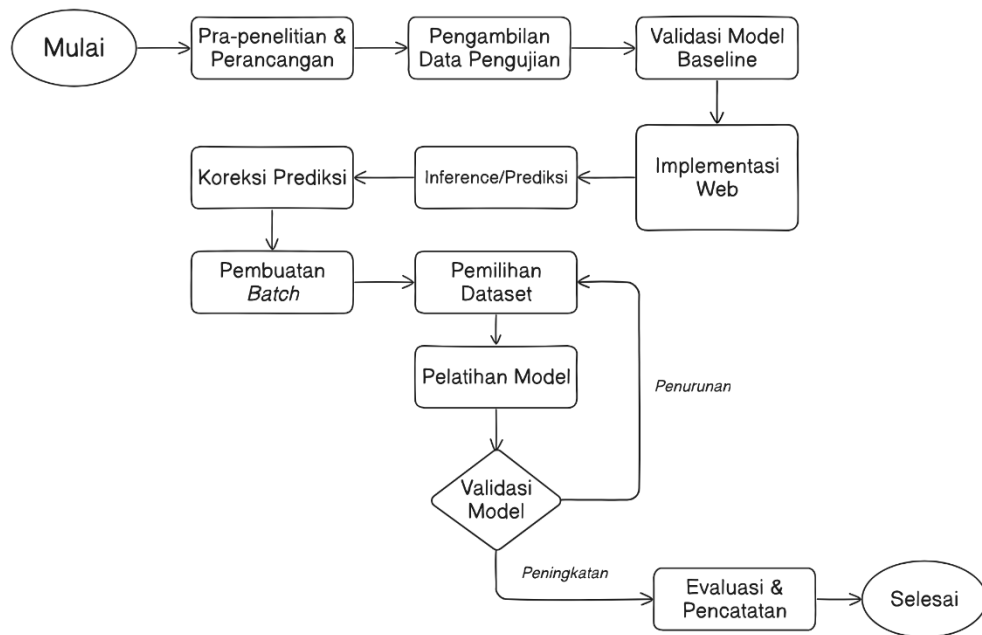
		YAML	
5.	gunicorn	Server WSGI HTTP yang digunakan untuk menjalankan aplikasi web Python dalam lingkungan produksi	23.0
6.	mysql-connector-python	onektor database untuk menghubungkan aplikasi Python dengan database MySQL	9.0
7.	tqdm	Pustaka untuk membuat progress bar yang berguna untuk memantau kemajuan operasi yang memakan waktu	4.66.5

Pustaka digunakan untuk menunjak pemrosesan machine learning pada proyek ini dapat dilihat pada table dibawah ini:

No	Nama	Keterangan	Versi
1.	torch	Pustaka pembelajaran mendalam	2.3.1
2.	torchvision	Pustaka yang menyediakan dataset, model, dan transformasi gambar.	0.18.1
3.	tensorflow	Kerangka kerja pembelajaran mesin	2.17
4.	opencv-python	Pustaka visi computer untuk lingkungan pemrograman python	4.10

3.2. Tahapan Proyek Akhir

Tahapan penelitian proyek akhir ini memiliki serangkaian tahapan. Mulai dari pra-penelitian, pengambilan data pengujian, melakukan pengujian model *baseline*, implementasi website, prediksi, pengumpulan data hasil prediksi, pemilihan dataset, pelatihan dengan *multi dataset*, hingga evaluasi. Serangkaian tahapan akhir tersebut dapat dilihat gambar 3.2 pada diagram alur berikut:



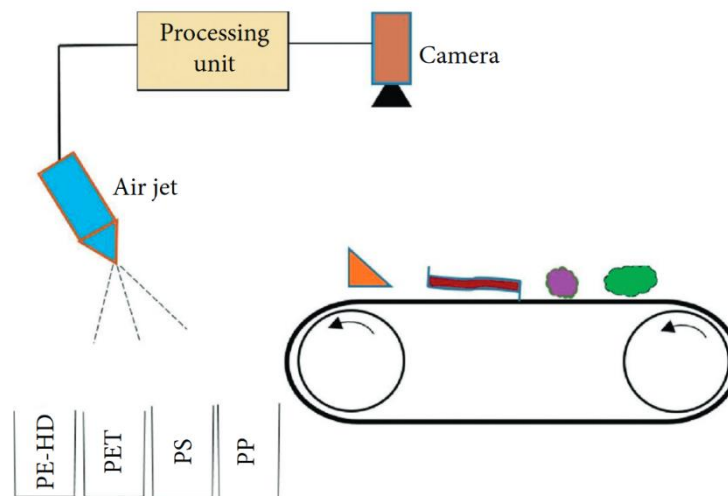
Gambar 3.2 Tahapan Proyek Akhir

3.2.1. Pra-penelitian

Tahap pra-penelitian dan perancangan mencakup serangkaian proses yang dimulai dari analisis permasalahan, studi literatur, penentuan metode yang sesuai, perancangan skenario penelitian, dan analisis pengembangan web.

Hasil analisis permasalahan menunjukkan bahwa model YOLOv8n-cls mengalami penurunan performa pada dataset pengujian WaDaBa [39] setelah dilakukan pelatihan ulang. Fenomena ini dikenal sebagai catastrophic forgetting, yaitu kondisi di mana model kehilangan pengetahuan yang telah dipelajari sebelumnya. Ultralytics [7], selaku pengembang YOLOv8, menyarankan penggunaan pendekatan replay/rehearsal sebagai solusi untuk mengatasi masalah tersebut. Pelatihan ulang diperlukan karena model belum mampu memprediksi jenis sampah plastik secara akurat dalam kondisi dunia nyata yang dinamis. Berdasarkan permasalahan tersebut, proyek akhir ini mengusulkan pengembangan aplikasi web yang dapat memantau hasil prediksi model dan melakukan pelatihan ulang menggunakan metode replay.

Studi literatur mengenai klasifikasi tipe sampah plastik atau tipe resin plastik berbasis *deep learning* seperti [3] mengusulkan sistem untuk pemilahan sampah plastik otomatis seperti yang ada pada Gambar 3.3.



Gambar 3.3 Sistem pemilahan sampah plastik yang diusulkan [3]

Penelitian tersebut mengimplementasikan sistem pemilahan sampah otomatis yang mengintegrasikan model *deep learning* dengan unit pengontrol. Sistem terdiri dari beberapa komponen utama: konveyor (ban berjalan) sebagai media transportasi sampah, kamera untuk menangkap citra objek, unit pemrosesan untuk analisis citra, dan *air jet* (semprotan udara) sebagai aktuator pemilah. Sampah plastik yang bergerak di atas konveyor akan melewati area pengamatan kamera, di mana citra yang ditangkap akan diproses oleh model untuk mengklasifikasikan jenis sampah (HDPE, PET, PS, atau PP). Berdasarkan hasil klasifikasi tersebut, sistem pengontrol akan mengaktifkan *air jet* pada waktu yang tepat untuk mengarahkan sampah ke wadah yang sesuai dengan jenisnya.

Penelitian ini berfokus pada pengembangan sistem yang menitikberatkan pada permasalahan performa model. Dalam skenario proyek ini, model akan diimplementasikan (*deployed*) pada sebuah platform web untuk memungkinkan pengguna mengklasifikasikan jenis-jenis sampah plastik. Mengacu pada Gambar 3.3, ruang lingkup penelitian ini dibatasi pada komponen kamera dan unit pemrosesan (perangkat lunak). Pembatasan ini bertujuan untuk memfokuskan proyek pada pemantauan hasil prediksi model dan upaya peningkatan performa model agar dapat menghasilkan prediksi jenis sampah plastik yang akurat dan andal.

3.2.2. Pengambilan Data dan Uji Model Baseline

Setelah merancang skenario model, dilakukan pengambilan data untuk pengujian. Data ini berupa gambar objek dari dunia nyata yang digunakan dalam dua komponen, yaitu sebagai dataset testing dan dataset *batch*. Data task berfungsi sebagai aliran data masukan untuk diprediksi, sedangkan dataset *testing* digunakan untuk mengevaluasi performa model. Tahap testing model baseline penting untuk mengetahui performa awal model sebelum pelatihan lebih lanjut.

3.2.3. Implementasi Web

Setelah performa awal model diketahui, dilakukan pengembangan website. Fungsi dari web ini adalah untuk memonitoring hasil prediksi model, melakukan anotasi dari data yang salah, membuat *replay buffer*, pemilihan data pelatihan, pelatihan model *machine learning*, melakukan testing, dan pencatatan evaluasi model. Dalam perancangannya, web ini didesain untuk memenuhi skenario metode *replay* untuk melakukan pelatihan pada model *machine learning*.

3.2.4. Prediksi dan Koreksi Data

Pada tahap ini melakukan prediksi dan melakukan anotasi label baru pada data prediksi yang salah. Data yang diprediksi oleh model kemudian dievaluasi dan ditinjau oleh pengguna. Pada tahap inilah pengguna menentukan apakah hasil prediksi benar atau salah. Proses validasi pengguna sangat dibutuhkan pada tahap ini. Proses tinjauan oleh pengguna masih menggunakan cara manual, yaitu dengan melihat gambar prediksi secara satu-persatu. Setelah selesai pengguna dapat menentukan kebenaran dari suatu prediksi. Jika prediksi salah pengguna dapat mengoreksi prediksi ke anotasi yang benar. Contoh hasil prediksi kelas PP, tetapi sebenarnya adalah kelas PS, user dapat mengubah data tersebut menjadi label PS pada opsi yang ada pada tampilan.

3.2.5. Data Batch

Data *batch* merupakan data hasil koreksi pengguna yang dikumpulkan. Data yang terkumpul akan disebut sebagai *batch*. Pengguna memiliki kebebasan untuk merekap data dari rentan waktu tertentu. *batch* secara adalah bagian dari

penyimpanan *replay buffer*. Pengguna bertanggung jawab untuk menghasilkan *replay buffer* yang nantinya digunakan dalam proses pelatihan. Pada tahap ini, pengguna juga dapat melakukan augmentasi data. Jika jumlah data dalam *batch* atau *replay buffer* terlalu besar dan dikhawatirkan meningkatkan beban komputasi saat pelatihan, pengguna dapat menerapkan random sampling untuk memilih data secara acak dari *replay buffer* tersebut.

3.2.6. Pemilihan dataset dan Pelatihan Model

Setelah dataset tergenerate pada proses *batching*, dataset dipilih sesuai dengan kebutuhan. Dataset dapat dipilih lebih dari 1 atau *multi-dataset* guna melakukan proses pelatihan dengan metode *replay*. Pada proses pelatihan dengan menggunakan *multi-dataset* pada proyek akhir ini melakukan penggabungan dataset. Proses penggabungan dilakukan berguna untuk menyesuaikan format direktori dari cara fungsi memulainya sebuah pelatihan model *machine learning*.

3.2.7. Testing dan Evaluasi Metrik

Setelah proses *training* selesai model akan melakukan proses testing dengan dataset testing *baseline* dan set data testing baru. Dataset testing merupakan sebuah acuan mulai dari proses uji performa model baseline, hingga nanti pelatihan model ke-i. Pada penelitian dan proyek akhir ini, akan mengevaluasi model menggunakan metrik *F1-score* dan *Backward-Transfer* (BWT).

3.3. Usulan Rancangan Monitoring dan Metode Replay

Pada upaya untuk melakukan peningkatan sebuah model *machine learning* dengan menggunakan metode *replay*. Berikut ini merupakan uraian mengenai metode dan skenario pada penelitian ini:

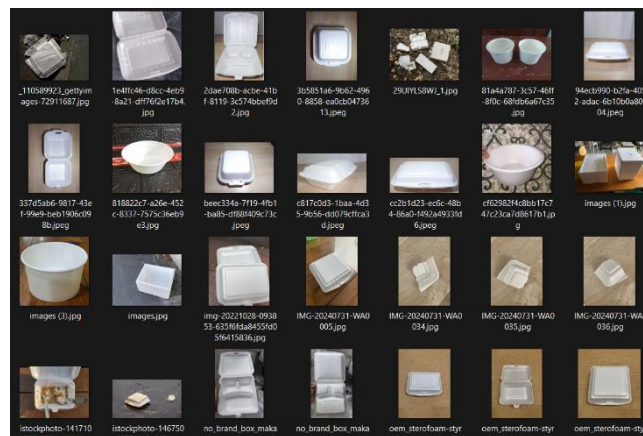
Dalam penelitian ini, tahap prediksi dirancang untuk mensimulasikan penerapan model *machine learning* (ML) pada aplikasi pendeteksian objek. Sistem yang dikembangkan berfungsi sebagai platform pemantauan hasil prediksi model ML. Hasil prediksi akan direkam dan dievaluasi oleh pengguna untuk mengidentifikasi kesalahan prediksi (*error*). Data kesalahan prediksi yang terkumpul selanjutnya akan digunakan untuk melakukan pelatihan ulang pada model yang telah ada. Proses pelatihan ulang ini menerapkan metode *replay*, yang

bertujuan untuk mengatasi permasalahan *catastrophic forgetting* (hilangnya pengetahuan yang telah dipelajari sebelumnya).

Berikut ini merupakan uraian dari beberapa tahapan skenario yang akan dilakukan.

3.3.1. Data survei dan Distribusi data

Penelitian ini menggunakan data survei yang menunjukkan beragam bentuk objek dan kondisi yang dinamis. Objek-objek tersebut dapat dilihat pada Gambar 3.4 berikut:



Gambar 3.4 Data survei

Pengumpulan data survei dilakukan dengan mengacu pada empat kelas klasifikasi yang ada pada model sebelumnya, yaitu:

1. High Density Polyethylene (HDPE)
2. Polyethylene Terephthalate (PET)
3. Polypropylene (PP)
4. Polystyrene (PS)

Total data survei yang berhasil dikumpulkan mencapai 811 gambar objek. Data tersebut kemudian dibagi menjadi tiga bagian yaitu data pengujian (testing), *task 1*, dan *task 2*.

Evaluasi performa model dilakukan menggunakan dua dataset pengujian: dataset dasar (*baseline*) dan dataset testing baru. Dataset *baseline* menggunakan dataset WaDaBa [37] yang sebelumnya digunakan untuk melatih model *baseline*. Dataset WaDaBa secara lengkap memuat tujuh label resin plastik (HDPE, PET, PP,

PS, LDPE, PVC, dan OTHER), namun model awal hanya menggunakan empat label tipe plastik (HDPE, PET, PP, PS). Contoh gambar dari dataset WaDaBa dapat dilihat pada Gambar 3.5.



Gambar 3.5 Sampel gambar dari basis data WaDaBa [39]

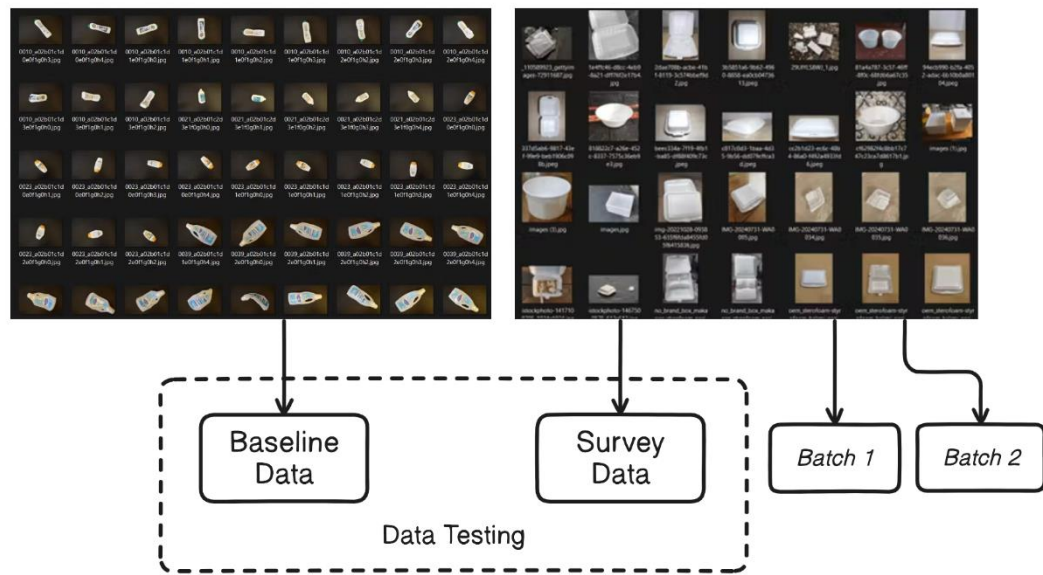
Data pengujian dasar (baseline testing data) terdiri dari data yang tidak digunakan dalam pengujian model dasar, sehingga menjamin objektivitas evaluasi model. Penggunaan dataset pengujian dasar bertujuan untuk mendeteksi fenomena *forgetting* (hilangnya pengetahuan sebelumnya) pada model. Dataset ini memiliki total 277 gambar dengan distribusi sebagai berikut:

1. HDPE : 69 gambar
2. PET : 132 gambar
3. PP : 28 gambar
4. PS : 48 gambar

Sementara itu, data pengujian baru digunakan untuk mengukur tingkat pembelajaran (*learning rate*) model. Dataset baru ini terdiri dari 401 gambar dengan distribusi:

1. HDPE : 149 gambar
2. PET : 103 gambar
3. PP : 87 gambar
4. PS : 62 gambar

Visualisasi pembagian data *task* dapat dilihat pada Gambar 3.6 berikut.:



Gambar 3.6 Distribusi Data

Data *task* merupakan kumpulan data yang diasumsikan sebagai pengetahuan baru. Data ini terbagi menjadi dua kelompok:

Batch 1 terdiri dari 202 data dengan distribusi sebagai berikut:

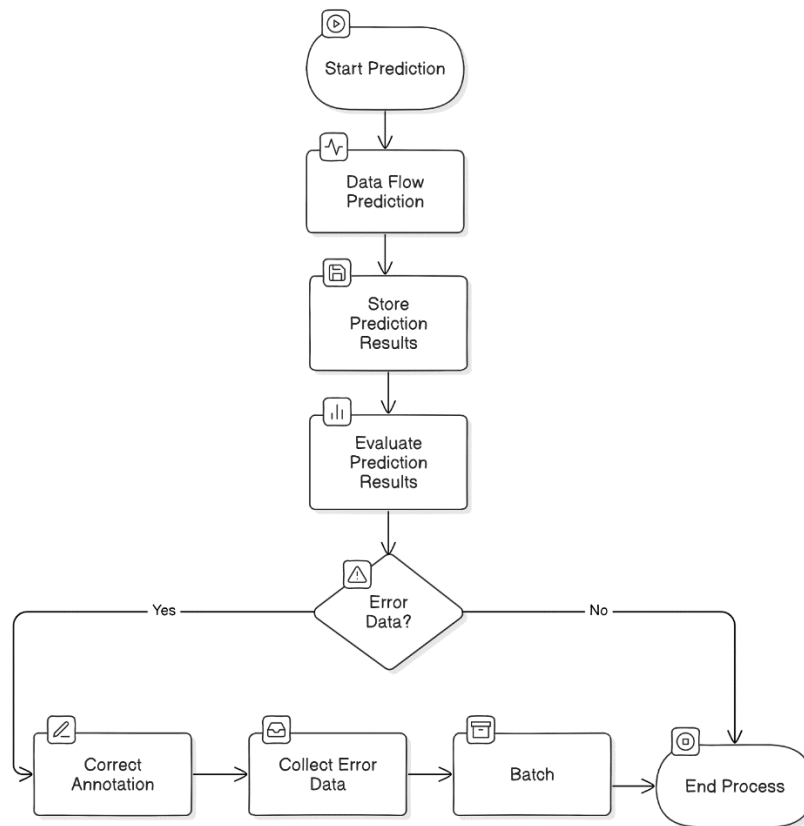
1. HDPE : 49 gambar
2. PET : 52 gambar
3. PP : 60 gambar
4. PS : 41 gambar

Batch 2 memiliki 208 data dengan distribusi:

1. HDPE : 81 gambar
2. PET : 20 gambar
3. PP : 41 gambar
4. PS : 66 gambar.

3.3.2. Tahap Prediksi dan PembuatanBatch

Proses prediksi dan evaluasi model dilaksanakan dalam beberapa tahap yang terstruktur, seperti yang diilustrasikan pada Gambar 3.5. Tahapan tersebut meliputi:..



Gambar 3.7 Flowchart proses prediksi hingga kumpulan koreksi

1. *Start Prediction*: Proses dimulai dengan tahap prediksi, di mana data batch diproses secara sekuensial untuk melakukan inferensi.
2. *Data Flow Prediction*: Model melakukan prediksi terhadap setiap data yang dimasukkan.
3. *Store Prediction Results*: Hasil prediksi disimpan dalam sistem untuk proses evaluasi selanjutnya.
4. *Evaluate Prediction Results*: Hasil prediksi dievaluasi untuk mengidentifikasi kesalahan prediksi.
5. Pada tahap *Error Data*, jika ditemukan kesalahan prediksi, sistem akan melanjutkan ke proses:
 - *Correct Annotation*: Pengguna melakukan koreksi dengan memberikan anotasi yang benar
 - *Collect Error Data*: Data yang telah dikoreksi dikumpulkan

- *Batch*: Data kesalahan prediksi yang telah dikoreksi disimpan dalam satu kelompok (*batch*)
6. Jika tidak ditemukan kesalahan, atau setelah proses koreksi selesai, proses akan berakhir (*End Process*).

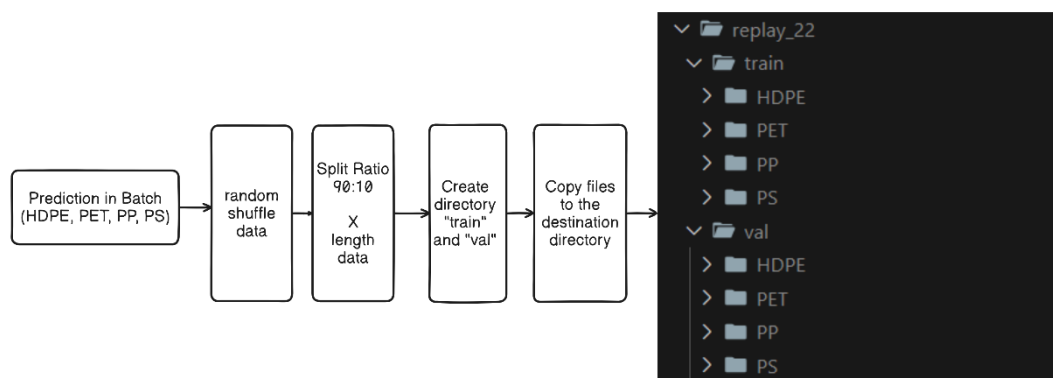
Batch dalam penelitian ini berfungsi sebagai wadah penyimpanan data hasil koreksi beserta identitasnya. Setelah data terkumpul dalam batch, sistem akan menghasilkan (*generate*) struktur direktori pelatihan yang terdiri dari dua folder utama: train dan val (validasi). Masing-masing folder tersebut memuat empat kelas sesuai dengan tipe plastik yang diklasifikasikan:

1. *High Density Polyethylene* (HDPE)
2. *Polyethylene Terephthalate* (PET)
3. *Polypropylene* (PP)
4. *Polystyrene* (PS)

Data dari batch akan didistribusikan secara acak ke dalam folder train dan val dengan mempertahankan kesesuaian identitas kelasnya. Pembagian data menggunakan rasio 90:10, di mana:

- 90% data digunakan untuk pelatihan (train)
- 10% data digunakan untuk validasi (val)

Gambar 3.8 berikut adalah visualisasi tingkat tinggi dalam melakukan distribusi data *batch* kedalam format dataset *training*:



Gambar 3.8 *Generate* prediksi di *batch* menjadi dataset

Proses *generate* data *batch* terdiri dari beberapa tahapan sekuensial yang terstruktur:

1. *Prediction in Batch*

- Tahap awal dimulai dengan mengumpulkan data hasil prediksi yang telah dikoreksi
- Data dikelompokkan sesuai dengan empat kelas utama: HDPE, PET, PP, dan PS
- Data yang terkumpul dalam batch sudah memiliki label yang terverifikasi

2. *Random Shuffle Data*

- Data dalam batch diacak (shuffle) secara sistematis
- Pengacakan dilakukan untuk memastikan distribusi data yang merata
- Proses ini penting untuk menghindari bias dalam pelatihan model

3. *Split Ratio 90:10*

- Data yang telah diacak kemudian dibagi dengan rasio 90:10
- 90% data dialokasikan untuk pelatihan (train)
- 10% data dialokasikan untuk validasi (val)
- Pembagian dilakukan berdasarkan panjang total data (length data)

4. *Create Directory*

- Sistem membuat struktur direktori baru
- Dua folder utama dibuat: 'train' dan 'val'
- Dalam masing-masing folder dibuat empat subfolder sesuai kelas (HDPE, PET, PP, dan PS)

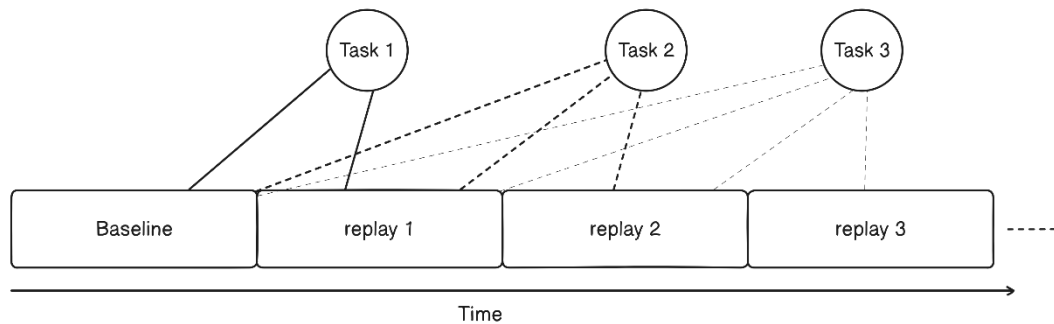
5. *Copy Files to Destination Directory*

- Data dipindahkan ke dalam struktur folder yang telah dibuat
- File-file gambar disalin ke subfolder yang sesuai dengan kelasnya
- Hasil akhir terlihat pada tampilan struktur folder di sebelah kanan gambar
- Direktori 'replay' menjadi wadah utama yang menampung seluruh data

Hasil implementasi terlihat pada tampilan struktur folder di sebelah kanan, yang menunjukkan organisasi data yang terstruktur dan siap digunakan untuk proses pelatihan model selanjutnya.

Dataset yang dihasilkan dari algoritma tersebut dapat digunakan untuk melatih model ke- i dan ke $i+1$. Dengan kata lain setiap direktori *replay* yang terbuat

dapat digunakan dalam suatu pelatihan untuk melakukan pelatihan dengan metode *replay* untuk mengatasi *catastrophic forgetting*. Gambar 3.9 menjelaskan bagaimana penggunaan *replay buffer*.



Gambar 3.9 Contoh penggunaan *replay buffer* pada setiap penugasan

Pada penelitian ini untuk mencapai nilai hasil *F1-score* yang paling maksimal. Penelitian ini berinisiatif menambahkan variabel *augmentation* pada data *batch* dan meneliti dampak penggunaan *augmentation* pada pengetahuan model, baik dalam mempertahankan pengetahuan sebelumnya atau baru. Data akan di-augmentasi dengan pengaturan +70% kecerahan, -70% kecerahan, dan +40% blur.

3.3.3. Pelatihan Model

Class YOLO dari Ultralytics sudah memiliki fungsi untuk pelatihan. Fungsi ini dapat menangani beberapa preferensi termasuk argumen iterasi (*epoch*), ukuran gambar atau *imgsz*, *learning rate*, *treshold*, dan *freeze* untuk membekukan lapisan *neural network* dengan begitu pengembang tidak perlu melakukan banyak hal untuk melakukan pelatihan yang diinginkan. Gambar 3.10 berikut ini adalah contoh kode untuk melakukan pelatihan menggunakan model *pre-trained* YOLO.

```
from ultralytics import YOLO
# Load a model
model = YOLO("yolo8n.pt")
# Train the model
results = model.train(data="coco8.yaml", epochs=100, imgsz=640)
```

Gambar 3.10 Kode pelatihan model YOLO

Training dengan metode transfer learning

Sebelum masuk ke dalam metode *training* menggunakan metode *replay*, penelitian ini akan membandingkan hasil performa dengan metode transfer learning dan *replay*. Metode *transfer learning* berfokus pada pemindahan pengetahuan antar domain, distribusi data, dan tugas yang berbeda untuk meningkatkan kinerja pembelajaran dalam domain target dengan memanfaatkan informasi dari domain sumber yang terkait [40].

Pada penelitian ini metode *transfer learning* tidak melakukan argumen *freeze*, yang berarti seluruh bobot di lapisan model akan diperbarui pada model YOLO *baseline*. Bobot nantinya akan diperbarui dengan dataset *batch* yang sudah dihasilkan. Dengan adanya pembaharuan bobot pada layer dengan pengetahuan yang baru dengan demikian model tersebut telah melakukan *transfer learning*.

Training dengan metode replay

Perbedaan yang mendasar pada metode *replay* dari *transfer learning* biasa yaitu pada pengguna datanya. Dalam konteks *continual learning*, *replay* adalah teknik di mana model mempertahankan sejumlah data dari tugas atau pengalaman sebelumnya disebut *episodic memory* dan menggunakan data tersebut selama pelatihan pada tugas baru untuk mencegah *catastrophic forgetting* [9].

Pada penelitian ini metode *replay* menggunakan dataset *batch* sebagai memori episodik untuk mempertahankan *domain* atau pengetahuan sebelumnya. Sebagai contoh pada *task 1* nantinya akan menggunakan sedikit dataset *baseline* (data yang sudah pernah dilatih) dan ditambah dataset *batch 1* untuk melakukan pelatihannya.

fungsi pelatihan *Ultralytics* disesuaikan agar mendukung penggunaan beberapa dataset sekaligus atau *multi-dataset*. Untuk memulai pelatihan, pengguna perlu memilih dataset yang dihasilkan pada tahap *batch*. Fungsi pelatihan agar dapat menggunakan *multi-dataset* bermaksud untuk melakukan *pre-processing* sebelum melakukan *training* atau seperti penggabungan dari beberapa dataset. Dalam penelitian ini menggunakan model YOLOv8 dari *Ultralytics*. Dengan begitu proses *training* menggunakan *multi-dataset* dapat dilakukan untuk memenuhi

skenario *replay* pada penelitian ini. Contoh visualisasi modifikasi pada fungsi yang dimaksud dapat dilihat pada Gambar 3.11.

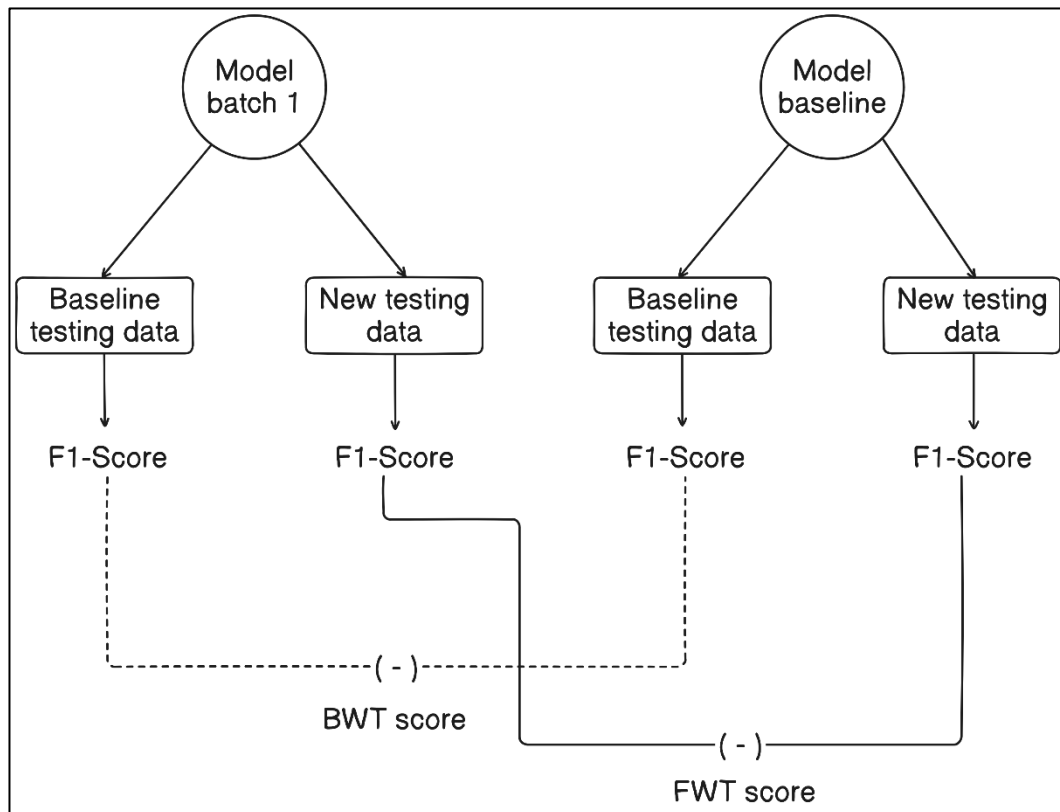
Train(data= [Dataset Dataset Dataset])

Gambar 3.11 Visualisasi modifikasi fungsi *training* pada pustaka *Ultralytics*

Keuntungan dari modifikasi fungsi pelatihan tersebut berguna untuk meminimalisir penggunaan penyimpanan. Dengan begitu program tidak perlu memerlukan komputasi untuk menduplikasi beberapa dataset menjadi satu dataset.

3.3.4. Pengujian Model

Proses pengujian model dilakukan menggunakan data uji yang terbagi menjadi dua set, yaitu set data baseline dan set data baru yang berisi pengetahuan baru. Set data *baseline* merupakan data uji yang digunakan dalam pelatihan sebelumnya dan bertujuan untuk mendeteksi adanya penurunan performa pada model baru. Set data baru digunakan untuk memvalidasi apakah model telah mempelajari pengetahuan baru. Hasil pengujian akan diukur menggunakan metrik *f1-score* untuk mengatasi ketidakseimbangan data. Dengan demikian, akan diperoleh dua hasil, yaitu *f1-score* untuk dataset baseline dan *f1-score* untuk dataset baru. Gambar 3.11 menggambarkan metode pengujian pada model



Gambar 3.12 Simulasi testing dan perhitungan model batch 1 dengan model *baseline*

Setiap nilai pengujian akan disimpan untuk dibandingkan dengan model hasil pelatihan ke- $i+1$. Metrik untuk analisis perbandingan performa model penelitian ini menggunakan *Backward Transfer* (BWT) dan *Forward Transfer* (FWT) untuk mengetahui performa pada pengetahuan sebelumnya dan pengetahuan baru.

Berikut ini adalah simulasi perhitungan dengan menggunakan data *dummy* untuk memperdalam pemahaman mengenai hasil testing pada penelitian ini:

1. Data hasil pengujian Model Baseline

- Confusion matrix Dataset Baseline

Tabel 3.1 Confusion matrix data baseline dengan model baseline

		Predicted			
		HDPE	PET	PP	PS
Actual	HDPE	45	3	1	1
	PET	2	42	4	2

	PP	1	3	44	2
	PP	2	1	3	44

- Confusion matrix Dataset Baru

Tabel 3.2 Confusion matrix data baru dengan model baseline

		Predicted			
		HDPE	PET	PP	PS
Actual	HDPE	40	3	3	2
	PET	4	38	5	3
	PP	3	4	39	4
	PP	4	3	5	38

2. Data hasil pengujian Model Batch 1

- Confusion matrix Dataset Baseline

Tabel 3.3 Confusion matrix data baseline dengan model batch 1

		Predicted			
		HDPE	PET	PP	PS
Actual	HDPE	43	4	2	1
	PET	3	41	4	3
	PP	2	4	42	2
	PP	3	2	2	43

- Confusion matrix Dataset Baru

Tabel 3.4 Confusion matrix data baru dengan model batch 1

		Predicted			
		HDPE	PET	PP	PS
Actual	HDPE	42	4	2	2
	PET	3	40	4	3
	PP	2	3	41	4
	PP	3	2	3	42

3. Perhitungan metrik evaluasi

- Model Baseline - Dataset Baseline

HDPE:

$$\text{True Positive (TP)} = 45$$

$$\text{False Positive (FP)} = 5 (2+1+2)$$

$$\text{False Negative (FN)} = 5 (3+1+1)$$

$$\text{Precision} = 45/(45+5) = 0.900$$

$$\text{Recall} = 45/(45+5) = 0.900$$

$$\text{F1-Score} = 2 \times (0.900 \times 0.900)/(0.900 + 0.900) = 0.900$$

PET:

$$\text{TP} = 42$$

$$\text{FP} = 7 (3+3+1)$$

$$\text{FN} = 8 (3+4+1)$$

$$\text{Precision} = 42/(42+7) = 0.857$$

$$\text{Recall} = 42/(42+8) = 0.840$$

$$\text{F1-Score} = 2 \times (0.857 \times 0.840)/(0.857 + 0.840) = 0.848$$

[Perhitungan serupa untuk PP dan PS]

$$\text{Rata-rata F1-Score Dataset Baseline} = 0.866$$

- Model Baseline – Dataset Baru

HDPE:

$$\text{TP} = 40$$

$$\text{FP} = 11 (4+3+4)$$

$$\text{FN} = 10 (5+3+2)$$

$$\text{Precision} = 40/(40+11) = 0.784$$

$$\text{Recall} = 40/(40+10) = 0.800$$

$$\text{F1-Score} = 2 \times (0.784 \times 0.800)/(0.784 + 0.800) = 0.792$$

[Perhitungan serupa untuk kelas lainnya]

$$\text{Rata-rata F1-Score Dataset Baru} = 0.775$$

- Model Batch 1 - Dataset Baseline

[Perhitungan serupa untuk kelas lainnya]

$$\text{Rata-rata F1-Score Dataset Baseline} = 0.848$$

- Model Batch 1 – Dataset Baru

[Perhitungan serupa untuk kelas lainnya]

Rata-rata F1-Score Dataset Baru = 0.828

4. Perhitungan BWT dan FWT

a. Backward Transfer (BWT)

$BWT = F1_baseline(\text{Model batch 1}) - F1_baseline(\text{Model baseline})$

$BWT = 0.848 - 0.866 = -0.018$

Interpretasi: Terjadi sedikit penurunan performa (*forgetting*) sebesar 0.018 pada pengetahuan lama setelah model mempelajari data baru.

b. Forward Transfer (FWT)

$FWT = F1_new(\text{Model batch 1}) - F1_new(\text{Model baseline})$

$FWT = 0.775 - 0.828 = 0.053$

Interpretasi: Model batch 1 menunjukkan peningkatan performa sebesar 0.053 pada data baru dibandingkan model baseline, menunjukkan adanya transfer pengetahuan yang positif. Model batch 1 juga mengalami penurunan performa sebesar

5. Ringkasan Akhir

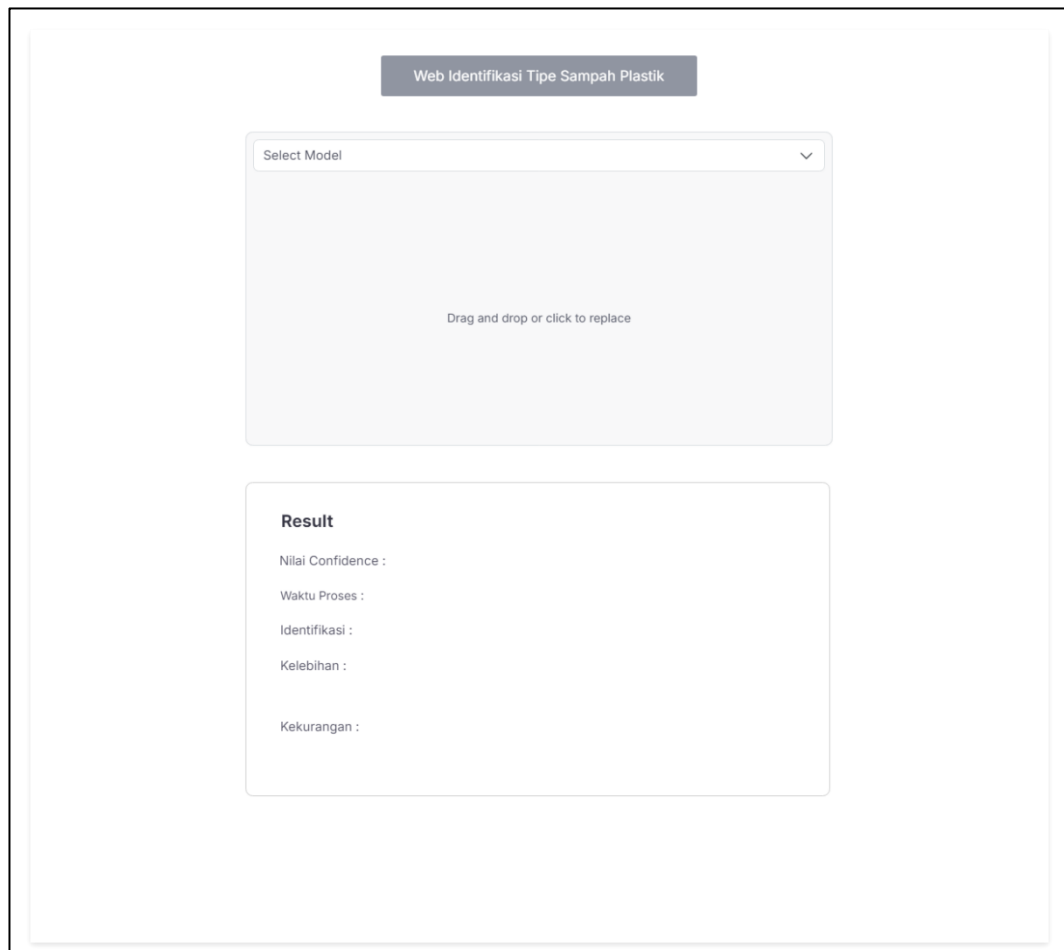
Model	F1-Score Baseline	F1-Score Baru	BWT	FWT
Baseline	0.866	0.775	-	-
Batch 1	0.848	0.828	- 0.018	0.053

3.4. Perancangan Aplikasi Web

Perancangan aplikasi web dalam penelitian ini bertujuan untuk memfasilitasi pengumpulan data, pelatihan model, dan evaluasi kinerja model *machine learning*. Struktur perancangan aplikasi web yang dilakukan adalah antarmuka pengguna, arsitektur sistem, desain basis data, *activity diagram*, *use case diagram*.

3.4.1. Wireframe Antarmuka Pengguna

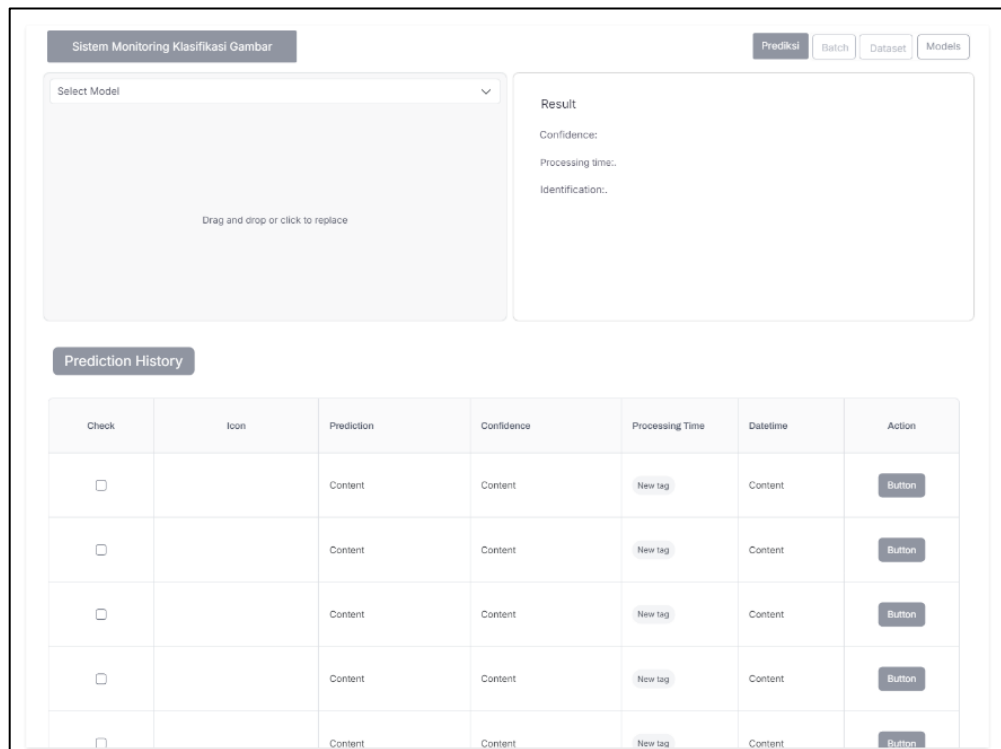
1. Halaman Utama Prediksi



Gambar 3.13 *Wireframe* fitur prediksi utama

2. Halaman Utama Monitoring

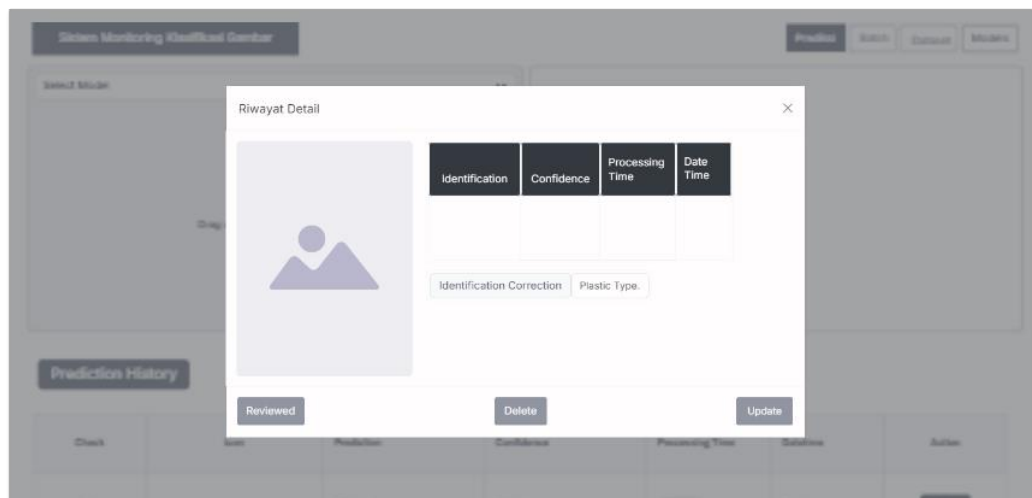
Halaman utama dari web memiliki fungsi yaitu prediksi gambar, hasil prediksi gambar akan tertulis pada kontainer *result*, dan dibawah merupakan tabel dari riwayat prediksi yang sudah dilakukan. Pada Halaman ini pengguna dapat meninjau hasil prediksi dengan menekan button *action*. Halaman utama dapat dilihat pada Gambar 3.14.



Gambar 3.14 Halaman utama

3. Modal meninjau prediksi

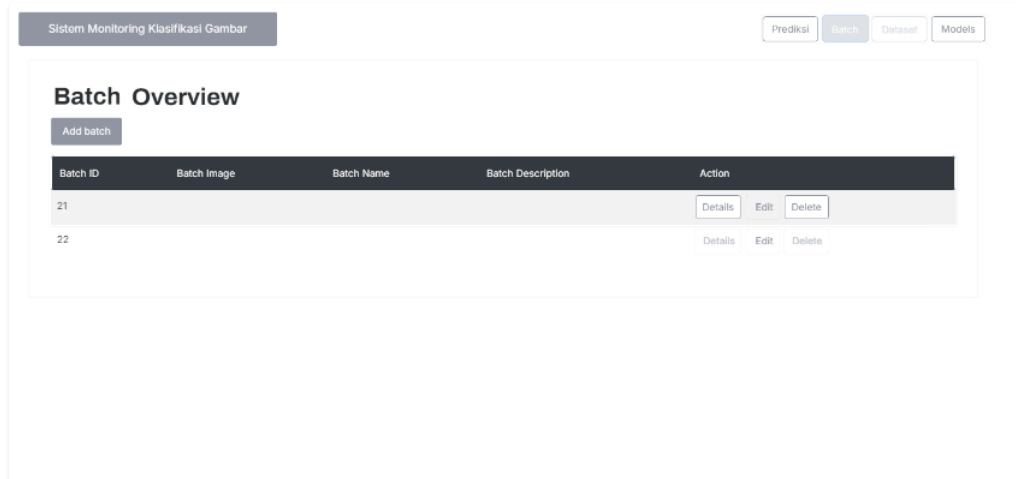
Pada tampilan modal berikut pengguna dapat meninjau prediksi dengan melihat hasil prediksi model dan gambar objek secara besar. Terdapat masukan *dropdown* untuk menganotasi gambar dengan label yang baru. Terdapat 3 tombol *review*, *delete*, *update*.



Gambar 3.15 Tampilan modal tinjauan prediksi

4. Halaman Batch

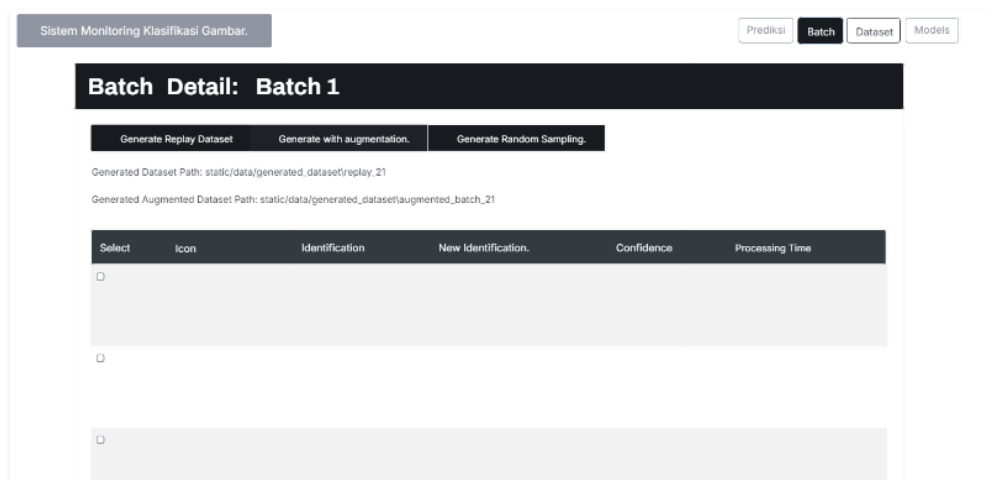
Pada Gambar 3.16 halaman *batch* dibawah ini menampilkan daftar *batch* yang sudah dibuat dan terdapat kolom-kolom untuk menampilkan informasi dari *batch*. Terdapat *action* seperti *detail* untuk meilhat *batch* lebih detail. Kemudian *edit* dan *delete batch* untuk mengedit dan menghapus *batch*.



Gambar 3.16 Halaman daftar *batch*

5. Halaman Detail Batch

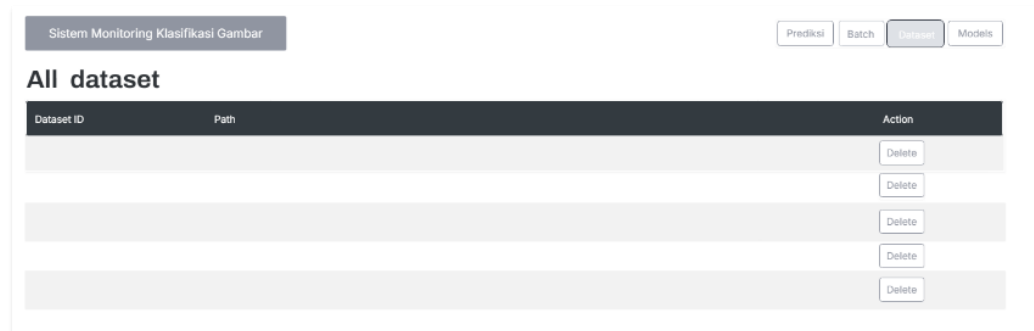
Pada Gambar 3.17 yang menampilkan antarmuka detail dari *batch* terdapat tombol yang digunakan untuk men-*generate* dataset replay, replay dengan augmentasi dan random sampling dari *replay*. Di bagian bawah tampilan terdapat daftar prediksi yang telah dikoreksi pada tahap tinjauan pengguna.



Gambar 3.17 Halaman detail batch

6. Halaman Dataset

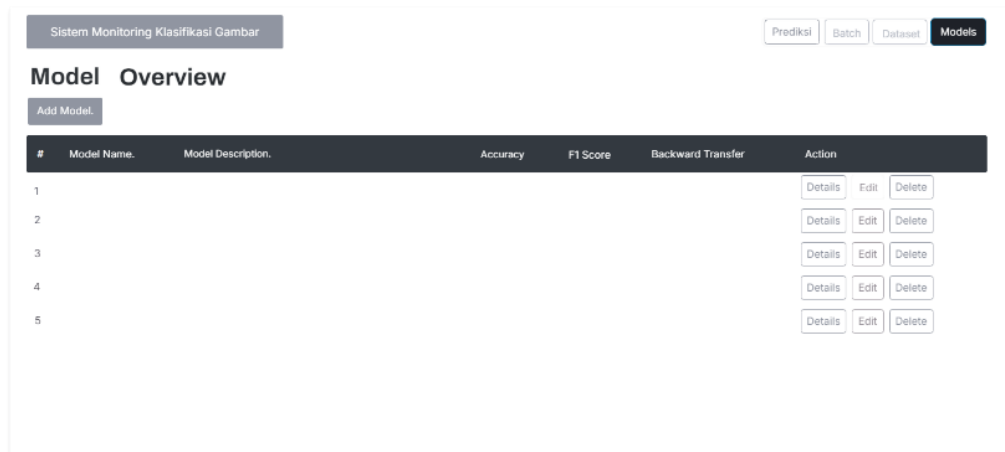
Pada Gambar 3.15 menampilkan antarmuka daftar dataset yang sudah terbuat dari tahapan *batch*. Kolom pada tabel bertujuan untuk dapat mengetahui informasi tentang dataset yang ingin digunakan.



Gambar 3.18 Tampilan daftar dataset

7. Halaman Evaluasi Model

Pada Gambar 3.19 menampilkan antarmuka daftar dari model lengkap dengan metrik evaluasi dari testing. Terdapat 3 tombol untuk melakukan aksi, yaitu *detail* dari model, *edit* model, dan *delete* model. Informasi nama dan deskripsi dari model sangat berguna untuk menandai dengan spesifikasi pelatihan yang ada.

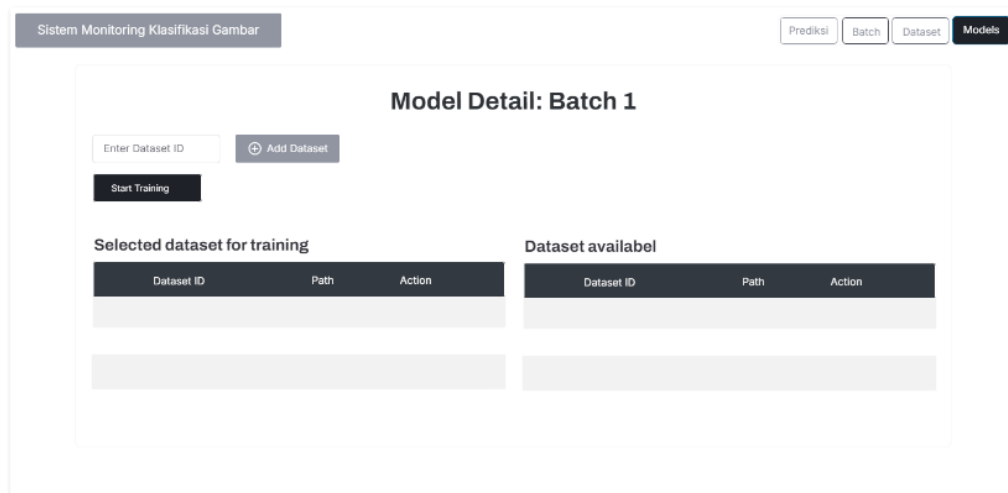


Gambar 3.19 Tampilan daftar evaluasi model

8. Halaman Detail Model

Pada Gambar 3.20 menampilkan antarmuka *detail* model. Pada tampilan detail model terdapat beberapa komponen yaitu masukan id model, tombol tambah model, tombol memulai pelatihan, informasi mengenai dataset yang tersedia dan

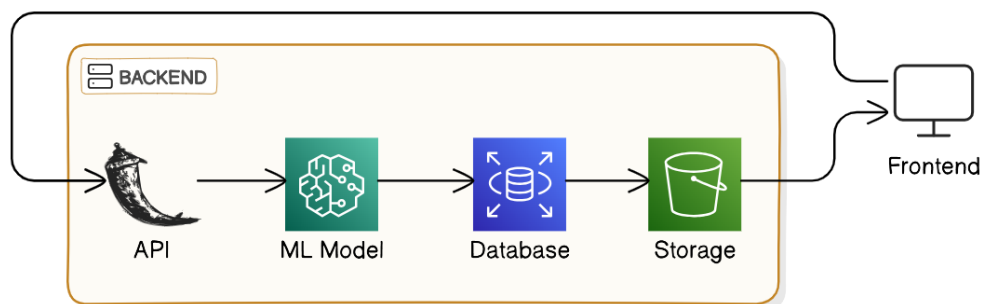
dataset yang terpilih untuk diproses pelatihan.



Gambar 3.20 Tampilan detail dari model

3.4.2. Arsitektur Sistem

Pada Gambar 3.21 menampilkan arsitektur sistem monitoring dan peningkatan sistem pembelajaran mesin. Komponen utama arsitektur ini dimulai dari *Application Programming Interface* (API) berbasis RESTful API, yang bertindak sebagai penghubung antara *frontend* dan *backend*. Melalui API, data dari pengguna atau sistem eksternal dikirimkan ke *backend* untuk diproses. Data tersebut kemudian diteruskan ke ML model, yang merupakan model pembelajaran mesin yang bertugas melakukan inferensi atau prediksi berdasarkan input yang diterima.



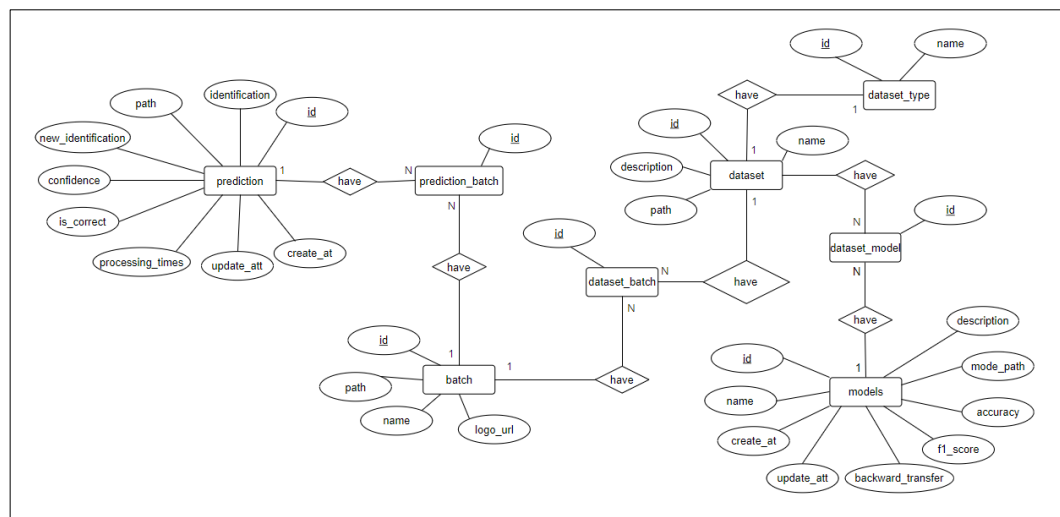
Gambar 3.21 Arsitektur web sistem monitoring dan peningkatan ML

Setelah model menghasilkan hasil prediksi, data disimpan di *Database*, yang berfungsi untuk menyimpan informasi yang relevan seperti hasil prediksi, konfigurasi model, atau data pengguna, dan lain-lain. *Storage* digunakan untuk

menyimpan file yang gambar, seperti dataset. Akhirnya, hasil dari *backend* dikirim kembali ke *Frontend*, yang menampilkan data kepada pengguna melalui antarmuka pengguna.

3.4.3. Desain Database

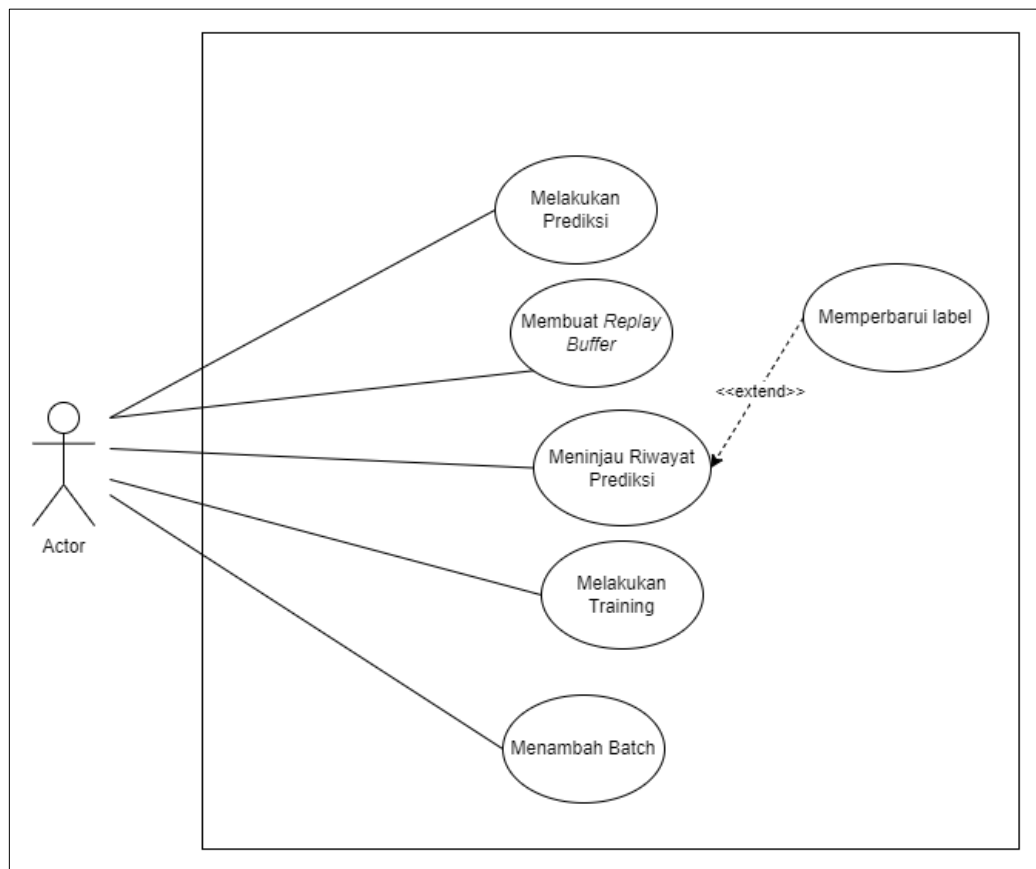
Desain basis data yang digunakan untuk merancang kebutuhan web nantinya dapat dijelaskan pada diagram *chen*. Diagram *chen* yang ditampilkan menjelaskan tentang entitas dan atribut data nya serta menjelaskan hubungan antar entitas. Rancangan *database* divisualisasi pada Gambar 3.22



Gambar 3.22 Diagram *Entity Relationship*

3.4.4. Use Case Diagram

Pada penjelasan berikut ini akan menjelaskan apa saja fitur apa saja yang dapat digunakan oleh pengguna. Pada rancangan *use case* diagram pada sistem ini hanya memiliki 1 aktor. Rancangan *use case* diagram dapat dilihat pada Gambar 3.23.

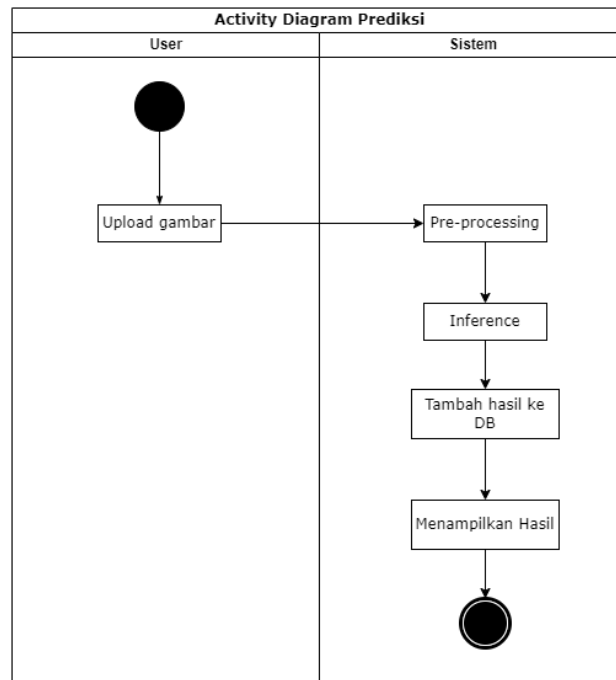


Gambar 3.23 *Use Case Diagram*

3.4.5. Activity Diagram

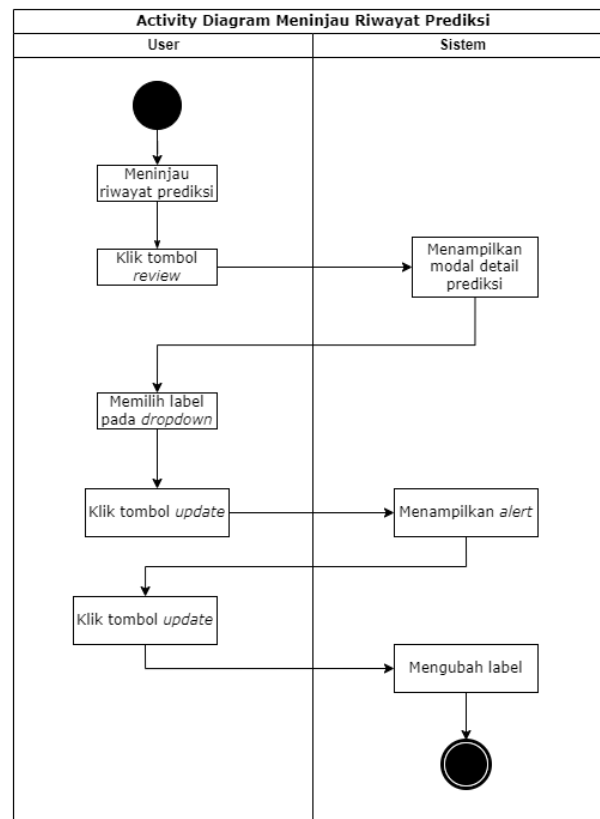
Dalam tahap ini rancangan mengenai aliran penggunaan fungsi jelaskan menggunakan *activity diagram*.

1. Activity Diagram melakukan prediksi



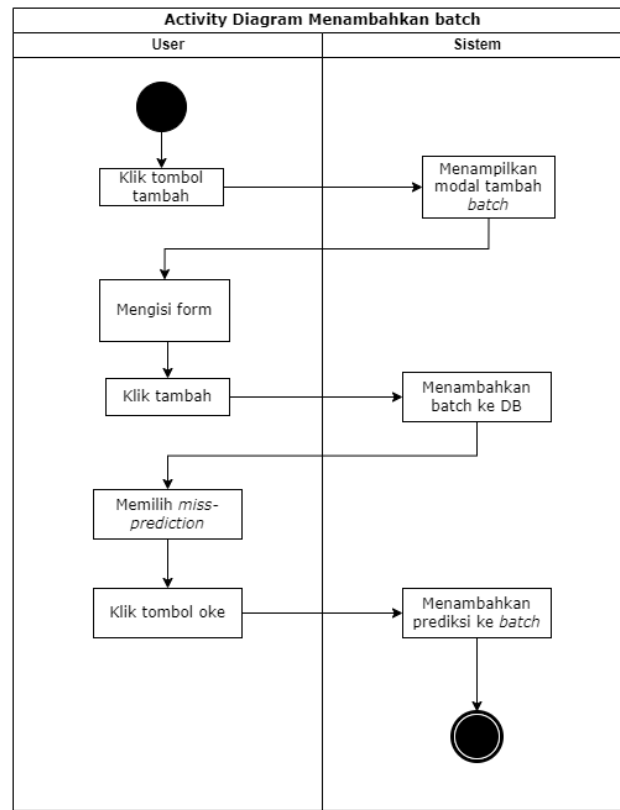
Gambar 3.24 *Activity* diagram melakukan prediksi

2. Activity diagram melakukan tinjauan riwayat prediksi



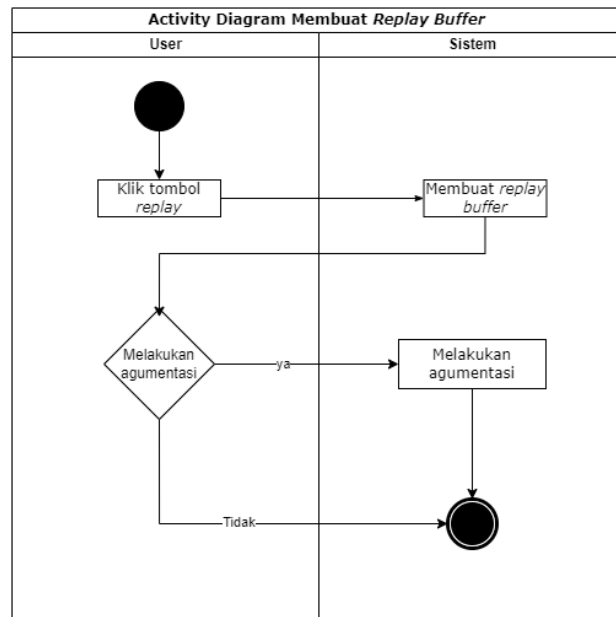
Gambar 3.25 *Activity* Diagram Meninjau Riwayat Prediksi

3. Activity diagram membuat *batch*



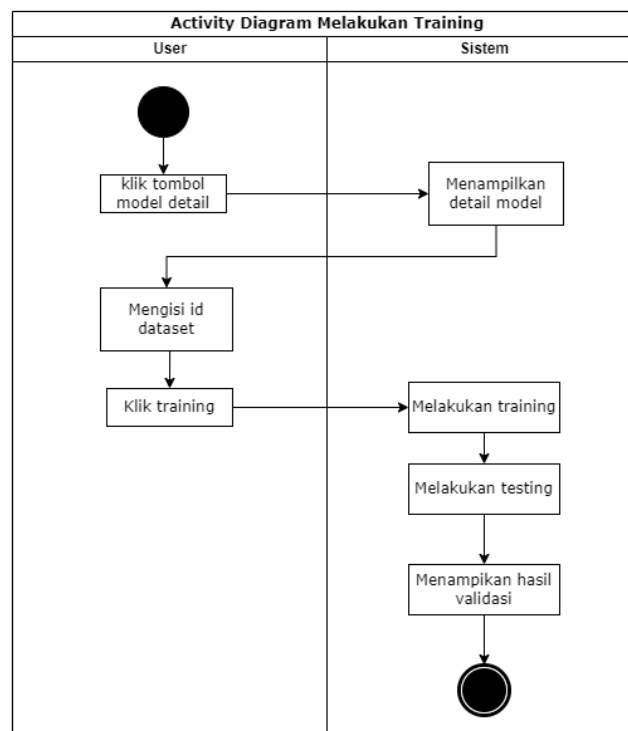
Gambar 3.26 Activity diagram menambahkan batch

4. Activity diagram membuat *replay buffer*



Gambar 3.27 Activity diagram membuat *replay buffer*

5. Activity diagram melakukan *training model*



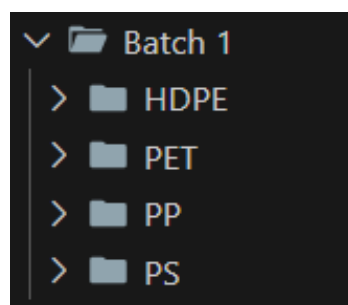
Gambar 3.28 Activity diagram melakukan Training

BAB IV

HASIL DAN PEMBAHASAN

4.1. Persiapan Data

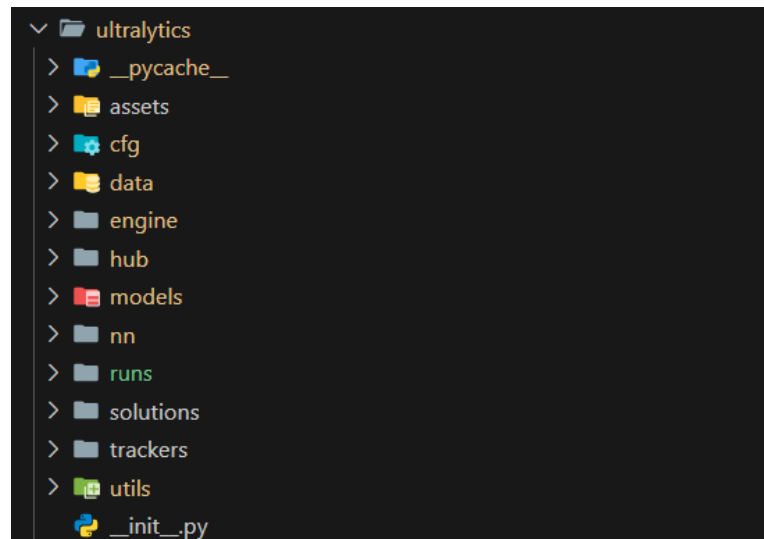
Hasil pembagian data yang dijelaskan pada Gambar 3.6, data terbagi menjadi 4 bagian, yaitu set data testing baseline, set data testing baru, set data *batch* 1, dan set data *batch* 2. Semua set data dibentuk menjadi struktur yang dapat dilihat pada Gambar 4.1.



Gambar 4.1 Contoh struktur direktori data pada set data *batch* 1

4.2. Analisis dan Implementasi Sumber Kode Ultralytics

Pada subbab ini akan membahas pustaka atau *library* *Ultralytics* yang digunakan pada penelitian ini. Ada 3 bagian yang menjadi perhatian pada penelitian ini yaitu pengujian (*validation function*), mendapatkan metrik *F1-Score*, pelatihan dengan multi dataset. Gambar 4.2 merupakan struktur direktori pustaka *ultralytics* yang akan digunakan



Gambar 4.2 Struktur pustaka Ultralytics

4.2.1. Kode Pengujian dan Metriks dengan Model Baseline

Pada poin ini akan fokus membahas tentang bagaimana melakukan pengujian dan mendapatkan nilai metriks yang dicari. Untuk melakukan pengujian, fungsi yang akan digunakan adalah *validation* atau `val()`. Mendapatkan nilai metriks *F1-Score*, diharuskan menghitung *confusion matrix* dari hasil fungsi validasi. gambar berikut ini merupakan contoh kode untuk melakukan pengujian dengan model baseline.

```

import numpy as np

from ultralytics.utils.metrics import ConfusionMatrix
from ultralytics import YOLO

def f1_score(confusion_matrix):
    if isinstance(confusion_matrix, ConfusionMatrix):
        # If it's a ConfusionMatrix object, extract the numpy array
        confusion_matrix = confusion_matrix.matrix
    true_positives = np.diag(confusion_matrix)
    false_positives = np.sum(confusion_matrix, axis=0) - true_positives
    false_negatives = np.sum(confusion_matrix, axis=1) - true_positives
    # Handle potential division by zero
    precision = np.divide(
        true_positives,
        (true_positives + false_positives),
        out=np.zeros_like(true_positives),
        where=(true_positives + false_positives) != 0,
    )
    recall = np.divide(
        true_positives,
        (true_positives + false_negatives),
        out=np.zeros_like(true_positives),
        where=(true_positives + false_negatives) != 0,
    )
    # Handle potential division by zero again
    f1_scores = np.divide(
        2 * (precision * recall),
        (precision + recall),
        out=np.zeros_like(precision),
        where=(precision + recall) != 0,
    )
    # Exclude the "background" class (usually the last one) when calculating the average
    avg_f1_score = np.mean(f1_scores[:-1])
    return avg_f1_score

# Load the YOLOv8 model
model_path = r"C:\Data\Software_Engineer_2024\Semester VIII (Tugas Akhir)\Machine-Learning\prediction\nano.pt"
# Load the YOLOv8 model
model = YOLO(model_path)
# Perform testing
baseline_result = model.val(data=r"testing_flat.yaml", split="test")
new_result = model.val(data=r"testing_background.yaml", split="test")
# get confusion matrix
baseline_conf = baseline_result.confusion_matrix
new_conf = new_result.confusion_matrix
# get f1 score
baseline_f1 = f1_score(baseline_conf)
new_f1 = f1_score(new_conf)
print("Baseline F1 Score:", baseline_f1)
print("Baseline F1 Score:", baseline_f1)

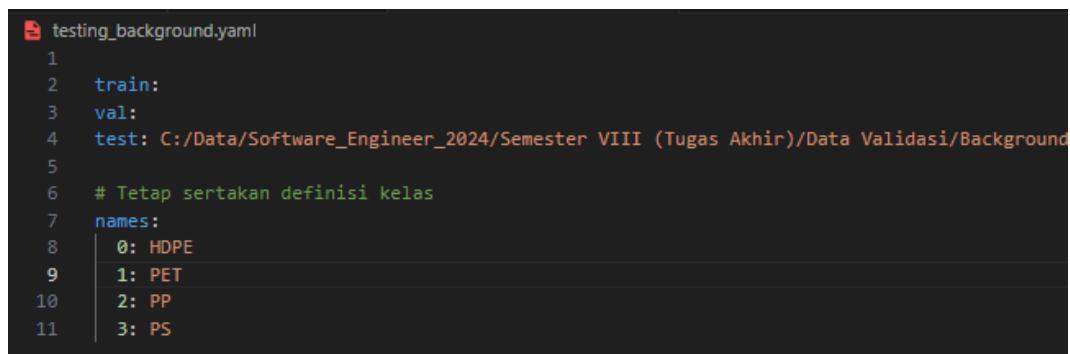
```

Gambar 4.3 Kode pengujian dan pengujian model baseline

Kode di atas merupakan implementasi pengujian model dengan model *baseline* menggunakan library *Ultralytics*. Pertama, modul yang diperlukan seperti *numpy*, *ConfusionMatrix* dari *ultralytics.utils.metrics*, dan *YOLO* dari *ultralytics*. Fungsi *f1_score* didefinisikan untuk menghitung skor F1 rata-rata dari *confusion matrix* yang diperoleh setelah proses validasi. Fungsi ini menangani input *confusion matrix* baik dalam bentuk objek *ConfusionMatrix* maupun *array numpy*, kemudian menghitung *true positives*, *false positives*, dan *false negatives*. *Precision* dan *recall* dihitung dengan penanganan pembagian oleh nol untuk menghindari *error*. Skor F1

per kelas dihitung berdasarkan *precision* dan *recall* tersebut, dan rata-rata skor F1 diperoleh dengan mengabaikan kelas "*background*" (biasanya kelas terakhir) untuk fokus pada kelas-kelas yang relevan.

Model dimuat menggunakan path model yang telah dilatih sebelumnya (*nano.pt*) adalah model *baseline*. Proses pengujian dilakukan dengan memanggil fungsi `val()` pada dua dataset yang berbeda, yaitu *testing_flat.yaml* dan *testing_background.yaml*, dengan parameter *split="test"* untuk menentukan bahwa data uji yang digunakan adalah bagian "*test*" dari dataset. Gambar 4.4 adalah contoh isi dari file *testing_background.yaml*.



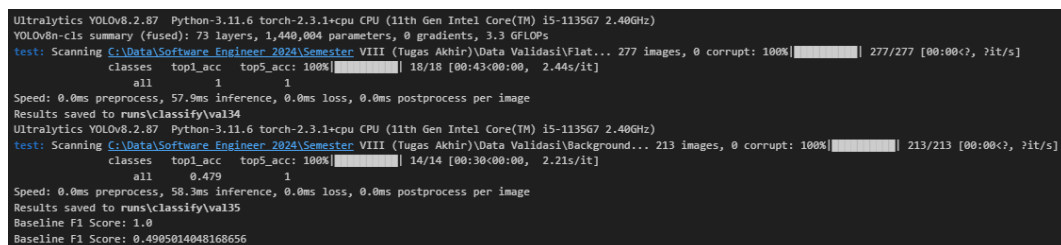
```

1
2 train:
3 val:
4 test: C:/Data/Software_Engineer_2024/Semester VIII (Tugas Akhir)/Data Validasi/Background
5
6 # Tetap sertakan definisi kelas
7 names:
8   0: HDPE
9   1: PET
10  2: PP
11  3: PS

```

Gambar 4.4 Contoh isi dari *testing_background.yaml*

Hasil pengujian disimpan dalam variabel *baseline_result* dan *new_result*. *Confusion matrix* diekstrak dari hasil pengujian tersebut melalui atribut *confusion_matrix*. Selanjutnya, fungsi *f1_score* digunakan untuk menghitung skor F1 rata-rata dari masing-masing *confusion matrix* yang diperoleh. Skor F1 ini memberikan metrik kinerja model yang lebih informatif dibandingkan akurasi, terutama pada dataset yang tidak seimbang. Gambar 4.5 Berikut adalah hasil jika program dijalankan, sekaligus mengetahui performa dari model *baseline*.



```

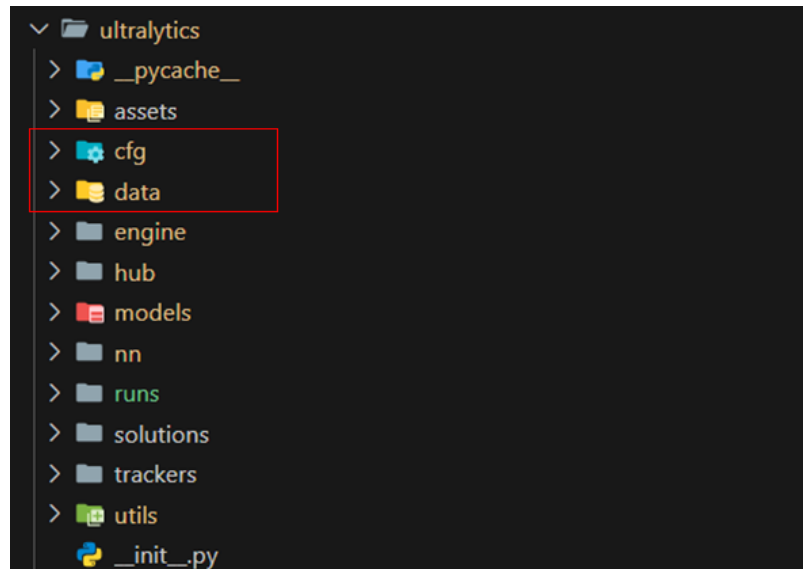
Ultralytics YOLOv8.2.87 Python-3.11.6 torch-2.3.1+cpu CPU (11th Gen Intel Core(TM) i5-1135G7 2.40GHz)
YOLOv8n-cs summary (fused): 73 layers, 1,440,004 parameters, 0 gradients, 3.3 GFLOPs
test: Scanning C:/Data/Software_Engineer_2024/Semester VIII (Tugas Akhir)/Data Validasi/Flat... 277 images, 0 corrupt: 100%| 277/277 [00:00<, ?it/s]
      classes top1_acc top5_acc 100%| 18/18 [00:43:00:00, 2.44s/it]
      all 1 1
Speed: 0.0ms preprocess, 57.9ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to runs\classify\val34
Ultralytics YOLOv8.2.87 Python-3.11.6 torch-2.3.1+cpu CPU (11th Gen Intel Core(TM) i5-1135G7 2.40GHz)
test: Scanning C:/Data/Software_Engineer_2024/Semester VIII (Tugas Akhir)/Data Validasi/Background... 213 images, 0 corrupt: 100%| 213/213 [00:00<, ?it/s]
      classes top1_acc top5_acc 100%| 14/14 [00:30:00:00, 2.21s/it]
      all 0.479 1
Speed: 0.0ms preprocess, 58.3ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to runs\classify\val35
Baseline F1 Score: 1.0
Baseline F1 Score: 0.4905014048168656

```

Gambar 4.5 *Output* program pengujian ketika dijalankan

4.2.2. Implementasi Modifikasi Fungsi Pelatihan

Melakukan pelatihan model menggunakan multi dataset, terdapat penyesuaian pada fungsi pelatihan (*training*) pada pustaka *ultralytics*. Hasil analisis menunjukkan, akan melakukan modifikasi pada 2 direktori yaitu folder *cfg* dan *data*, Gambar 4.6.



Gambar 4.6 Folder yang akan modifikasi

File *dataset.py* dalam folder *data* pada library *Ultralytics* merupakan komponen esensial yang mengelola semua aspek terkait data dalam proses pelatihan dan evaluasi model YOLO. Dengan menyediakan mekanisme untuk pemuatan data yang efisien, augmentasi, dan batching, *dataset.py* memastikan bahwa model menerima data dalam format yang tepat dan dalam aliran yang optimal. Hal ini sangat penting untuk mencapai performa model yang tinggi dan efisiensi dalam proses pelatihan. Gambar 4.7 Berikut adalah penambahan baris program pada file *dataset.py*.

```

class ConcatImageFolde :
    def __init__(self, image_folder_lis=[]):
        self.ifl = image_folder_lis
        t

    def add(self, imgfold):
        self.ifl.append(imgfold)

    @property
    def samples(self):
        all_samples = []
        for imgfold in self.ifl:
            all_samples.extend(imgfold.samples)
        return all_samples

    @property
    def roots(self):
        return [imgfold.root for imgfold in self.ifl]

    def __len__(self):
        return sum([len(imgfold) for imgfold in self.ifl])

class ClassificationDataset :
    """
    Extends torchvision ImageFolder to support YOLO classification tasks, offering functionalities like image
    augmentation, caching, and verification. It's designed to efficiently handle large datasets for training dee
    p learning models, with optional image transformations and caching mechanisms to speed up trainin
    g.

    This class allows for augmentations using both torchvision and Albumentations libraries, and supports caching image
    s in RAM or on disk to reduce IO overhead during training. Additionally, it implements a robust verification proces
    s to ensure data integrity and consistenc
    y.

    Attribute
    s:
    cache_ram (bool): Indicates if caching in RAM is enable
    d.
    cache_disk (bool): Indicates if caching on disk is enable
    d.
    samples (list): A list of tuples, each containing the path to an image, its class index, path to its .npz cach
    e
    file (if caching on disk), and optionally the loaded image array (if caching in RA
    M).
    torch_transforms (callable): PyTorch transforms to be applied to the image
    s.
    """

    def __init__(self, root, args, augment=False, prefix=""):
        print('ClassificationDataset root, arg , root, str(args))
        """ s:'
        Initialize YOLO object with root, image size, augmentations, and cache setting
    s.

    Arg
    s:
    root (str): Path to the dataset directory where images are stored in a class-specific folder structur
    e.
    args (Namespace): Configuration containing dataset-related settings such as image size, augmentatio
    n
    parameters, and cache settings. It includes attributes like 'imgsz' (image size), 'fraction' (fractio
    n
    of data to use), 'scale', 'fliplr', 'flipud', 'cache' (disk or RAM caching for faster trainin
    g),
    'auto_augment', 'hsv_h', 'hsv_s', 'hsv_v', and 'crop_fraction
    '.
    augment (bool, optional): Whether to apply augmentations to the dataset. Default is Fals
    e.
    prefix (str, optional): Prefix for logging and cache filenames, aiding in dataset identification an
    d
    debugging. Default is an empty strin
    g.
    """
    B.
    import torchvision # scope for faster 'import ultralytic
    s'

    root_type = os.path.split(root)[1]
    root = os.path.split(root)[0]
    all_dataset_path = [root]+list(args.rehearsal_replay_paths if not (args.rehearsal_replay_paths is None) else [])
    all_image_folder = []
    s
    for data_path in all_dataset_path :
        data_path = os.path.join(data_path, root_type)
        if TORCHVISION_0_18: # 'allow_empty' argument first introduced in torchvision 0.1
            dataset = torchvision.datasets.ImageFolder root=data_path, allow_empty=True)
        else:
            dataset = torchvision.datasets.ImageFolder root=data_path)
        all_image_folder .append(dataset)
    s
    self.concat_dataset = ConcatImageFolde (all_image_folder )
    self.samples = self.concat_dataset .samples
    self.roots = self.concat_dataset .roots
    print('ACTUAL DATASET COUN , len(self.concat_dataset ))
    T-'

```

Gambar 4.7 Modifikasi pada *file dataset.py*

Pada penelitian ini, dilakukan modifikasi pada bagian akhir *file dataset.py* dengan menambahkan kelas *ConcatImageFolder* dan mengintegrasikannya ke

dalam kelas *ClassificationDataset*. Kelas *ConcatImageFolder* dirancang untuk menggabungkan beberapa *instance ImageFolder* dari pustaka *torchvision* menjadi satu dataset terpadu. Hal ini memungkinkan pengumpulan sampel dari berbagai folder gambar yang berbeda untuk digunakan secara simultan dalam proses pelatihan atau validasi model.

Dalam konstruktor kelas *ClassificationDataset*, inisialisasi dataset dimodifikasi dengan menambahkan daftar *all_dataset_paths*, yang mencakup jalur dataset utama serta jalur tambahan yang diperoleh dari argumen *args.rehearsal_replay_paths*. Untuk setiap jalur dataset, dibuat *instance ImageFolder* yang kemudian ditambahkan ke dalam daftar *all_image_folders*. Selanjutnya, kelas *ConcatImageFolder* digunakan untuk menggabungkan semua *instance ImageFolder* tersebut menjadi satu dataset terintegrasi.

Dengan modifikasi ini, atribut *self.samples* dan *self.roots* dalam kelas *ClassificationDataset* kini mengambil data dari dataset gabungan. Pendekatan ini memungkinkan model memanfaatkan data dari berbagai sumber secara bersamaan, yang berpotensi meningkatkan kinerja model. Selain itu, modifikasi ini mendukung skenario pembelajaran berkelanjutan (*continual learning*), di mana data baru dapat ditambahkan ke dataset yang sudah ada tanpa mengganggu struktur data sebelumnya.

Error! Reference source not found. Penambahan parameter *rehearsal_replay_paths* memungkinkan konfigurasi model mendukung jalur tambahan ke dataset untuk mekanisme rehearsal replay dalam pembelajaran berkelanjutan. Dengan ini, pengguna dapat menentukan daftar jalur ke dataset sebelumnya, sehingga model dapat menggunakan kembali data lama selama pelatihan untuk mengurangi catastrophic forgetting.

```

121
122 # Custom config.yaml -----
123 cfg: # (str, optional) for overriding defaults.yaml
124 rehearsal_replay_paths: # (dict, optional)
125

```

Gambar 4.8 Modifikasi file *default.yaml*

Integrasi *rehearsal_replay_paths* dalam *default.yaml* memastikan parameter

ini dapat diakses di seluruh pipeline pelatihan tanpa mengubah kode inti, meningkatkan modularitas dan fleksibilitas konfigurasi model. Hal ini memfasilitasi eksperimen dengan berbagai dataset dan strategi pembelajaran berkelanjutan.

Dengan menyelaraskan konfigurasi ini dengan modifikasi pada `dataset.py`, di mana kelas `ClassificationDataset` kini mendukung beberapa sumber data melalui `ConcatImageFolder`, model dapat menggabungkan data dari tugas saat ini dan sebelumnya. Ini penting dalam penelitian pembelajaran berkelanjutan untuk menyeimbangkan pengetahuan baru dan lama, sehingga model menjadi lebih andal dan adaptif.

4.3. Hasil Implementasi Backend

4.3.1. Inference dan History Prediction

Tahap ini merupakan implementasi sebuah endpoint pada *framework Flask* dengan rute `/scan` yang menerima permintaan HTTP metode POST. *Endpoint* ini digunakan digunakan pada halaman prediksi yang akan digunakan oleh pengguna. Ketika permintaan POST diterima, sistem terlebih dahulu memeriksa apakah parameter `"model_path"` disertakan dalam *request.form*. Jika tersedia, variabel `yolo_model` akan menggunakan nilai dari `"model_path"` tersebut; jika tidak, sistem akan mengambil model terakhir yang digunakan melalui `model.get_last_model()[0]["model_path"]`. Selanjutnya, file gambar yang diunggah oleh pengguna diakses melalui `request.files["file"]` dan dibaca ke dalam objek *BytesIO*. Gambar tersebut kemudian didekode menggunakan *OpenCV* (`cv2.imdecode`) untuk mengubahnya menjadi array numerik yang dapat diproses.

Sistem memastikan keberadaan direktori penyimpanan `"static/data/prediction/"` dan membuatnya jika belum ada. Gambar yang telah didekode disimpan ke dalam direktori tersebut dengan nama file asli menggunakan `cv2.imwrite`. Setelah itu, fungsi `predict_label` dipanggil dengan parameter model YOLO yang dipilih dan gambar yang telah diproses untuk melakukan prediksi. Hasil prediksi disimpan dalam variabel `predicted_image`, dari mana informasi seperti tingkat kepercayaan (*confidence*), identifikasi (*identification*), dan waktu

pemrosesan (*processingTime*) diekstrak. Kode dapat dilihat pada Gambar 4.9 di bawah ini.

```
@app.route("/scan", methods=["POST"])
def predict():
    if request.method == "POST":
        yolo_model = ""
        if "model_path" in request.form and request.form["model_path"]:
            yolo_model = request.form["model_path"]
        else:
            yolo_model = model.get_last_model()[0]["model_path"]
        the_file = request.files["file"]
        file_stream = BytesIO(the_file.read())
        image = cv2.imdecode(
            np.frombuffer(file_stream.read(), np.uint8), cv2.IMREAD_COLOR
        )
        # Ensure the directory exists
        save_dir = r"static/data/prediction/"
        if not os.path.exists(save_dir):
            os.makedirs(save_dir)
        # Generate a unique filename for the image
        save_path = os.path.join(save_dir, the_file.filename)
        # Save image to the specified path
        cv2.imwrite(save_path, image)
        predicted_image = predict_label(yolo_model, image)
        max_prob = predicted_image["confidence"]
        identification = predicted_image["identification"]
        processing_times = predicted_image["processingTime"]
        model.save_prediction(
            {
                "path": save_path,
                "identification": identification,
                "confidence": float(max_prob),
                "processing_times": processing_times,
                "is_correct": True,
            }
        )
        data = {
            "identification": identification,
            "confidence": float(max_prob),
            "processingTime": float(processing_times),
            "manfaat": predicted_image["manfaat"],
            "kekurangan": predicted_image["kekurangan"],
            "logo": predicted_image["logo"],
        }
        return jsonify(data), 200

    return jsonify({"error": True, "message": "Method not allowed"}), 405
```

Gambar 4.9 Kode *endpoint inference* dan menyimpan prediksi

Prediksi tersebut kemudian disimpan ke dalam sistem melalui pemanggilan *model.save_prediction*, yang mencatat jalur penyimpanan gambar, identifikasi hasil prediksi, tingkat kepercayaan, waktu pemrosesan, dan status kebenaran prediksi (*is_correct* diatur sebagai *True*). Akhirnya, data yang mencakup identifikasi, tingkat

kepercayaan, waktu pemrosesan, serta informasi tambahan seperti "manfaat", "kekurangan", dan "logo" dari hasil prediksi dikemas dalam format JSON dan dikirim kembali kepada klien dengan kode status HTTP 200. Jika metode permintaan bukan POST, maka sistem akan mengembalikan respons JSON dengan pesan kesalahan dan kode status HTTP 405, menandakan bahwa metode yang digunakan tidak diizinkan.

4.3.2. Riwayat Prediksi dan Menambah Prediksi ke Batch

Kode untuk menangani fungsi riwayat prediksi dan menambahkan prediksi ke batch dapat pada *endpoint* `"/monitoring"`. Kode dapat dilihat pada Gambar 4.10 dibawah ini.

```
@app.route("/monitoring", methods=["GET", "POST"])
def monitoring():
    # tambah data ke batch
    if request.method == "POST":
        json_data = request.get_json()
        print(json_data)
        model.add_predictions_to_batch(json_data)

    # Pagination
    page = request.args.get("page", 1, type=int)
    per_page = 10
    # Ambil semua data prediksi
    all_predictions = model.get_prediction_with_sliced_path()

    # Hitung total data dan total halaman
    total_data = len(all_predictions)
    total_pages = (total_data + per_page - 1) // per_page
    # Hitung start dan end index untuk slicing
    start = (page - 1) * per_page
    end = start + per_page
    # Ambil potongan data untuk halaman ini
    prediction = all_predictions[start:end]

    all_batches = model.get_all_batches()

    # Tambahkan per_page ke render_template
    return render_template(
        "monitor.html",
        prediction=prediction,
        page=page,
        total_pages=total_pages,
        per_page=per_page,
        app=app,
        all_batches=all_batches,
        all_models=model.get_all_models_desc(),
    )
```

Gambar 4.10 *Endpoint* untuk riwayat prediksi dan menambahkan prediksi ke *batch*

Terdapat metode HTTP GET dan POST, ketika menerima permintaan dengan metode POST, fungsi ini mengambil data dalam format JSON dari permintaan melalui *request.get_json()*. Data JSON tersebut kemudian dicetak ke konsol untuk keperluan *debugging* dan ditambahkan ke *batch* prediksi menggunakan metode *model.add_predictions_to_batch(json_data)*. Hal ini memungkinkan sistem untuk secara dinamis memperbarui *batch* prediksi dengan data baru yang diterima.

Selanjutnya, fungsi ini mengimplementasikan mekanisme paginasi untuk menampilkan data prediksi secara terstruktur. Variabel *page* digunakan untuk menentukan halaman saat ini, dengan nilai *default* 1 jika parameter tersebut tidak disertakan dalam permintaan. Variabel *per_page* ditetapkan dengan nilai 10 untuk menentukan jumlah data prediksi yang ditampilkan per halaman. Fungsi kemudian mengambil semua data prediksi melalui *model.get_prediction_with_sliced_path()*, dan menghitung total data serta total halaman menggunakan perhitungan matematika yang sesuai.

Indeks awal (*start*) dan akhir (*end*) untuk *slicing* data ditentukan berdasarkan nomor halaman dan jumlah data per halaman. Data prediksi yang sesuai dengan halaman saat ini kemudian disimpan dalam variabel *prediction*. Selain itu, fungsi mengambil semua *batch* yang tersedia melalui *model.get_all_batches()* untuk ditampilkan atau digunakan dalam proses selanjutnya.

4.3.3. Membuat Replay Buffer (Batch)

Fitur membuat *replay buffer* merupakan fitur utama dalam sistem ini. Yaitu membentuk format dataset pelatihan dengan menggunakan data hasil koreksi pengguna. Gambar 4.11 yang disajikan mendefinisikan sebuah endpoint dalam framework Flask yang digunakan untuk mengelola dan memproses batch data gambar dalam konteks pembelajaran mesin. Endpoint */batch/<int:batch_id>* mendukung metode HTTP GET, POST, DELETE, dan PATCH, yang masing-masing menyediakan fungsionalitas spesifik untuk manipulasi batch dataset, khususnya dalam hal augmentasi data dan persiapan untuk pelatihan model.

```

@app.route("/batch/<int:batch_id>", methods=["GET", "POST", "DELETE", "PATCH"])
def batch_detail(batch_id):
    print("batch_detail method:", request.method)
    all_preds = model.get_predictions_in_batch(batch_id)
    dataset_in_batch = model.get_all_dataset_in_batch(batch_id)
    dataset_by_type = {d["dataset_type_id"]: d["path"] for d in dataset_in_batch}
    if request.method == "DELETE":
        data = request.get_json()
        model.delete_predictions_from_batch(data, batch_id)
    elif request.method == "POST":
        if len(all_preds) < 10:
            print("Not enough data to generate")
            return {"message": "Not enough data to generate"}
        else:
            data_split_ratio = {"train": 0.9, "val": 0.1}
            generated_dataset = train_stream.generate_dataset_dir(
                batch_id, data_split_ratio
            )
            model.update_generated_dataset(
                batch_id, generated_dataset["dataset_batch_dir"], 1
            )

    elif request.method == "PATCH":
        augment_out_dir = os.path.join(
            app.config["DATASET_GENERATED_DIR"], f"augmented_batch_{batch_id}"
        )
        if not os.path.isdir(augment_out_dir):
            os.makedirs(augment_out_dir)

        # Mengambil label unik dari prediksi
        select_valid = lambda l: [i for i in l if i[0]]
        all_unique_labels = set(
            [
                select_valid(
                    [pred["new_identification"], pred["identification"], "OTHER"]
                )
                for pred in all_preds
            ]
        )
        # Inisialisasi dictionary untuk menyimpan path gambar per label
        image_paths = {label: [] for label in all_unique_labels}

        # Mengumpulkan semua path gambar dari dataset asli (sebelum augmentasi)
        for data_type in os.listdir(dataset_by_type[1]): # train, valid
            for label in all_unique_labels: # PP, OTHER, etc
                label_dir = os.path.join(dataset_by_type[1], data_type, label)
                if not os.path.isdir(label_dir):
                    continue
                for fn in os.listdir(label_dir):
                    f_path = os.path.join(label_dir, fn)
                    image_paths[label].append(f_path)

        # Mengacak urutan gambar dalam setiap label
        for label in image_paths:
            random.shuffle(image_paths[label])

        # Membagi gambar ke dalam set pelatihan dan validasi sebelum augmentasi
        for label in image_paths:
            total_images = len(image_paths[label])
            if total_images == 0:
                continue
            train_split = int(0.9 * total_images) # 90% untuk training
            train_images = image_paths[label][:train_split]
            valid_images = image_paths[label][train_split:]

            # Membuat direktori untuk train dan val
            train_label_dir = os.path.join(augment_out_dir, 'train', label)
            val_label_dir = os.path.join(augment_out_dir, 'val', label)
            os.makedirs(train_label_dir, exist_ok=True)
            os.makedirs(val_label_dir, exist_ok=True)

            # Melakukan augmentasi dan menyimpan gambar training
            for f_path in train_images:
                fn = os.path.basename(f_path)
                # Terapkan augmentasi pada gambar training
                augmented_images = augmentation.mix_augmentations(f_path)
                for i, aug_img in enumerate(augmented_images):
                    aug_fn = f"{os.path.splitext(fn)[0]}_aug_{i}{os.path.splitext(fn)[1]}"
                    f_out = os.path.join(train_label_dir, aug_fn)
                    # Simpan gambar hasil augmentasi
                    cv2.imwrite(f_out, aug_img)

            # Menyimpan gambar validasi tanpa augmentasi
            for f_path in valid_images:
                fn = os.path.basename(f_path)
                f_out = os.path.join(val_label_dir, fn)
                # Salin gambar asli ke folder validasi
                shutil.copy(f_path, f_out)
            model.update_generated_dataset(batch_id, augment_out_dir, 2)

    all_preds_in_batch = model.get_predictions_path_slice_in_batch(batch_id)
    return render_template(
        "batch_detail.html",
        batch=model.get_batch(batch_id)[0],
        all_predictions=all_preds_in_batch,
        dataset_count=len(dataset_in_batch),
        all_datasets=dataset_by_type,
    )

```


Gambar 4.11 Endpoint untuk membuat batch atau *replay buffer*

Fungsi `batch_detail(batch_id)` berfungsi sebagai pusat pengolahan untuk operasi pada batch data yang ditentukan oleh `batch_id`. Pada awal eksekusi, fungsi ini mengambil semua prediksi yang terkait dengan batch tersebut menggunakan `model.get_predictions_in_batch(batch_id)`. Selain itu, ia mengumpulkan informasi tentang dataset yang termasuk dalam batch melalui `model.get_all_dataset_in_batch(batch_id)`, kemudian mengorganisasikannya dalam bentuk dictionary `dataset_by_type`, yang memetakan ID tipe dataset ke path masing-masing.

1. Metode DELETE: Menghapus Prediksi dari Batch

Ketika metode permintaan adalah DELETE, fungsi ini menangani penghapusan prediksi tertentu dari batch. Data yang diperlukan diekstrak dari payload JSON permintaan, dan fungsi `model.delete_predictions_from_batch(data, batch_id)` dipanggil untuk melakukan penghapusan. Fitur ini memungkinkan manajemen dataset yang dinamis, memungkinkan pengguna untuk menghilangkan data yang salah atau tidak diinginkan sebelum proses selanjutnya.

2. Metode POST: Menghasilkan Dataset untuk Pelatihan

Pada metode POST, fungsi ini memfasilitasi pembuatan direktori dataset baru yang siap untuk pelatihan model. Fungsi memeriksa apakah batch memiliki cukup prediksi (minimal sepuluh) untuk melanjutkan proses. Jika jumlah prediksi tidak mencukupi, fungsi mengembalikan pesan yang menunjukkan kekurangan data. Jika cukup, fungsi menetapkan rasio pembagian data untuk set pelatihan dan validasi (90% untuk pelatihan dan 10% untuk validasi) dan memanggil `train_stream.generate_dataset_dir(batch_id, data_split_ratio)` untuk menghasilkan struktur direktori dataset sesuai dengan pembagian tersebut. Path dataset yang dihasilkan kemudian diperbarui dalam catatan model menggunakan `model.update_generated_dataset(batch_id, generated_dataset["dataset_batch_dir"], 1)`.

3. Metode PATCH: Augmentasi Data dan Persiapan Dataset

Metode PATCH memainkan peran penting dalam meningkatkan dataset melalui augmentasi data dan persiapan untuk pelatihan model yang efektif. Langkah-langkah yang dilakukan adalah sebagai berikut:

- Pembuatan Direktori Output Augmentasi

Fungsi membangun path untuk direktori dataset hasil augmentasi dengan menggabungkan direktori dasar dataset yang dihasilkan (`app.config["DATASET_GENERATED_DIR"]`) dengan subdirektori yang dinamai berdasarkan `batch_id` (misalnya, `augmented_batch_1`). Fungsi memastikan bahwa direktori ini ada dengan membuatnya jika belum ada.

- Ekstraksi Label Unik

Untuk mengidentifikasi semua kelas yang ada dalam batch, fungsi mengekstrak label unik dari prediksi. Ini dilakukan dengan menggunakan fungsi `lambda select_valid` yang memilih label valid pertama dari `pred["new_identification"]`, `pred["identification"]`, atau default "OTHER" jika keduanya tidak tersedia. Label unik disimpan dalam sebuah set `all_unique_labels`.

- Pengumpulan Path Gambar

Dictionary `image_paths` diinisialisasi untuk memetakan setiap label ke daftar path file gambar yang sesuai. Fungsi mengiterasi melalui tipe data dalam direktori dataset asli (biasanya 'train' dan 'valid') dan, untuk setiap label, mengumpulkan semua path file gambar, sehingga mengumpulkan seluruh set gambar sebelum augmentasi.

- Pengacakan Gambar

Untuk meningkatkan randomness dan mengurangi bias dalam dataset, fungsi mengacak daftar path gambar untuk setiap label menggunakan `random.shuffle(image_paths[label])`. Ini memastikan distribusi data yang acak saat pembagian menjadi set pelatihan dan validasi.

- **Pembagian Menjadi Set Pelatihan dan Validasi**

Fungsi menghitung total jumlah gambar untuk setiap label dan menentukan indeks pembagian berdasarkan rasio pelatihan yang diinginkan (90%). Daftar yang telah diacak kemudian dibagi menjadi `train_images` dan `valid_images` untuk pelatihan dan validasi. Penting untuk dicatat bahwa pembagian ini dilakukan sebelum augmentasi untuk mencegah kebocoran data, sehingga versi augmentasi dari gambar yang sama tidak muncul di kedua set.

- **Penyiapan Struktur Direktori**

Untuk setiap label, fungsi membuat direktori dalam direktori output augmentasi untuk gambar pelatihan dan validasi ('train' dan 'val'), memastikan struktur dataset yang terorganisir dengan baik. Penggunaan `os.makedirs` dengan parameter `exist_ok=True` memastikan bahwa direktori yang diperlukan tersedia tanpa menimbulkan error jika sudah ada.

- **Augmentasi pada Gambar Pelatihan**

Fungsi mengiterasi melalui gambar pelatihan dan menerapkan teknik augmentasi pada setiap gambar dengan memanggil `augmentation.mix_augmentations(f_path)`. Fungsi augmentasi ini diharapkan mengembalikan daftar gambar yang telah diaugmentasi. Setiap gambar hasil augmentasi kemudian disimpan ke direktori pelatihan yang sesuai dengan nama file yang unik untuk mencegah penimpaan, misalnya dengan menambahkan indeks augmentasi pada nama file.

- **Penyimpanan Gambar Validasi Tanpa Augmentasi**

Untuk menjaga integritas set validasi, fungsi menyalin gambar validasi asli langsung ke direktori validasi tanpa menerapkan augmentasi. Pendekatan ini memastikan bahwa evaluasi model dilakukan pada data yang belum pernah dilihat dan tidak dimodifikasi, memberikan penilaian yang lebih akurat terhadap kemampuan generalisasi model.

- **Pembaruan Catatan Dataset dalam Model**

Setelah proses augmentasi dan persiapan dataset selesai, fungsi memperbarui catatan model dengan path ke dataset hasil augmentasi dengan memanggil `model.update_generated_dataset(batch_id, augment_out_dir, 2)`. Parameter tambahan (misalnya, 2) dapat menunjukkan status atau tipe dataset dalam konteks aplikasi.

Fungsi `generate_dataset_dir` bertujuan untuk mengotomatisasi proses pembuatan direktori dataset yang terstruktur untuk keperluan pelatihan model pembelajaran mesin, khususnya dalam tugas klasifikasi gambar dan pembelajaran berkelanjutan (continual learning). Fungsi ini mengambil data prediksi dari batch tertentu, membaginya menjadi set pelatihan dan validasi berdasarkan rasio yang ditentukan, serta mengorganisasikannya ke dalam struktur folder yang sesuai untuk digunakan dalam pelatihan model.. Kode fungsi `generate_dataset_dir` dapat dilihat pada Gambar 4.3

Gambar 4.3 Kode pengujian dan pengujian model baseline

```

def generate_dataset_dir(
    batch_id=0, data_split_ratio={"train": 0.9, "validation": 0.1}
):
    all_preds = db_model.get_predictions_in_batch(batch_id)
    select_valid = lambda l: [i for i in l if i][0]

    # Group predictions by label
    labels_preds = {}
    for pred in all_preds:
        label = select_valid([pred["new_identification"], pred["identification"]])
        if label not in labels_preds:
            labels_preds[label] = []
        labels_preds[label].append(pred)

    # Initialize the splitted_predictions
    splitted_predictions = {
        "info": {
            "count": {key: 0 for key in data_split_ratio.keys()},
            "error_file": [],
        }
    }
    for key in data_split_ratio.keys():
        splitted_predictions[key] = []

    # For each label, split the predictions into train and validation
    for label, preds in labels_preds.items():
        random.shuffle(preds)
        total_count = len(preds)
        ratios = list(data_split_ratio.items())
        counts = []

        # Calculate counts using round()
        for key, ratio in ratios:
            counts.append(round(total_count * ratio))

        # Adjust counts to sum up to total_count
        diff = total_count - sum(counts)
        if diff != 0:
            max_index = counts.index(max(counts))
            counts[max_index] += diff

        # Assign predictions to splits
        start_idx = 0
        for (key, _), count in zip(ratios, counts):
            splitted_predictions["info"]["count"][key] += count
            splitted_predictions[key].extend(preds[start_idx : start_idx + count])
            start_idx += count

    batch_id_str = str(batch_id)
    dataset_batch_dir = os.path.join(
        config.DATASET_GENERATED_DIR, f"replay_{batch_id_str}"
    )
    if not os.path.isdir(dataset_batch_dir):
        os.makedirs(dataset_batch_dir)

    # Create directories for each label in each dataset type
    for dataset_type, pred_split in splitted_predictions.items():
        if dataset_type == "info":
            continue
        unique_labels_in_split = set(
            select_valid([pred["new_identification"], pred["identification"]])
            for pred in pred_split
        )
        for label in unique_labels_in_split:
            label_dir = os.path.join(dataset_batch_dir, dataset_type, label)
            if not os.path.isdir(label_dir):
                os.makedirs(label_dir)

    # Copy files to their respective directories
    for dataset_type, pred_split in splitted_predictions.items():
        if dataset_type == "info":
            continue
        for pred in pred_split:
            src = pred["path"]
            dst = os.path.join(
                dataset_batch_dir,
                dataset_type,
                select_valid([pred["new_identification"], pred["identification"]]),
                os.path.basename(pred["path"]),
            )
            if not os.path.isfile(src):
                print(f"File not found: {src}")
                splitted_predictions["info"]["error_file"].append(src)
                continue
            shutil.copyfile(src, dst)

    print(splitted_predictions["info"])
    return {"dataset_batch_dir": dataset_batch_dir, "split": splitted_predictions}

```

Gambar 4.12 Fungsi untuk membuat *replay buffer*

Fungsi `generate_dataset_dir` bertujuan untuk mengotomatisasi proses pembuatan direktori dataset yang terstruktur untuk keperluan pelatihan model pembelajaran mesin, khususnya dalam tugas klasifikasi gambar dan pembelajaran berkelanjutan (*continual learning*). Fungsi ini mengambil data prediksi dari batch tertentu, membaginya menjadi set pelatihan dan validasi berdasarkan rasio yang ditentukan, serta mengorganisasikannya ke dalam struktur folder yang sesuai untuk digunakan dalam pelatihan model.

Proses dimulai dengan mengambil semua prediksi yang terkait dengan `batch_id` tertentu menggunakan `db_model.get_predictions_in_batch(batch_id)`. Fungsi `lambda select_valid` digunakan untuk memilih label yang valid dari setiap prediksi, dengan prioritas pada `new_identification` jika tersedia, atau `identification` sebagai alternatif. Pendekatan ini memastikan bahwa label terbaru atau yang telah dikoreksi digunakan dalam pembentukan dataset.

Selanjutnya, prediksi dikelompokkan berdasarkan labelnya untuk memastikan distribusi kelas yang seimbang dalam dataset. Pengelompokan ini dilakukan dengan mengiterasi setiap prediksi dan menambahkannya ke `dictionary labels_preds` sesuai dengan labelnya. Dengan demikian, setiap label memiliki daftar prediksi yang dapat dibagi secara proporsional ke dalam set pelatihan dan validasi.

Fungsi kemudian menginisialisasi struktur untuk menyimpan prediksi yang telah dibagi melalui `dictionary splitted_predictions`. Di sini, informasi seperti jumlah prediksi per set dan daftar file yang mengalami error dicatat dalam `sub-dictionary info`. Untuk setiap tipe dataset yang ditentukan dalam `data_split_ratio` (misalnya, 'train' dan 'validation'), daftar kosong disiapkan untuk menampung prediksi yang akan dimasukkan.

Pembagian prediksi ke dalam set pelatihan dan validasi dilakukan per label untuk menjaga keseimbangan kelas. Prediksi untuk setiap label diacak menggunakan `random.shuffle` untuk mengurangi bias yang mungkin timbul akibat urutan data. Jumlah prediksi yang dialokasikan ke setiap set dihitung berdasarkan rasio yang ditentukan, dengan penyesuaian menggunakan fungsi `round` untuk memastikan nilai integer. Jika terdapat selisih antara total prediksi dan jumlah yang

dibagi, penyesuaian dilakukan dengan menambahkannya ke set dengan jumlah terbesar.

Setelah prediksi dibagi, fungsi membuat struktur direktori untuk dataset. Direktori utama diberi nama berdasarkan `batch_id` dan dibuat jika belum ada. Di dalamnya, subdirektori untuk setiap tipe dataset ('train' dan 'validation') serta subdirektori untuk setiap label dibuat. Struktur ini memudahkan pengorganisasian data dan kompatibilitas dengan framework pembelajaran mesin yang umum digunakan.

Proses selanjutnya adalah menyalin file gambar ke direktori yang sesuai. Fungsi mengiterasi setiap prediksi dalam set pelatihan dan validasi, menentukan path sumber dan tujuan, lalu menyalin file menggunakan `shutil.copyfile`. Jika file sumber tidak ditemukan, informasi ini dicatat dalam daftar `error_file` tanpa menghentikan keseluruhan proses. Hal ini penting untuk memastikan bahwa proses tetap robust dan error pada beberapa file tidak mengganggu keseluruhan operasi.

Di akhir fungsi, informasi mengenai jumlah prediksi dalam setiap set dan daftar file error dicetak untuk keperluan monitoring. Fungsi mengembalikan sebuah dictionary yang berisi path direktori dataset yang dihasilkan dan detail pembagian prediksi, sehingga dapat digunakan untuk proses selanjutnya seperti pelatihan model atau analisis lebih lanjut.

Untuk mengatasi keterbatasan data dalam dataset hasil *generate batch* dan meningkatkan generalisasi, fungsi *mix_augmentations*, yang dirancang untuk melakukan augmentasi gambar secara dinamis dengan berbagai metode dan fungsi augmentasi. Fungsi ini bertujuan untuk memperkaya dataset gambar melalui teknik augmentasi, yang sangat berguna dalam konteks pembelajaran mesin dan visi komputer untuk meningkatkan generalisasi model dan kinerja pada data yang belum pernah dilihat sebelumnya. Fungsi *mix_augmentations* menerima input berupa gambar yang akan di-augmentasi, yang dapat berupa *array NumPy* atau *path* ke file gambar. Jika input adalah *path*, fungsi akan membaca gambar menggunakan *OpenCV*. Fungsi ini juga menerima parameter *method*, yang merupakan *dictionary* untuk menentukan metode augmentasi yang akan digunakan. Tiga metode yang tersedia adalah "*separate*", "*flow*", dan "*combination*", namun hanya satu metode

yang dapat diaktifkan (*True*) pada satu waktu untuk memastikan operasi yang jelas dan terkontrol. Program augmentation dapat dilihat pada Gambar 4.13.


```

DEFAULT_MIX_AUGMENTATIONS_FUNCS_ARGS = (
    (change_brightness, (0.7,)),
    (change_brightness, (-0.7,)),
    (add_blur, (0.4,)),
)

def mix_augmentations(
    img,
    out=None,
    method={"seperate": True, "flow": False, "combination": False},
    funcs_args=DEFAULT_MIX_AUGMENTATIONS_FUNCS_ARGS,
):
    selected_method = [(k, v) for k, v in method.items() if v]
    assert (
        len(selected_method) == 1
    ), f"Need one method to be True in method, found: {len(selected_method)}"

    if type(img) == np.ndarray:
        pass
    elif type(img) == str:
        if (not os.path.isdir(img)) and os.path.isfile(img):
            img_path = img
            img = cv2.imread(img_path)
        else:
            raise ValueError(f"Image at path {img} is not accessible")
    else:
        raise ValueError(
            f"Invalid img type: {type(img)}"
            , required np.ndarray or str of img path"
        )

    method_mapping = {
        "seperate": seperate_augmentation,
        "flow": flow_augmentation,
        "combination": combination_augmentation,
    }

    func, kwargs = selected_method[0]
    func = method_mapping[func]
    results = (
        func(img, funcs_args)
        if type(kwargs) is bool
        else func(*kwargs, img, funcs_args)
    )
    if type(out) == str:
        out_dir = os.path.split(out)[0]
        img_fn = os.path.split(out)[1]
        if not os.path.isdir(out_dir):
            os.makedirs(out_dir)
        for i, im_result in enumerate(results):
            cv2.imwrite(os.path.join(out_dir, f"{i}_{img_fn}"), im_result)
    else:
        return results

```

Gambar 4.13 Kode program augmentation

Parameter penting lainnya adalah *funcs_args*, yang merupakan *tuple* berisi pasangan fungsi augmentasi dan argumen yang akan diterapkan. Dalam kode yang diberikan, fungsi *default* yang digunakan adalah *change_brightness* dengan variasi peningkatan dan penurunan kecerahan, serta *add_blur* untuk menambahkan efek blur pada gambar. Setiap fungsi augmentasi disertai dengan argumen spesifik yang mengontrol intensitas atau parameter lain dari augmentasi tersebut.

Setelah validasi input dan metode augmentasi yang dipilih, fungsi akan memetakan metode tersebut ke fungsi implementasi yang sesuai melalui *method_mapping*. Fungsi implementasi seperti *seperate_augmentation*, *flow_augmentation*, dan *combination_augmentation* bertanggung jawab untuk menerapkan strategi augmentasi yang berbeda sesuai dengan metode yang dipilih. Misalnya, metode "separate" akan menerapkan setiap fungsi augmentasi secara terpisah pada gambar asli, sedangkan "flow" akan menerapkan fungsi augmentasi secara berurutan, dan "combination" akan menggabungkan fungsi-fungsi augmentasi tersebut.

Hasil dari augmentasi disimpan dalam variabel *results*, yang merupakan kumpulan gambar yang telah diaugmentasi. Jika parameter *out* diberikan sebagai string yang menunjukkan path direktori output, fungsi akan menyimpan setiap gambar hasil augmentasi ke direktori tersebut dengan penamaan yang diindeks. Jika tidak, fungsi akan mengembalikan hasil augmentasi sebagai objek, memungkinkan pengguna untuk memproses hasil lebih lanjut dalam kode mereka.

Antisipasi kedua yaitu, meminimalisir penggunaan data *replay buffer* terlalu banyak yang berdampak pada biaya komputasi. Penggunaan *random sampling* pada penelitian ini awalnya digunakan dalam dataset *baseline*. Fungsi *random sampling* diimplementasi langsung pada *route framework flask* yang dapat dilihat pada Gambar 4.14.

```

@app.route("/random_sampling", methods=["POST"])
def random_sampling():
    batch_id = request.get_json()["batch_id"]
    # Mendapatkan dataset replay berdasarkan batch_id
    replay_dataset_id = model.get_all_dataset_in_batch(batch_id)[0]["id"]
    replay_path = model.get_dataset_by_id(replay_dataset_id)[0]["path"]
    # Path baru untuk replay buffer hasil random sampling, di luar folder replay_21
    new_replay_path = os.path.join(
        "static/data/generated_dataset", "replay_random" + f"_{batch_id}"
    )
    # Create the new replay folder if it doesn't exist
    if not os.path.exists(new_replay_path):
        os.makedirs(new_replay_path)
    # Define train and val directories
    train_dir = os.path.join(replay_path, "train")
    val_dir = os.path.join(replay_path, "val")
    # Sampling ratios
    train_sample_ratio = 0.5 # 20% gambar dari train
    val_sample_ratio = 0.5 # 10% gambar dari val
    # List all classes (subfolders) in train directory
    classes = os.listdir(train_dir)
    for class_name in classes:
        class_train_dir = os.path.join(train_dir, class_name)
        class_val_dir = os.path.join(val_dir, class_name)
        # Skip if it's not a directory
        if not os.path.isdir(class_train_dir):
            continue
        # --- RANDOM SAMPLING UNTUK TRAIN ---
        # Get list of all images in train directory for the current class
        train_images = [
            f
            for f in os.listdir(class_train_dir)
            if f.endswith(".jpg") or f.endswith(".png") or f.endswith(".jpeg")
        ]
        # --- RANDOM SAMPLING UNTUK VAL ---
        # Get list of all images in val directory for the current class
        val_images = [
            f
            for f in os.listdir(class_val_dir)
            if f.endswith(".jpg") or f.endswith(".png") or f.endswith(".jpeg")
        ]
        # --- HITUNG SAMPEL TRAIN & VAL ---
        train_sample_size = (
            int(train_sample_ratio * len(train_images)) if train_images else 0
        )
        val_sample_size = (
            int(val_sample_ratio * len(val_images))
            if len(val_images) > 1
            else len(val_images)
        )
        # --- PILIH SAMPEL SECARA ACAK ---
        train_sample = (
            np.random.choice(train_images, train_sample_size, replace=False)
            if train_images
            else []
        )
        val_sample = (
            np.random.choice(val_images, val_sample_size, replace=False)
            if val_images
            else []
        )
        # --- BUAT SUBFOLDER UNTUK TRAIN DI REPLAY BUFFER ---
        replay_train_class_dir = os.path.join(new_replay_path, "train", class_name)
        if not os.path.exists(replay_train_class_dir):
            os.makedirs(replay_train_class_dir)
        # Copy sampled images from train
        for img in train_sample:
            src_path = os.path.join(class_train_dir, img)
            dst_path = os.path.join(replay_train_class_dir, img)
            shutil.copy(src_path, dst_path)
            print(f"Copied {src_path} to {dst_path}")
        # --- BUAT SUBFOLDER UNTUK VAL DI REPLAY BUFFER ---
        replay_val_class_dir = os.path.join(new_replay_path, "val", class_name)
        if not os.path.exists(replay_val_class_dir):
            os.makedirs(replay_val_class_dir)
        # Copy sampled images from val
        for img in val_sample:
            src_path = os.path.join(class_val_dir, img)
            dst_path = os.path.join(replay_val_class_dir, img)
            shutil.copy(src_path, dst_path)
            print(f"Copied {src_path} to {dst_path}")
    # Update the dataset with the new replay path
    model.update_generated_dataset(batch_id, new_replay_path, 3)
    return jsonify({"status": "success", "message": "Random sampling completed."})

```

Gambar 4.14 Program *random sampling* untuk *replay buffer*

Ketika menerima permintaan POST dengan `batch_id`, program ini melakukan langkah-langkah berikut:

1. Pengambilan Dataset: Program mengambil dataset replay yang terkait dengan `batch_id` melalui metode `get_all_dataset_in_batch` dan `get_dataset_by_id` dari objek model. Path dataset ini digunakan sebagai direktori sumber untuk proses sampling.
2. Pembuatan Replay Buffer Baru: Program membangun path baru untuk menyimpan dataset hasil sampling acak, yang dinamai `replay_random_{batch_id}` di dalam direktori `static/data/generated_dataset`. Jika direktori ini belum ada, program akan membuatnya menggunakan `os.makedirs`.
3. Definisi Direktori Training dan Validasi: Program menetapkan path untuk direktori training (`train_dir`) dan validasi (`val_dir`) dari dataset asli yang akan digunakan sebagai sumber data.
4. Penentuan Rasio Sampling: Rasio sampling ditetapkan dengan `train_sample_ratio` dan `val_sample_ratio` masing-masing sebesar 0.5, yang berarti 50% gambar dari dataset training dan validasi akan dipilih secara acak.
5. Proses Sampling per Kelas: Program melakukan iterasi melalui setiap kelas (subdirektori) dalam direktori training. Untuk setiap kelas:
 - Pengambilan Daftar Gambar: Mengumpulkan daftar file gambar dalam direktori training dan validasi untuk kelas tersebut.
 - Perhitungan Ukuran Sampel: Menghitung jumlah sampel yang akan diambil berdasarkan rasio sampling dan jumlah gambar yang tersedia.
 - Pemilihan Sampel Acak: Menggunakan `numpy.random.choice` untuk memilih sampel gambar secara acak tanpa penggantian.
 - Pembuatan Direktori Kelas di Replay Buffer: Membuat subdirektori untuk kelas tersebut dalam direktori replay buffer baru untuk data training dan validasi jika belum ada.
 - Penyalinan Gambar Sampel: Menyalin file gambar yang terpilih ke

direktori replay buffer yang sesuai menggunakan `shutil.copy`.

6. Pembaruan Dataset dalam Model: Setelah semua kelas diproses, program memperbarui informasi dataset dalam model dengan memanggil `update_generated_dataset`, memberikan `batch_id`, path replay baru, dan kode status (dalam hal ini 3).
7. Pengembalian Respon: Program mengembalikan respon JSON yang menunjukkan bahwa proses sampling acak telah selesai dengan sukses.

Melalui proses ini, program secara efektif mengurangi ukuran dataset dengan memilih subset gambar secara acak dari setiap kelas dalam set data training dan validasi. Teknik ini berguna ketika berhadapan dengan dataset besar di mana sumber daya komputasi terbatas.

4.3.4. Pelatihan Model

Setelah dataset terbuat dengan menyimpan `id_dataset` dan path direktori dataset. Fungsi pelatihan akan mengambil id atau beberapa id dataset yang dipilih. Gambar 4.16 mendefinisikan sebuah endpoint Flask dengan URL `/model-detail/<int:model_id>`, yang mendukung metode HTTP GET dan POST. Fungsi `model_detail` bertanggung jawab untuk menampilkan detail model tertentu dan memfasilitasi proses pelatihan model baru menggunakan dataset yang dipilih oleh pengguna.

Pada metode GET, fungsi ini mengambil informasi model berdasarkan `model_id` menggunakan `model.get_model(model_id)[0]`. Selain itu, fungsi mengambil daftar dataset yang terkait dengan model tersebut melalui `model.get_all_dataset_on_model(model_id)` dan semua dataset yang tersedia menggunakan `model.get_all_dataset()`. Data ini kemudian digunakan untuk merender template `model_detail.html`, yang menampilkan detail model dan informasi dataset terkait kepada pengguna.

```

@app.route("/model-detail/<int:model_id>", methods=["GET", "POST"])
def model_detail(model_id):
    the_model = model.get_model(model_id)[0]
    dataset_on_model = model.get_all_dataset_on_model(model_id)
    all_dataset = model.get_all_dataset()

    if request.method == "POST":
        data = request.get_json()
        last_model_path = model.get_last_model_path()[0]["model_path"]
        model_name = data["name"]
        model_description = data["description"]
        dataset_id = data["all_ids"]
        selected_datasets = [
            model.get_dataset_by_id(ds_id)[0] for ds_id in data["all_ids"]
        ]
        dataset_paths = [ds["path"] for ds in selected_datasets]
        the_thread = Thread(
            target=train_stream.train_yolo_multiple_datasets,
            args=[
                last_model_path,
                dataset_paths,
                model_name,
                model_description,
                dataset_id,
            ],
        )
        the_thread.start()

    return render_template(
        "model_detail.html",
        model=the_model,
        all_dataset=all_dataset,
        dataset_on_model=dataset_on_model,
    )

```

Gambar 4.15 Endpoint model-detail

Pada metode POST, fungsi ini menangani permintaan untuk melatih model baru. Data yang diterima melalui `request.get_json()` mencakup `name` (nama model), `description` (deskripsi model), dan `all_ids` (daftar `dataset_id` yang dipilih). Fungsi mengambil path model terakhir yang telah dilatih menggunakan `model.get_last_model_path()[0]["model_path"]`. Berdasarkan `all_ids`, fungsi mengumpulkan dataset yang dipilih dengan `model.get_dataset_by_id(ds_id)[0]` dan mengekstrak path masing-masing dataset.

Proses pelatihan model baru dilakukan dengan memulai thread baru yang menjalankan fungsi `train_stream.train_yolo_multiple_datasets`. Fungsi ini menerima argumen berupa path model terakhir, path dataset yang dipilih, nama model baru, deskripsi model, dan ID dataset. Dengan menjalankan proses pelatihan dalam thread terpisah (`the_thread.start()`), aplikasi memastikan bahwa operasi pelatihan yang mungkin memakan waktu tidak menghalangi responsivitas server

terhadap permintaan lain.

Selanjutnya untuk melakukan pelatihan model, fungsi yang digunakan adalah `train_yolo_multiple_datasets`, yang dirancang untuk melatih model YOLO (You Only Look Once) untuk tugas klasifikasi gambar menggunakan multiple dataset. Fungsi ini mengintegrasikan proses pelatihan, evaluasi, dan pembaruan basis data, sehingga sangat berguna dalam konteks penelitian di mana pelacakan eksperimen dan hasil sangat penting. Program dapat dilihat pada Gambar 4.16.

```
import prediction.test_model as test_model
sys.path.insert(1, config.CUSTOM_LIBRARY_DIR)
from ultralytics import YOLO

def train_yolo_multiple_datasets(
    last_model_path, dataset_paths, name, description, dataset_id
):
    if len(dataset_paths) < 1:
        raise ValueError(
            "At least need one dataset path to start training the model")
    elif len(dataset_paths) == 1:
        main_ds = dataset_paths[0]
        ext_ds = []
    else:
        main_ds = dataset_paths[0]
        ext_ds = dataset_paths[1:]
    model = YOLO(last_model_path, task="classify")
    model.train(data=main_ds, epochs=5, imgsz=640, rehearsal_replay_paths=
ext_ds)
    save_dir = model.trainer.save_dir
    df_training_results = pd.read_csv(os.path.join(save_dir, "results.csv"))
    img_training_result = os.path.join(save_dir, "results.png")
    last_weight = os.path.join(save_dir, "weights", "last.pt")
    best_weight = os.path.join(save_dir, "weights", "best.pt")
    out = save_dir, df_training_results, img_training_result, last_weight,
best_weight
    print("OUT:", out)
    testing_model = test_model.testing(last_model_path, best_weight)
    data = {
        "name": name,
        "description": description,
        "model_path": best_weight,
        "f1_score_baseline": testing_model["f1_score_baseline"],
        "f1_score_new": testing_model["f1_score_new"],
        "forgetting_rate": testing_model["forgetting_rate"],
        "learning_rate": testing_model["learning_rate"],
    }
    save_model = db_model.add_model(data)
    for i in dataset_id:
        dataset_in_model = {
            "model_id": save_model,
            "dataset_id": i,
        }
        saved_dataset_in_model = db_model.add_dataset_in_model(
dataset_in_model)
    return saved_dataset_in_model
```

Gambar 4.16 Fungsi pelatihan dengan multi dataset

Fungsi ini dimulai dengan validasi input untuk memastikan bahwa setidaknya satu path dataset diberikan. Jika hanya satu dataset yang disediakan, dataset tersebut ditetapkan sebagai dataset utama (`main_ds`), dan daftar dataset eksternal (`ext_ds`) dibiarkan kosong. Jika lebih dari satu dataset diberikan, dataset pertama dianggap sebagai dataset utama, sementara sisanya digunakan sebagai dataset tambahan untuk proses rehearsal replay. Pendekatan ini memungkinkan implementasi continual learning, di mana model dilatih secara bertahap dengan data baru sambil mempertahankan pengetahuan dari data sebelumnya.

Selanjutnya, fungsi menginisialisasi model YOLO untuk tugas klasifikasi dengan memuat bobot model terakhir dari `last_model_path`. Proses pelatihan dimulai dengan memanggil metode `model.train`, di mana model dilatih pada dataset utama (`main_ds`) selama lima epoch dengan ukuran gambar 640 piksel. Dataset tambahan (`ext_ds`) digunakan dalam parameter `rehearsal_replay_paths`, yang membantu model untuk mengulang data sebelumnya dan mengurangi efek catastrophic forgetting.

Setelah pelatihan selesai, fungsi mengumpulkan hasil pelatihan dari direktori output model (`save_dir`). Ini termasuk membaca file `results.csv` menggunakan Pandas untuk mendapatkan metrik pelatihan dan menentukan path untuk gambar hasil pelatihan (`results.png`), serta bobot model terakhir (`last.pt`) dan terbaik (`best.pt`). Informasi ini penting untuk analisis kinerja model dan pemilihan bobot terbaik untuk evaluasi lebih lanjut.

Fungsi kemudian melakukan evaluasi model dengan memanggil `test_model.testing`, yang menerima path model terakhir dan bobot model terbaik. Fungsi pengujian ini mengembalikan metrik evaluasi seperti `f1_score_baseline`, `f1_score_new`, `forgetting_rate`, dan `learning_rate`. Metrik ini krusial untuk menilai efektivitas proses pelatihan dan dampaknya terhadap kemampuan generalisasi model.

Data hasil evaluasi kemudian disusun dalam sebuah dictionary yang mencakup nama model, deskripsi, path ke bobot model terbaik, dan metrik performa. Data ini disimpan ke dalam basis data dengan memanggil `db_model.add_model(data)`, yang menambahkan entri baru ke tabel model. Fungsi

juga mengaitkan setiap dataset yang digunakan dalam pelatihan dengan model yang baru dilatih dengan iterasi melalui `dataset_id` dan memanggil `db_model.add_dataset_in_model(dataset_in_model)`. Langkah ini memastikan bahwa ada rekaman jelas mengenai dataset apa saja yang digunakan untuk melatih model tertentu, yang esensial untuk reproduktibilitas dan pelacakan eksperimen dalam penelitian.

4.3.5. Pengujian Model

Pada program ini memainkan peran penting dalam evaluasi model dalam skenario pembelajaran berkelanjutan, di mana penting untuk memonitor keseimbangan antara akuisisi pengetahuan baru dan retensi pengetahuan lama. Dengan menghitung metrik seperti forgetting rate dan learning rate, peneliti dapat menilai efektivitas strategi pelatihan yang digunakan dan melakukan penyesuaian yang diperlukan untuk meningkatkan kinerja model secara keseluruhan.

Gambar 4.17 mendefinisikan dua fungsi utama, yaitu `f1_score` dan `testing`, yang digunakan untuk mengevaluasi kinerja model YOLO (You Only Look Once) dalam konteks pembelajaran berkelanjutan (continual learning). Fungsi `f1_score` bertanggung jawab untuk menghitung skor F1 rata-rata dari matriks kebingungan (confusion matrix), sementara fungsi `testing` mengevaluasi model pada dataset baseline dan dataset baru, menghitung tingkat pelupaan (forgetting rate) dan tingkat pembelajaran (learning rate), serta mengembalikan metrik-metrik tersebut untuk analisis lebih lanjut.

Secara spesifik, fungsi `f1_score` menerima input berupa matriks kebingungan yang mungkin dalam bentuk instance `ConfusionMatrix` atau array NumPy. Fungsi ini menghitung true positives, false positives, dan false negatives untuk setiap kelas. Dengan menggunakan nilai-nilai tersebut, fungsi menghitung precision dan recall, sambil memastikan tidak terjadi pembagian dengan nol dengan menggunakan parameter `where` dalam fungsi `np.divide`. Skor F1 rata-rata diperoleh dengan menghitung mean dari skor F1 semua kelas, biasanya dengan mengecualikan kelas terakhir yang mungkin mewakili kelas latar belakang atau kelas tidak terdefinisi.

```

import sys
import config
import numpy as np
import database.model as db_model

sys.path.insert(1, config.CUSTOM_LIBRARY_DIR)
from ultralytics import YOLO
from ultralytics.utils.metrics import ConfusionMatrix

yaml_baseline = "prediction/testing_flat.yaml"
yaml_new = "prediction/testing_background.yaml"

def f1_score(confusion_matrix):
    if isinstance(confusion_matrix, ConfusionMatrix):
        confusion_matrix = confusion_matrix.matrix
    true_positives = np.diag(confusion_matrix)
    false_positives = np.sum(confusion_matrix, axis=0) - true_positives
    false_negatives = np.sum(confusion_matrix, axis=1) - true_positives
    precision = np.divide(
        true_positives,
        (true_positives + false_positives),
        out=np.zeros_like(true_positives),
        where=(true_positives + false_positives) != 0,
    )
    recall = np.divide(
        true_positives,
        (true_positives + false_negatives),
        out=np.zeros_like(true_positives),
        where=(true_positives + false_negatives) != 0,
    )
    f1_scores = np.divide(
        2 * (precision * recall),
        (precision + recall),
        out=np.zeros_like(precision),
        where=(precision + recall) != 0,
    )
    avg_f1_score = np.mean(f1_scores[:-1])
    return avg_f1_score

def testing(last_model_path, model_path):
    model = YOLO(model_path)
    testing_baseline_dataset = model.val(data=yaml_baseline, split="test")
    testing_new_dataset = model.val(data=yaml_new, split="test")
    confusion_matrix_baseline = testing_baseline_dataset.confusion_matrix
    confusion_matrix_new = testing_new_dataset.confusion_matrix
    avg_f1_scores_baseline = f1_score(confusion_matrix_baseline)
    avg_f1_scores_new = f1_score(confusion_matrix_new)
    get_model = db_model.get_model_by_path(last_model_path)[0]
    forgetting_rate = avg_f1_scores_baseline - get_model["f1_score_baseline"]
    learning_rate = avg_f1_scores_new - get_model["f1_score_new"]
    data = {
        "f1_score_baseline": avg_f1_scores_baseline,
        "f1_score_new": avg_f1_scores_new,
        "forgetting_rate": forgetting_rate,
        "learning_rate": learning_rate,
    }
    return data

```

Gambar 4.17 Program testing

Fungsi testing memulai dengan memuat model YOLO menggunakan `model_path` yang diberikan. Model tersebut kemudian dievaluasi pada dua dataset: satu dataset baseline yang dikonfigurasi dalam `yaml_baseline`, dan satu dataset baru yang dikonfigurasi dalam `yaml_new`. Evaluasi dilakukan menggunakan metode

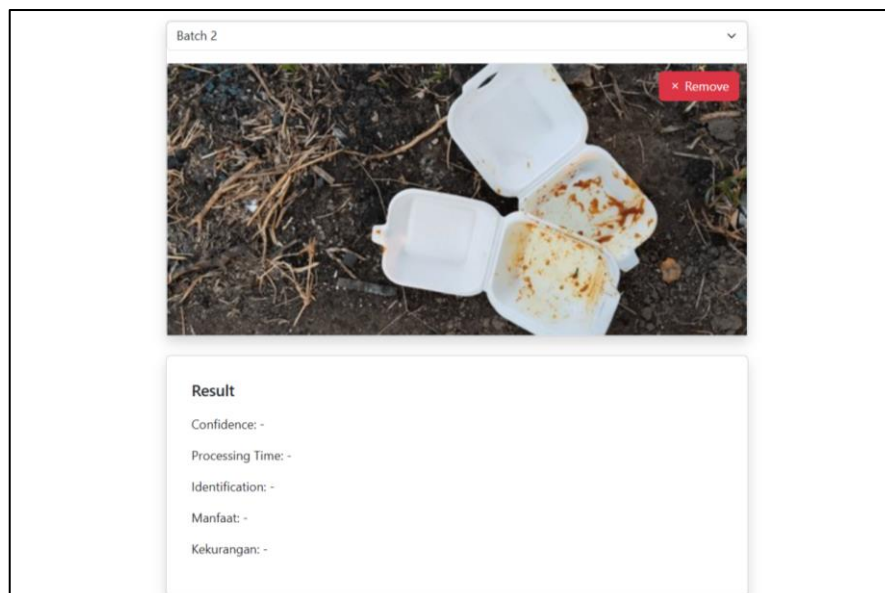
model.val, dan matriks kebingungan untuk masing-masing dataset diperoleh. Skor F1 rata-rata untuk masing-masing dataset dihitung dengan memanggil fungsi `fl_score` yang telah dijelaskan sebelumnya.

Selanjutnya, fungsi mengambil metrik performa model sebelumnya dari basis data menggunakan `db_model.get_model_by_path(last_model_path)[0]`. Forgetting rate dihitung sebagai selisih antara skor F1 baru pada dataset baseline dan skor F1 baseline model sebelumnya, yang menunjukkan sejauh mana model mengalami pelupaan terhadap data lama setelah pembelajaran baru. Learning rate dihitung sebagai selisih antara skor F1 baru pada dataset baru dan skor F1 baru model sebelumnya, merefleksikan kemampuan model untuk mempelajari informasi baru. Fungsi kemudian mengemas semua metrik ini ke dalam sebuah dictionary data dan mengembalikannya untuk digunakan dalam analisis atau penyimpanan lebih lanjut.

4.4. Hasil Implementasi Frontend

4.4.1. Antarmuka Prediksi dan Kode

Berikut ini adalah hasil implementasi antarmuka untuk pengguna pada fitur prediksi atau scan tipe sampah plastik. Gambar 4.18 merupakan tangkapan layar peramban pada fitur prediksi untuk *end-user*.



Gambar 4.18 Antarmuka prediksi untuk *end-user*

4.4.2. Antarmuka Riwayat Prediksi

Pada tampilan awal untuk memonitoring dan melakukan pelatihan model riwayat prediksi adalah tampilan utama. Antarmuka riwayat prediksi dapat dilihat pada tangkapan layar pada . Dan pada riwayat prediksi terdapat aksi untuk melihat detail dari informasi prediksi yang akan menampilkan modal detail.

Sistem Monitoring Klasifikasi Gambar Prediksi Batch Dataset Models

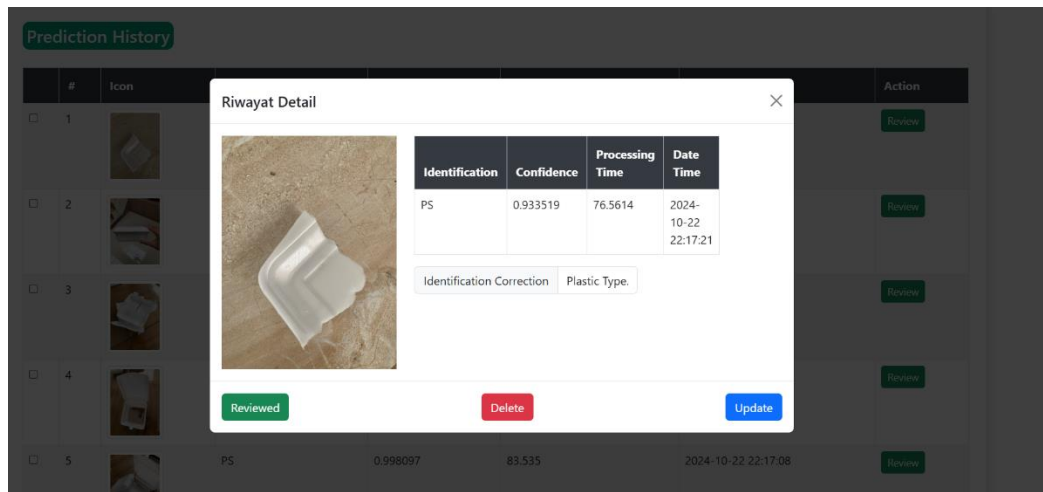
Prediction History

	#	Icon	Identification	Confidence	Processing Time	Date Time	Action
☐	1		PS	0.955519	76.5614	2024-10-22 22:17:21	Review
☐	2		PS  PS ✓	0.999753	79.551	2024-10-22 22:17:16	Review
☐	3		PS	0.999781	74.7869	2024-10-22 22:17:14	Review
☐	4		PS	0.957384	73.0135	2024-10-22 22:17:11	Review
☐	5		PS	0.998097	83.535	2024-10-22 22:17:08	Review
☐	6		PS	0.998998	97.0774	2024-10-22 22:17:06	Review
☐	7		PS	0.997092	57.9998	2024-10-22 22:17:03	Review
☐	8		PS	0.993749	73.004	2024-10-22 22:17:01	Review
☐	9		PS	0.869042	62.0041	2024-10-22 22:16:58	Review
☐	10		PS	0.937861	63.5312	2024-10-22 22:16:56	Review

1 2 3 56

Gambar 4.19 Antarmuka riwayat prediksi

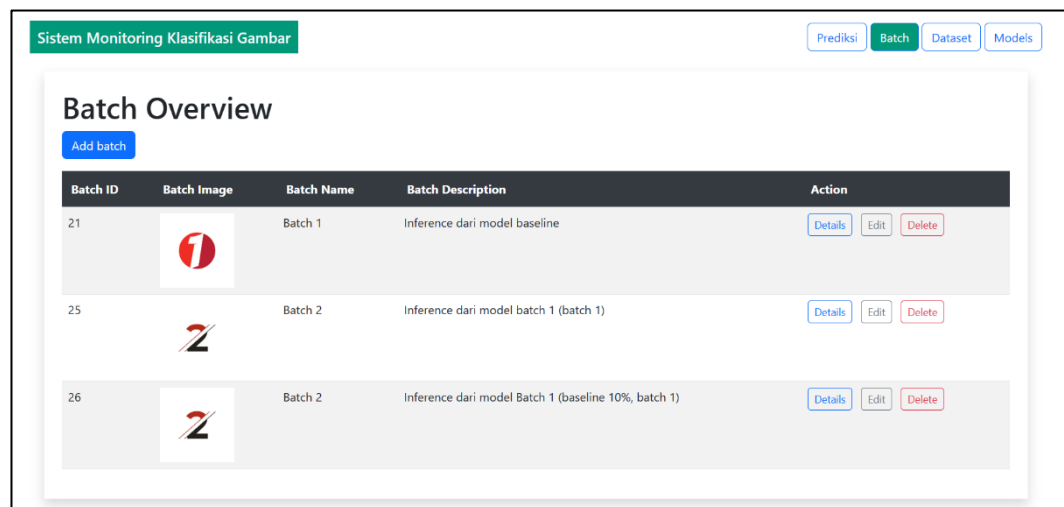
Pada detail riwayat prediksi pengguna dapat melihat gambar objek secara lebih jelas dan mengoreksi identifikasi. Gambar 4.28 berikut ini adalah tangkapan layar pada tampilan detail riwayat prediksi



Gambar 4.20 Antarmuka detail prediksi

4.4.3. Antarmuka Batch

Pada tab batch kita dapat membuat sebuah rekaptulasi untuk menampung prediksi yang telah dipilih. Terdapat aksi detail pada setiap batch untuk melihat data prediksi apa saja yang telah ditambahkan. Antarmuka batch dapat dilihat pada

Gambar 4.21 Antarmuka fitur *batch*

Berikut ini adalah tangkapan layar untuk aksi dari detail batch yang dapat dilihat pada Gambar 4.22.

Sistem Monitoring Klasifikasi Gambar

Prediksi Batch Dataset Models

Batch Detail: Batch 1

Generate Replay Dataset Generate with augmentation Generate Random Sampling

Generated Dataset Path: static/data/generated_dataset/replay_21
Generated Augmented Dataset Path: static/data/generated_dataset/augmented_batch_21

Select	Icon	Identification	New Identification	Confidence	Processing Time
☐		PP	PS	0.676008	67.0257
☐		PP	PS	0.999671	72.8085
☐		PET	PS	0.555459	55.5258
☐		PP	PS	0.999937	67.0085
☐		PP	PS	0.979275	51.5182
☐		PET	PS	0.99917	54.0216

Gambar 4.22 Antarmuka detail batch

Aksi yang dapat dilakukan pada fitur batch adalah *generate replay dataset*, *generate with augmentation*, dan *generate random sampling*.

4.4.4. Antarmuka Pelatihan

Antarmuka dari fitur pelatihan adalah tabel yang berisi metrik dari hasil pelatihan. Ada beberapa aksi dari tabel tersebut yaitu detail model untuk melakukan pelatihan dari model yang dipilih. Antarmuka dari tabel yang dimaksud dapat dilihat pada .

Sistem Monitoring Klasifikasi Gambar

Prediksi Batch Dataset Models

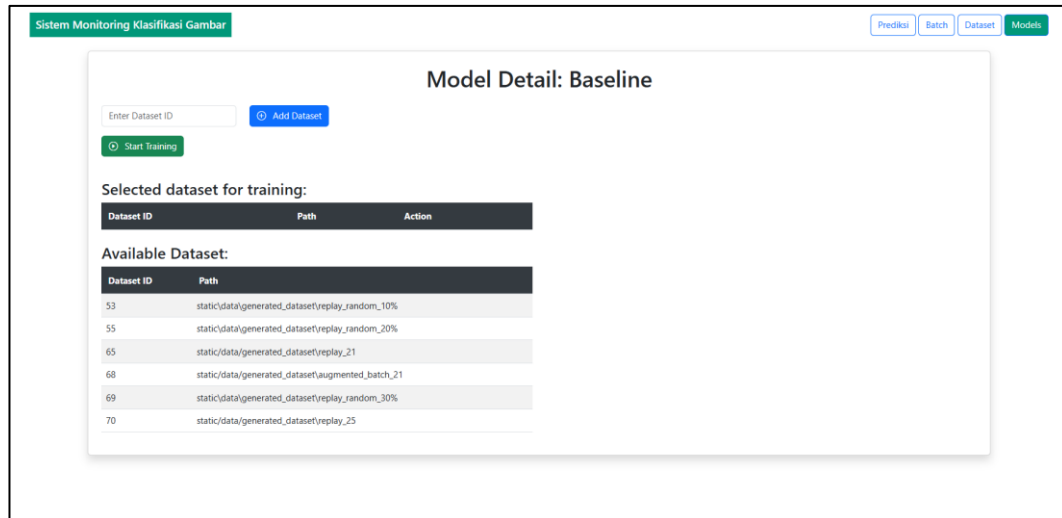
Model Overview

Model ID	Model Name	Model Description	F1 Score Baseline	F1 Score New	BWT	FWT	Action
1	Baseline	Model awal	100.0%	45.757%	0.0%	0.0%	Details Edit Delete
2	Batch 1	training dataset Batch 1	100.0%	66.301%	0.0%	21.281%	Details Edit Delete
3	Batch 1	Training dengan dataset batch 1 augmented	98.908%	74.183%	-1.092%	28.902%	Details Edit Delete
4	Batch 1-best	Training dengan dataset: batch 1, baseline 10%	100.0%	73.668%	0.0%	27.919%	Details Edit Delete
5	Batch 1	Training dengan dataset: baseline 10%, batch 1 augmented	94.195%	67.928%	-5.806%	22.618%	Details Edit Delete
6	Batch 1	Training dataset: batch 1 augmented, baseline 20%	98.556%	68.553%	-1.444%	22.795%	Details Edit Delete
7	Batch 2-a	Training dataset: batch 2	98.467%	69.585%	-1.533%	-4.082%	Details Edit Delete
8	Batch 2	Training dataset: batch 1, batch 2	96.763%	82.219%	-3.237%	8.551%	Details Edit Delete
9	Batch 2	Menggunakan dataset: baseline 10%, batch 1, batch 2	98.228%	76.723%	-1.772%	3.055%	Details Edit Delete
10	Batch 2	Training dataset: batch 1, batch 2, baseline 20%	99.288%	77.104%	-0.712%	3.437%	Details Edit Delete
11	Batch x	Pretrain baseline use batch 1, batch 2	98.467%	77.468%	-1.533%	31.711%	Details Edit Delete
12	Batch 2	Pretrain batch 1-best with: baseline 30%, batch 1, batch 2	99.288%	78.694%	-0.712%	5.026%	Details Edit Delete

Gambar 4.23 Antarmuka daftar model

Pada detail model terdapat beberapa informasi seperti daftar dataset yang sudah dibuat dan masukan id untuk memilih dataset yang akan dilakukan pelatihan.

Gambar 4.28 berikut ini adalah hasil tangkapan layar dari detail model untuk mulai pelatihan model



Gambar 4.24 Antarmuka detail model dan fitur pelatihan model

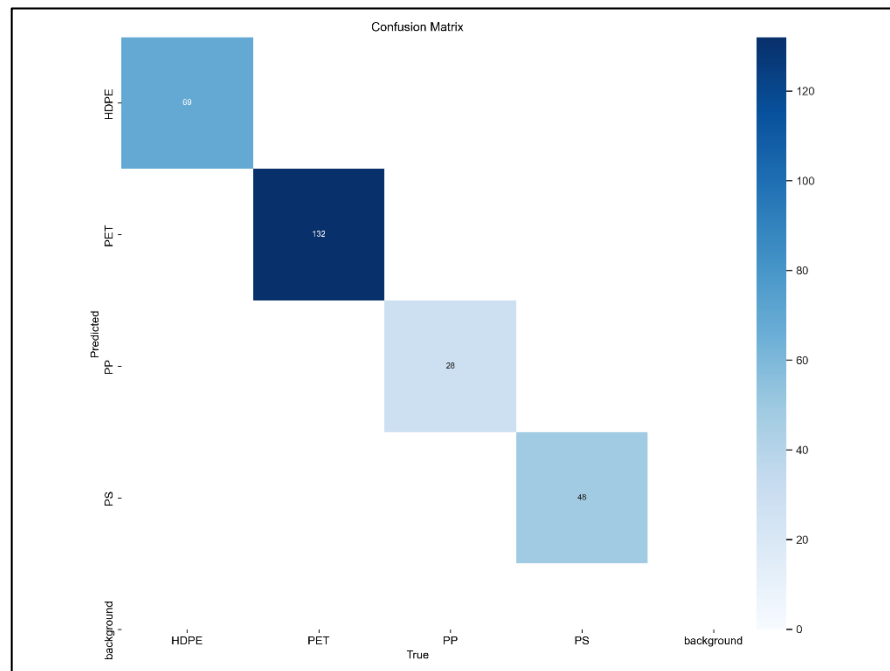
4.5. Hasil Pengujian dan Evaluasi Model

Pada subab akan menjelaskan proses pengujian, yang akan dimulai dari prediksi data atau inferensi, hasil koreksi data (*batch*), hasil pelatihan dan pengujian oleh setiap batch. Pengujian *batch* 1 dan *batch* 2 dimulai dengan *inference* pada web yang sudah dikembangkan.

4.5.1. Evaluasi Model Baseline dan Dataset Baseline

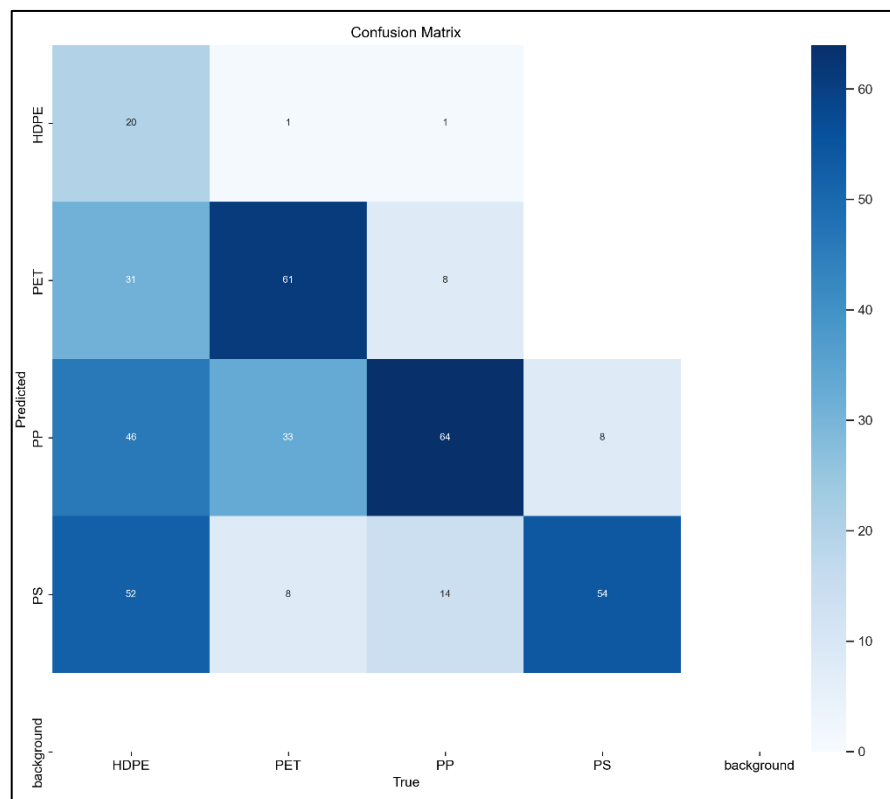
Hasil pengujian model *baseline* pada 2 dataset yang telah dirancang menghasilkan nilai dari metrik *F1-score*. Nilai evaluasi pada model *baseline* berguna untuk membandingkan hasil dari pelatihan *batch* 1. Nilai evaluasi model *baseline* dapat dilihat pada Tabel 4.1.

1. Confusion matrix dataset baseline



Gambar 4.25 *Confusion matriks* model *baseline* dengan dataset testing *baseline*

2. Confusion matrix dataset baru



Gambar 4.26 *Confusion matrix* model *baseline* dengan dataset testing baru

Maka nilai evaluasi metrics *F1-score* hasil kalkulasi *confusion matrix* dapat dilihat pada Tabel 4.1.

Tabel 4.1 Matriks evaluasi model *baseline*

	F1-Score (%)
Dataset Baseline	100
Dataset Baru	45.757

Pada penelitian ini demi mempertahankan pengetahuan sebelumnya pada dataset baseline dengan total data 11,253 akan digunakan kembali. Data tersebut digunakan untuk pengetahuan episodik. Berikut ini adalah hasil *random sampling* dari dataset pelatihan baseline yang akan digunakan sebagai *replay buffer*.

- *Random sampling* sebanyak 10%

Hasil Random Sampling:

Total Data Baseline (10%): 1.402 gambar

Pembagian Data ke dalam Set Pelatihan dan Validasi:

Set Pelatihan:

Distribusi per Kelas:

HDPE: 318 gambar

PET: 314 gambar

PP: 242 gambar

PS: 249 gambar

Set Validasi:

Distribusi per Kelas:

HDPE: 79 gambar

PET: 78 gambar

PP: 60 gambar

PS: 62 gambar

- *Random sampling* sebanyak 20%

Total Data Baseline (20%): 2.809 gambar

Pembagian Data ke dalam Set Pelatihan dan Validasi:

Set Pelatihan:

HDPE: 637 gambar

PET: 628 gambar

PP: 485 gambar

PS: 498 gambar

Set Validasi:

HDPE: 159 gambar

PET: 157 gambar

PP: 121 gambar

PS: 124 gambar

- Random sampling sebanyak 30%

Total Data Baseline (30%): 4215 gambar

Pembagian Data ke dalam Set Pelatihan dan Validasi:

Set Pelatihan:

HDPE: 956 gambar

PET: 942 gambar

PP: 728 gambar

PS: 747 gambar

Set Validasi:

HDPE: 239 gambar

PET: 235 gambar

PP: 182 gambar

PS: 186 gambar

4.5.2. Task 1

Pada penelitian ini, untuk mempermudah dan mempercepat melakukan simulasi prediksi atau inferensi. Penelitian ini menggunakan program python untuk melakukan masukan data. Gambar 4.27 merupakan kode otomasi untuk melakukan *request* ke *endpoint* prediksi sekaligus mengoreksi hasil prediksi dengan identifikasi yang benar.

```

from unittest import result
import requests
import os

# search all image in path and fetch to api url
def fetch_image_to_api(path, api_url):
    for root, dirs, files in os.walk(path):
        for file in files:
            if file.endswith(".jpg") or file.endswith(".jpeg") or file.endswith(".png"):
                image_path = os.path.join(root, file)
                print(f"Fetching image {image_path}")
                with open(image_path, "rb") as f:
                    files = {"file": (f)}
                    response = requests.post(api_url, files=files)
                    true_label = image_path.split("\\")[-2]
                    predict_label = response.json()["identification"]
                    if predict_label != true_label:
                        print("Prediction is not correct")

                        new_identification = true_label
                        data = {
                            "path": "static/data/prediction/" + image_path.split("\\")[-1],
                            "new_identification": new_identification,
                        }
                        response = requests.patch("http://localhost:5000/update_identification", json=data)

                        print(f"Update identification from {predict_label} to {new_identification}")
                    else:
                        print("Prediction is correct")

path = r"C:\Data\Software_Engineer_2024\Semester VIII (Tugas Akhir)\Data Testing Prediction\Batch 1\HDPE"
api_url = "http://localhost:5000/scan"
fetch_image_to_api(path, api_url)

```

Gambar 4.27 Automasi *inference* pada endpoint prediksi

Secara teknis, skrip dimulai dengan mengimpor modul `requests` untuk melakukan permintaan HTTP ke API, dan modul `os` untuk navigasi sistem file. Fungsi utama dalam skrip adalah `fetch_image_to_api`, yang menerima dua parameter: `path` dan `api_url`. Parameter `path` adalah string yang menunjukkan direktori tempat gambar-gambar uji berada, sementara `api_url` adalah URL endpoint API yang digunakan untuk melakukan prediksi.

Fungsi `fetch_image_to_api` menggunakan `os.walk(path)` untuk melakukan iterasi rekursif melalui semua subdirektori dan file dalam direktori yang ditentukan. Untuk setiap file yang ditemukan, skrip memeriksa apakah file tersebut adalah gambar dengan ekstensi `.jpg`, `.jpeg`, atau `.png`. Jika ya, skrip akan melakukan langkah-langkah berikut:

1. Membaca dan Mengirim Gambar ke API: Gambar dibuka dalam mode biner ("`rb`") dan dikirim ke endpoint API menggunakan `requests.post(api_url, files=files)`, di mana `files` adalah dictionary yang berisi pasangan kunci-nilai dengan kunci "file" dan nilai objek file. Ini memungkinkan API menerima file

gambar dalam format yang tepat untuk pemrosesan.

2. Mendapatkan Prediksi dari API: Respons dari API diharapkan dalam format JSON, sehingga skrip memanggil `response.json()` untuk mengonversi respons menjadi objek Python (dictionary). Label prediksi (`predict_label`) kemudian diambil dari kunci "identification" dalam dictionary tersebut.
3. Menentukan Label Asli: Label asli (`true_label`) diekstraksi dari path gambar dengan menggunakan `image_path.split("\\")[-2]`. Metode ini memecah string path berdasarkan karakter "\\" (karakter pemisah path pada sistem Windows) dan mengambil elemen kedua terakhir, yang merupakan nama folder kelas dari gambar tersebut.
4. Membandingkan Prediksi dengan Label Asli: Skrip membandingkan `predict_label` dengan `true_label`. Jika keduanya tidak sama, skrip mencetak pesan bahwa prediksi tidak benar dan menyiapkan data untuk memperbarui identifikasi.
5. Memperbarui Identifikasi melalui API: Data yang akan dikirim ke API untuk pembaruan identifikasi disusun dalam dictionary `data`, yang berisi path relatif gambar (`"static/data/prediction/" + image_path.split("\\")[-1]`) dan `new_identification` yang diatur ke `true_label`. Skrip kemudian mengirim permintaan PATCH ke endpoint `"http://localhost:5000/update_identification"` menggunakan `requests.patch`, dengan `json=data` untuk mengirim data dalam format JSON.
6. Feedback ke Pengguna: Skrip mencetak pesan yang menunjukkan bahwa identifikasi telah diperbarui dari `predict_label` ke `new_identification`. Jika prediksi awal sudah benar, skrip mencetak pesan bahwa prediksi tersebut benar.

Pada akhir skrip, variabel `path` dan `api_url` ditetapkan dengan nilai spesifik. `path` adalah direktori yang berisi gambar-gambar uji, dan `api_url` adalah endpoint API yang akan menerima gambar untuk prediksi. Fungsi `fetch_image_to_api` kemudian dipanggil dengan parameter tersebut untuk memulai proses evaluasi.

Selanjutnya adalah merekap data menjadi *batch* 1, program akan merekap data yang memiliki status `is_correct = 1`, program dapat dilihat pada Gambar 4.28.

```

from urllib import response
import requests

response = requests.get("http://localhost:5000/prediction")

all_predictions = response.json()

selectedPredictions = []
for prediction in all_predictions:
    if prediction["is_correct"] == False:
        selectedPredictions.append(prediction["id"])

data = {
    "checkedPredictions": selectedPredictions,
    "selectedBatch": 21
}

add_predictions_to_batch = requests.post("http://localhost:5000/monitoring", json=data)

print("success")

```

Gambar 4.28 Program untuk merekap data prediksi

Pertama, skrip mengirim permintaan GET ke endpoint "http://localhost:5000/prediction" untuk mengambil semua data prediksi yang ada di server. Respons dari permintaan ini diuraikan dalam format JSON dan disimpan dalam variabel `all_predictions`.

Selanjutnya, skrip menginisialisasi daftar kosong bernama `selectedPredictions` yang akan menampung ID dari prediksi-prediksi tertentu yang memenuhi kriteria spesifik. Skrip melakukan iterasi melalui setiap item dalam `all_predictions`, dan untuk setiap prediksi, skrip memeriksa kondisi "Apakah prediksi tersebut tidak benar (`"is_correct" == False`)".

Jika kondisi tersebut terpenuhi, ID prediksi tersebut ditambahkan ke dalam daftar `selectedPredictions`.

1. Setelah proses penyaringan selesai, skrip menyiapkan payload data dalam bentuk dictionary `data`, yang berisi:
2. `"checkedPredictions"`: Daftar ID prediksi yang telah dipilih dan memenuhi kriteria.
3. `"selectedBatch"`: Nilai integer 21, yang kemungkinan besar merupakan identifikasi batch atau kelompok tertentu untuk keperluan monitoring atau

analisis lebih lanjut.

Skrip kemudian mengirim permintaan POST ke endpoint "http://localhost:5000/monitoring" dengan menyertakan data sebagai payload dalam format JSON. Tindakan ini bertujuan untuk memperbarui server dengan menambahkan prediksi yang telah dipilih ke dalam batch monitoring yang ditentukan, sehingga dapat digunakan untuk proses seperti analisis kesalahan atau retraining model.

Setelah menjalankan dua program tersebut, kita memperoleh gambaran menyeluruh mengenai performa model terhadap Batch 1. Berikut ini adalah rekapitulasi data awal dan hasil setelah inferensi serta koreksi.

Task 1 terdiri dari 202 data dengan distribusi sebagai berikut:

1. HDPE : 49 gambar
2. PET : 52 gambar
3. PP : 60 gambar
4. PS : 41 gambar

Distribusi ini menunjukkan jumlah gambar per kelas dalam Batch 1 sebelum dilakukan proses inferensi.

- Hasil Inferensi dan Rekap Data *Task 1* yang Dikoreksi

Setelah model melakukan prediksi pada gambar-gambar dalam *Task 1*, terdapat 69 gambar yang prediksinya tidak sesuai dengan label asli dan memerlukan koreksi.

Total Prediksi yang Dikoreksi: 96 gambar

HDPE: 43 gambar dikoreksi

PET: 26 gambar dikoreksi

PP: 11 gambar dikoreksi

PS: 16 gambar dikoreksi

Berikut adalah hasil dari data yang telah didistribusi dengan ratio 90:10.

Tabel 4.2 Distribusi dataset *batch 1*

Class	Training (90%)	Validation (10%)
-------	----------------	------------------

HDPE	39	4
PET	23	3
PP	10	1
PS	14	2

1. Hasil pelatihan ulang model baseline dengan data *batch* 1

Hasil eksperimen pelatihan dengan dataset *batch* 1 yang dikurangi dengan nilai pengujian performa model *baseline* dapat dilihat pada Tabel 4.3. Pada kolom

Tabel 4.3 Hasil eksperimen pelatihan *batch* 1

No.	Metode	Penggunaan Dataset	F1-Score Dataset Baseline	F1-Score dataset Baru	Backward Transfer	Forward Transfer
		Baseline	100%	45.757%	0	0
1	<i>Transfer learning (Fine tune)</i>	Batch 1	100%	66.301%	0.0%	21.281%
2	<i>Transfer learning (fine tune)</i>	Batch 1 Augmented	98.908%	74.183%	-1.092%	28.902%
3	<i>Continual learning (Replay)</i>	Baseline 10%, Batch 1 Augmented	94.195%	67.928%	-5.806%	22.618%
4	<i>Continual learning (Replay)</i>	Batch 1 Augmented, Baseline 20%	98.556%	68.553%	-1.444%	22.795%
5	<i>Continual learning</i>	Baseline 10%, Batch 1	100.0%	73.668%	0.0%	27.919%

	(Replay)					
--	----------	--	--	--	--	--

Hasil pengujian menunjukkan bahwa pelatihan ke 1 tidak mengalami penurunan performa walaupun tidak menggunakan metode replay. Hal tersebut bisa terjadi dimungkinkan *batch* 1 tidak memiliki data yang banyak, sehingga *weight* pada model tidak berubah secara signifikan pada pengetahuan sebelumnya. Dan hasil pada pelatihan ke 1 menunjukkan bahwa performa model untuk mempelajari objek baru meningkat dengan selisih 21,281% dari *f1-score* dari model baseline. Bisa dikatakan model YOLOv8-clas dapat dilatih dengan data yang sedikit untuk mempelajari pengetahuan baru.

Hasil pelatihan dan pengujian ke 5 bisa dikatakan lebih baik. Pelatihan pertama menggunakan metode replay menggunakan dataset baseline 10% hasil *random sampling* dan menggunakan data *batch* 1 untuk pelatihan. Model tidak mengalami penurunan, dan performa model pada dataset baru lebih sebesar 27,919% lebih unggul dibanding pelatihan pertama.

4.5.3. Task 2

Sama seperti *Task* 1 menggunakan 2 program otomatisasi untuk melakukan *inference*. *Inference* dilakukan pada model terbaik pada *batch* 1 yaitu pelatihan ke 5 yang menggunakan dataset baseline 10% dan *batch* 1. Berikut ini adalah rekapitulasi data awal dan hasil setelah inferensi.

Task 2 memiliki 208 data dengan distribusi:

1. HDPE : 81 gambar
2. PET : 20 gambar
3. PP : 41 gambar
4. PS : 66 gambar.

Setelah model melakukan prediksi pada gambar-gambar dalam *Task* 2, terdapat 96 gambar yang prediksinya tidak sesuai dengan label asli dan memerlukan koreksi.

Total Prediksi yang Dikoreksi: 96 gambar

1. HDPE: 14 gambar dikoreksi
2. PET: 17 gambar dikoreksi

3. PP: 36 gambar dikoreksi
4. PS: 14 gambar dikoreksi

Berikut adalah hasil dari data yang telah didistribusi dengan ratio 90:10.

Tabel 4.4 Distribusi dataset *batch 2*

Class	<i>Training</i> (90%)	<i>Validation</i> (10%)
HDPE	13	1
PET	15	2
PP	32	2
PS	13	1

1. Hasil pelatihan ulang model *batch 1* dengan data *batch 2*

Hasil eksperimen pelatihan dengan dataset *batch 2* yang dikurangi dengan nilai pengujian performa model *batch 1* terbaik dapat dilihat pada Tabel 4.5.

Tabel 4.5 Seluruh eksperimen pada pelatihan *batch 2*

No	Metode	Penggunaan Dataset	F1-Score Dataset Baseline	F1-Score Dataset Baru	Backward Transfer	Forward Transfer
		Baseline 10%, Batch 1	100.0%	73.668%	0.0%	27.919%
1	<i>Transfer learning (Fine Tune)</i>	Batch 2	98.467%	69.585%	-1.533%	-4.082%
2	<i>Continual learning (Replay)</i>	Batch 1, Batch 2	96.763%	82.219%	-3.237%	8.551%
3	<i>Continual</i>	Baseline	98.228%	76.723%	-1.772%	3.055%

	<i>learning</i> (<i>Replay</i>)	10%, Batch 1, Batch 2				
4	<i>Continual</i> <i>learning</i> (<i>Replay</i>)	Baseline 20%, Batch 1, Batch 2	99.288%	77.104%	-0.712%	3.437%
5	<i>Continual</i> <i>learning</i> (<i>Replay</i>)	Baseline 30%, Batch 1, Batch 2	99.288%	78.694%	-0.712%	5.026%

Dari seluruh hasil pelatihan *batch* 2, terdapat 2 hasil terbaik. Pada hasil pelatihan 2 menggunakan dataset *batch* 2 dan *replay buffer batch* 1 dengan melakukan *re-train* model terbaik *batch* 1, menghasilkan nilai F1-score terbaik sebesar 82.219%. Tetapi memiliki nilai *forgetting* terbesar dari semua eksperimen sebesar -3.237%. Dapat disimpulkan bahwa model berfokus pada pengetahuan baru yang mana lebih bisa memprediksi objek baru nyata tetapi model kehilangan pengetahuan sebelumnya. Pada pelatihan ke-5 model lebih dapat mempertahankan pengetahuan sebelumnya dengan nilai sebesar -0,712% dikurangi. Tetapi model tetap dapat meningkatkan pengetahuannya dengan selisih 5.026% dari model terbaik *batch* 1.

BAB V
KESIMPULAN DAN SARAN

- 5.1 Kesimpulan
- 5.2 Saran

DAFTAR PUSTAKA

- [1] “Daerah DIY - Pengelolaan Sampah.” Accessed: Sep. 25, 2024. [Online]. Available: https://bappeda.jogjaprovo.go.id/dataku/data_dasar/cetak/208-pengelolaan-sampah?id_skpd=91
- [2] “SIPSN - Sistem Informasi Pengelolaan Sampah Nasional.” Accessed: Sep. 25, 2024. [Online]. Available: <https://sipsn.menlhk.go.id/sipsn/public/data/komposisi>
- [3] J. Bobulski and M. Kubanek, “Deep Learning for Plastic Waste Classification System,” *Appl. Comput. Intell. Soft Comput.*, vol. 2021, pp. 1–7, May 2021, doi: 10.1155/2021/6626948.
- [4] H. C. Chandara, S. -, and S. -, “PLASTIC RECYCLING IN INDONESIA BY CONVERTING PLASTIC WASTES (PET, HDPE, LDPE, and PP) INTO PLASTIC PELLETS,” *ASEAN J. Syst. Eng.*, vol. 3, no. 2, p. 65, Dec. 2016, doi: 10.22146/ajse.v3i2.17162.
- [5] K. Kumar Jha and T. T. M. Kannan, “Recycling of plastic waste into fuel by pyrolysis - a review,” *Mater. Today Proc.*, vol. 37, pp. 3718–3720, 2021, doi: 10.1016/j.matpr.2020.10.181.
- [6] O. Tamin, E. G. Mounq, J. Ahmad Dargham, F. Yahya, S. Omatu, and L. Angeline, “Machine Learning for Plastic Waste Detection: State-of-the-art, Challenges, and Solutions,” in *2022 International Conference on Communications, Information, Electronic and Energy Systems (CIEES)*, Veliko Tarnovo, Bulgaria: IEEE, Nov. 2022, pp. 1–6. doi: 10.1109/CIEES55704.2022.9990703.
- [7] G. Jocher, “Is there any solution for Catastrophic Inference with YOLOv8 yet? · Issue #4282 · ultralytics/ultralytics,” GitHub. Accessed: Aug. 13, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics/issues/4282>
- [8] B. Wickramasinghe, G. Saha, and K. Roy, “Continual Learning: A Review of Techniques, Challenges, and Future Directions,” *IEEE Trans. Artif. Intell.*, vol. 5, no. 6, pp. 2526–2546, Jun. 2024, doi: 10.1109/TAI.2023.3339091.
- [9] G. Merlin, V. Lomonaco, A. Cossu, A. Carta, and D. Bacciu, “Practical

- Recommendations for Replay-based Continual Learning Methods,” vol. 13374, 2022, pp. 548–559. doi: 10.1007/978-3-031-13324-4_47.
- [10] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “iCaRL: Incremental Classifier and Representation Learning,” Apr. 14, 2017, *arXiv*: arXiv:1611.07725. Accessed: Aug. 19, 2024. [Online]. Available: <http://arxiv.org/abs/1611.07725>
- [11] A. Chaudhry *et al.*, “On Tiny Episodic Memories in Continual Learning,” Jun. 04, 2019, *arXiv*: arXiv:1902.10486. Accessed: Sep. 03, 2024. [Online]. Available: <http://arxiv.org/abs/1902.10486>
- [12] M. Yang, U. Cappellazzo, X. Li, and B. Raj, “Improving Continual Learning of Acoustic Scene Classification via Mutual Information Optimization,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Seoul, Korea, Republic of: IEEE, Apr. 2024, pp. 7105–7109. doi: 10.1109/ICASSP48485.2024.10446846.
- [13] J. Choi, B. Lim, and Y. Yoo, “Advancing Plastic Waste Classification and Recycling Efficiency: Integrating Image Sensors and Deep Learning Algorithms,” *Appl. Sci.*, vol. 13, no. 18, p. 10224, Sep. 2023, doi: 10.3390/app131810224.
- [14] D. Ziouzos, N. Baras, V. Balafas, M. Dasygenis, and A. Stimoniaris, “Intelligent and Real-Time Detection and Classification Algorithm for Recycled Materials Using Convolutional Neural Networks,” *Recycling*, vol. 7, no. 1, p. 9, Feb. 2022, doi: 10.3390/recycling7010009.
- [15] C. Zhuang, S. Huang, G. Cheng, and J. Ning, “Multi-criteria Selection of Rehearsal Samples for Continual Learning,” *Pattern Recognit.*, vol. 132, p. 108907, Dec. 2022, doi: 10.1016/j.patcog.2022.108907.
- [16] G. Graffieti, G. Borghi, and D. Maltoni, “Continual Learning in Real-Life Applications,” *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 6195–6202, Jul. 2022, doi: 10.1109/LRA.2022.3167736.
- [17] Y. K. Viswanadham, R. S. Rani, V. M. S. L. S. Mai, V. N. V. Krishna, and Y. V. Vamsi, “An Identification and Categorization of Plastic Material using Deep Learning Approach,” in *2023 3rd International Conference on Pervasive*

- Computing and Social Networking (ICPCSN)*, Salem, India: IEEE, Jun. 2023, pp. 289–294. doi: 10.1109/ICPCSN58827.2023.00054.
- [18] I. W. Raka Ardana, I. Bagus Irawan Purnama, and I. M. Sumerta Yasa, “Application of Object Recognition for Plastic Waste Detection and Classification Using YOLOv3,” in *2020 International Conference on Applied Science and Technology (iCAST)*, Padang, Indonesia: IEEE, Oct. 2020, pp. 652–656. doi: 10.1109/iCAST51016.2020.9557735.
- [19] S. D. Burrows *et al.*, “The message on the bottle: Rethinking plastic labelling to better encourage sustainable use,” *Environ. Sci. Policy*, vol. 132, pp. 109–118, Jun. 2022, doi: 10.1016/j.envsci.2022.02.015.
- [20] T. M. Mitchell, *Machine learning*, Nachdr. in McGraw-Hill series in Computer Science. New York: McGraw-Hill, 2013.
- [21] C. M. Bishop, *Pattern Recognition and Machine Learning*, Softcover reprint of the original 1st edition 2006 (corrected at 8th printing 2009). in Information science and statistics. New York, NY: Springer New York, 2016.
- [22] E. Alpaydin, *Introduction to machine learning*, Fourth edition. in Adaptive computation and machine learning series. Cambridge, Massachusetts: The MIT Press, 2020.
- [23] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. in Adaptive computation and machine learning. Cambridge, Mass: The MIT press, 2016.
- [24] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: 10.1038/nature14539.
- [25] M. I. Jordan and T. M. Mitchell, “Machine learning: Trends, perspectives, and prospects,” *Science*, vol. 349, no. 6245, pp. 255–260, Jul. 2015, doi: 10.1126/science.aaa8415.
- [26] G. Brod, M. Werkle-Bergner, and Y. L. Shing, “The Influence of Prior Knowledge on Memory: A Developmental Cognitive Neuroscience Perspective,” *Front. Behav. Neurosci.*, vol. 7, 2013, doi: 10.3389/fnbeh.2013.00139.
- [27] G. M. Van De Ven, T. Tuytelaars, and A. S. Tolias, “Three types of incremental learning,” *Nat. Mach. Intell.*, vol. 4, no. 12, pp. 1185–1197, Dec. 2022, doi:

- 10.1038/s42256-022-00568-3.
- [28] Z. Chen and B. Liu, *Lifelong Machine Learning*. in Synthesis Lectures on Artificial Intelligence and Machine Learning. Cham: Springer International Publishing, 2018. doi: 10.1007/978-3-031-01581-6.
 - [29] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” 2015, *arXiv*. doi: 10.48550/ARXIV.1506.02640.
 - [30] “A Review on YOLOv8 and Its Advancements,” in *Algorithms for Intelligent Systems*, Singapore: Springer Nature Singapore, 2024, pp. 529–545. doi: 10.1007/978-981-99-7962-2_39.
 - [31] D. K. Barozai, “Top 8 Machine Learning Image Classification Datasets,” Folio3AI Blog. Accessed: Sep. 15, 2024. [Online]. Available: <https://www.folio3.ai/blog/ml-image-classification-datasets/>
 - [32] “WordNet,” WordNet. Accessed: Sep. 15, 2024. [Online]. Available: <https://wordnet.princeton.edu/homepage>
 - [33] A. Rahaman, V. Gayatri, C. S. Kiran, K. S. Pavan, B. Bhumika, and V. Sateesh, “Development of Web Applications by Integrating Frontend and Backend Tools,” vol. 12, no. 01, 2022.
 - [34] K. Olson, “What Are Data?,” *Qual. Health Res.*, vol. 31, no. 9, pp. 1567–1569, Jul. 2021, doi: 10.1177/10497323211015960.
 - [35] D. Winata, “SYSTEM DEVELOPMENT AND DATABASE SECURITY,” Feb. 10, 2019. doi: 10.31219/osf.io/4573w.
 - [36] N. W. S. Wardhani, M. Y. Rochayani, A. Iriany, A. D. Sulistyono, and P. Lestantyo, “Cross-validation Metrics for Evaluating Classification Performance on Imbalanced Data,” in *2019 International Conference on Computer, Control, Informatics and its Applications (IC3INA)*, Tangerang, Indonesia: IEEE, Oct. 2019, pp. 14–18. doi: 10.1109/IC3INA48034.2019.8949568.
 - [37] International Software Testing Qualifications Board (ISTQB), “Standard Glossary of Terms Used in Software Testing.” ISTQB, 2018.
 - [38] *IEEE Standard for Software and System Test Documentation*. doi:

10.1109/IEEESTD.2008.4578383.

- [39] J. Bobulski and J. Piatkowski, “PET waste clasification method and Plastic Waste DataBase WaDaBa”.
- [40] S. J. Pan and Q. Yang, “A Survey on Transfer Learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010, doi: 10.1109/TKDE.2009.191.